

01. Manual Testing Workflow

1. Tester Responsibilities (*)

1. Defining Test Scenarios Workflow (*)

1. How to generate test scenarios with AI - UiPath Test Manager (*)

- **Steps to follow in Test Manager**
- **Prompts in Test Manager**
- **AI-Generated Test Scenarios Evaluation Guide**
- **Saving test scenarios in Test Manager and sending them to JIRA**
- **Steps to follow in Jira**

2. Creating tests in Xray (*)

- **Archiving LXs (*)**
- **Call Test (*)**

1. Test Scenarios: rules and structure (*)

- 1. Positive LXs (*)**
- 2. Negative LXs (*)**
- 3. Regression LXs (*)**
- 4. UI\UX LXs (*)**
- 5. Smoke LXs (*)**
- 6. Non-functional LXs (*)**

4. LX Priorities (*)

5. Validating LXs\Test Scenarios (*)

- 1. What does validating LXs\test scenarios mean? (*)**
- 2. Validating LXs\test scenarios workflow (*)**
- 3. To whom are the scenarios sent for validation (*)**

6. Create Defining Scenarios sub-task (*)

7. Adding LX to the Defining Scenarios Subtask (*)

2. Manual Test Scenarios Definition Workflow (*)

2. Executing Test Scenarios Workflow (*)

1. Create Sub test execution and Transition (*)

1. Adding LXs to Sub Test Execution (*)

2. UAT Testing (*)

3. Smoke Testing (*)

4. **PRL (Prelive) Testing (*)**
5. **Regression Testing (*)**
3. **Bugs Workflow (*)**
 1. **What is a Bug (*)**
 1. **Types of bugs / "Category" Field (*)**
 2. **Examples of bugs (*)**
 3. **Reporting a Bug in Jira (*)**
 - **Bug reporting during the execution of an LX (*)**
 - **Bug reporting at Scrum/Jira Project level (-)**
 4. **Bug Categories (*)**
 - **Performance Bug (*)**
 - **Improvement Bug (-)**
 - **Legacy Bug (*)**
 - **Regression Bug (*)**
 - **Smoke Bug (*)**
 - **Incomplete specifications Bug (-)**
 - **Integrated Testing Bug**
 - **UI/UX Bug (*)**
 4. **Retesting a bug fix (*)**
 5. **Jira Bugs Workflow (*)**
4. **Incidents Workflow (*)**
 1. **What is an Incident? (*)**
 1. **Defining Test Scenarios for Incidents (*)**
 2. **Executing Test Scenarios for Incidents (*)**
 3. **Jira Incidents Workflow (-)**
 - **Reporting an Incident (*)**
5. **Working with Xray Library (Xray Test Repository) (*)**
 - **Creating folder/subfolders in Scrum projects**
 - **Rules\Structure for the Xray Library (*)**
 - **Flows/functionalities/sections subfolders**
 - **Smoke Tests Subfolder**
 - **Regression tests subfolders**

- **Test Manager Drafts Subfolder**
- **Archived tests subfolders**
- **Filtering LXs in Xray**

6. Regression Testing Workflow (*)

1. Smoke Tests Workflow (*)

- 1. What are Smoke Tests (*)**
- 2. Defining Smoke tests (*)**
 - 1. How to create Smoke tests in Jira\Xray (*)**
- 3. Smoke Tests Execution (*)**
- 4. Smoke Tests Bug Reporting (*)**

2. Regression Tests Workflow (*)

- 1. What are Regression Tests? (*)**
- 2. Defining Regression Test Scenarios (-)**
 - 1. Initial Elicitation of Business Flows from the Product Owner (-)**
 - 2. Evaluating existing tests for Regression (*)**
 - 3. How to create regression tests in Jira/Xray (*)**
 - 4. Regression tests priority (*)**
 - 5. Orientation guide for regression test priorities (*)**
- 3. Regression Tests for new implementations (*)**

4. Regression Tests Execution (*)

- **Guide for the Tester - Best practices for Regression Test Cases (-)**
- **Scenarios validation (optional for regression tests) (-)**

7. Agile\SCRUM Workflow (*)

1. What is Agile\SCRUM?

- 1. What is a User Story?**
- 2. Ceremonies in Agile\SCRUM**
- 3. Components of a User Story**
- 4. The role of a Tester in Agile\SCRUM**

1. Defining Test Scenarios workflow in Agile\SCRUM

2. Executing Test Scenarios workflow in Agile\SCRUM

3. Bug Reporting in Agile\SCRUM

4. DEMO Presentation in Agile\SCRUM

- **DEMO Checklist**
- **DEMO Preview**

1. Flow validation - DEMO Preview e-mail

2. Flow validation - DEMO Preview meeting

- **Best practices for DEMO**
- **Standard structure for DEMO presentation**
- **Definition of Done - informative**
- **How do we hold a demo?**

- **Estimates in Agile\SCRUM (Story Points Estimation)**
- **Meetings within Agile/Scrum teams**

8. Waterfall Workflow (*)

1. What is Waterfall? (*)

1. What is a Task in Waterfall? (*)

2. Testing Process in Waterfall (*)

3. The role of a tester in Waterfall projects (*)

1. Defining test scenarios in Waterfall (*)

2. Executing Test Scenarios in Waterfall (*)

3. Bug Reporting in Waterfall (*)

4. Waterfall Estimates - Projects\ICDs (*)

9. Mobile Testing Workflow (-)

1. Defining Mobile Testing Test Scenarios (-)

2. Executing Mobile Testing Scenarios (-)

3. Mobile Testing Bugs Reporting (-)

- **ApowerMirror**
- **Smartwatch testing (-)**
- **UI\UX Testing for Mobile (-)**
 - **UI\UX Mobile Testing Orientation Guide\Checklist (-)**

10. Logging time in Jira (*)

- **Logging in Jira tasks outside the team\project**

11. Scrum teams back-up system workflow (*)

1. Allocations primary team vs secondary team

12. Integrated Testing Workflow

- 1. Create Integrated testing issue type**
- 2. Create Xray sub-folder**
- 3. Defining scenarios**
- 4. UAT Integrated testing**
- 5. PRL Integrated testing**
- 6. Integrated Testing Bug Reporting**

13. Testing Technical Tasks Workflow

- 1. Exploratory Testing Task (-)**
- 2. Update Regression\Smoke Tests Suite Task**
- 3. Execute Regression Tests on area Task**
- 4. Business Flows Testing Documentation Task**
- 5. Analysis Task**
- 6. Testing processes improvement Task**
- 7. Automation Testing activities Task**

- **02. Guides for specific testing types**

-

3. Guide\Workflow using AI

- 1. Creating test cases using AI**
 - **Chrome DevTools Recorder**
 - **UiPath Test Manager**
- 2. How to split LX type tests into smaller Test Cases using AI**

- **Planview\Tasktop Configuration**
- **Bug Generation from Google Chrome Developer Tools - Network using AI**
- **Linking Test Manager drafts to Xray folders**
- **UiPath Test Manager Workflow - Creatio Client Opening Pilot**
- **UiPath Test Manager Workflow - Digital team Pilot**
- **Useful Prompts**
- **Proposal for Manual-Automation Testing workflow in teams**

- **API Testing**
- **Windows Game Bar screen\video capture**
- **Database Testing**

- **Business Testing**
- **03. Test Manager and Jira integration from Plan View**
 - 1. **Test Manager-Uipath**
 - 2. **Jira Integration from Plan View**
- **Automation Testing**
- **Useful Information**

01. Manual Testing Workflow

(*) - Everything marked with this symbol is validated and applied as a working procedure.

(-) - Everything marked with this symbol is currently being drafted and is not an applied working procedure.

001. Tester Responsibilities (*)

- Complies with the working procedure described in 01. Manual Testing Workflow - QA - Confluence.
- Analyzes and understands the functional requirements and specifications;
- Tests the incidents for which they are the Reporter or which are assigned to them by the Product Owner of the team they are part of, according to 04. Incidents Workflow (*).
- Tests the User Stories assigned within the team they are part of in both the UAT test environment and Prelive, according to 07. Agile\SCRUM Workflow () and 11. *Scrum teams back-up system workflow* ().
- Validates the builds that have reached the Prelive environment by executing Smoke tests at the project\Epic level 01. Smoke Tests Workflow (*).
- Ensures the quality of the tested applications by executing regression tests in the Prelive environment at the project\Epic level 02. Regression Tests Workflow (*).
- Participates in specific meetings: Daily Scrum, Sprint Planning, Sprint Review, Sprint Retrospective, and analysis meetings according to 07. Agile\SCRUM Workflow (*).
- Updates the statuses for the tasks/sub-tasks/User Stories assigned to them according to 01. Defining Test Scenarios Workflow (), 02. *Executing Test Scenarios Workflow* (), 03. Bugs Workflow (), 04. *Incidents Workflow* (), 06. Regression Testing Workflow (*).
- Requests clarifications from the Business Analyst when necessary to know that they are validating the agreed behavior;
- Defines test scenarios in accordance with the specifications mentioned on the user stories and the working procedure from Confluence, populating and updating the test scenario library. Whenever specifications change, or during testing, updates the test scenarios according to 01. Defining Test Scenarios Workflow (*).
- Transmits the test scenarios for validation to the Product Owner, the Application Owner, and the Project Beneficiary according to 05. Validating LXs\Test Scenarios (*).
- Reports time daily in Jira according to 10. Logging time in Jira (*).
- Reports in Jira the defects resulting from the testing performed and re-tests the fixes according to 03. Bugs Workflow (*).
- Holds the Demo meeting for presenting the testing results to the Product Owner, the Application Owner, the Project Beneficiary, the Marketing division, and the designer, on the projects where they were involved in order to obtain acceptance according to 04. DEMO Presentation in Agile\SCRUM.
- Provides detailed estimates for the testing effort according to Estimates in Agile\SCRUM (Story Points Estimation) and 04. Waterfall Estimates - Projects\ICDs (*).
- Understands the application they are testing from the end-user's perspective.
- Has the responsibility to understand the KPIs, to reach the KPI targets monthly.

- Has the responsibility to understand the Monthly Productivity Report and to expose any issue found in the report.
- Escalates any issue to their Team Lead: announces blockages on testing activities or any other situation that arises that can delay the completion of the tested project. Sends an email to TLTesting@librabank.ro, and escalates any element that appears during the Testing activity and affects the deadline.
- Has the responsibility to ensure activity in Terramind.
- Has the responsibility to ensure they always have testing activity in the team they are allocated to, but also in their back-up team, according to 00. Scrum teams back-up system workflow - QA - Confluence.

01. Defining Test Scenarios Workflow (*)

This page documents the working procedure regarding the definition of test scenarios for applications and functionalities developed within SCRUM or Waterfall teams.

The purpose is to ensure consistency, traceability, and efficiency in creating, reviewing, and maintaining test scenarios, whether they are generated manually or with the help of AI tools.

Objectives

- Aligning the team on how to create and document test scenarios.
- Reducing the time required to define tests by using AI for automatic generation.
- Increasing functional coverage and testing quality through clear, complete, and easily updatable scenarios.

The definition of test scenarios is done using AI, described here: 01. A AI test scenario generation workflow - UiPath (draft) - QA - Confluence.

If for any reason UiPath Test Manager cannot be used, the Team Leader must be notified, and then the test scenarios must be defined manually, described here: 01. B Manual Test Scenarios Definition Workflow (*) - QA - Confluence.

01. How to generate test scenarios with AI - UiPath Test Manager (*)

Accessing the Scrum team's project from the UiPath Test Manager module

Can be done in two ways:

- from Jira
- by accessing the UiPath application URL

1. From Jira, under the Links and others tab on the User Story for which you want to generate scenarios, access the link in the Test Manager URL field

- authentication is always done via the email option
- log in with one of the email addresses and passwords below:

No. User	Email/User	Password
1	laura.mitroi@librabank.ro	11Bra#QA-tes!
2	ciprian.oros@librabank.ro	11Bra#QA-tes!
3	alina.creata@librabank.ro	11Bra#QA-tes!
4	raluca.tabarcea@librabank.ro	11Bra#QA-tes!
5	mircea.naghi@librabank.ro	11Bra#QA-tes!
6	elena.cretu@librabank.ro	11Bra#QA-tes!
7	ioana.mercan@librabank.ro	11Bra#QA-tes!
8	isabela.calin@librabank.ro	11Bra#QA-tes!
9	andreea.popaluca@librabank.ro	11Bra#QA-tes!
10	elena.adar@librabank.ro	11Bra#QA-tes!

- the main page of the User Story in Test Manager is displayed

2. Access the application URL, <https://cloud.uipath.com/librabzqkvzf/portal /home>

- follow the steps described above regarding authentication using only the email option and using one of the credentials
- access Test Manager

- from the All projects tab, search and/or directly access the Scrum where you want to search/create test scenarios
- access the Requirements section, and to view/search for a requirement (User Story), click on the Key related to the requirement or enter the requirement name (a part of the name) in the Search field

To continue with generating test scenarios for a requirement, follow the steps here.

Steps to follow in Test Manager

- From the main page of the requirement in Test Manager, access the **+Optimize coverage** menu and select the **Generate tests** option
- **IMPORTANT!** In the **Provide additional context** window that opens, the user must ensure that all relevant documents for creating the test scenarios displayed on the page are selected by checking the corresponding checkbox; also, attachments from the Jira US as well as any other documents considered relevant for the user can be added here using the **Add a file** option
- Click **Next**
- In the **Provide additional guidance** window, select the prompt based on which specific sets of test scenarios will be generated; details related to the prompts available in Test Manager can be found here.

Important! After selecting one of the prompts, the user can edit the prompt directly in the open window, modifying it/adding additional context/instructions considered relevant.

- It is recommended not to modify the default selected model - Claude
 - Click the **Generate tests** button to generate the test scenarios
 - After generation, the user performs an initial check/evaluation of each test scenario and selects the corresponding checkbox only for the scenarios considered relevant, usable, and having the highest possible degree of coverage of the necessary steps. The initial evaluation of the test scenarios generated with Test Manager involves going through the stages provided here.
-

Prompts in Test Manager

The complete library of available prompts, with the corresponding details, can be found in the main menu of Test Manager, by selecting **Project Settings** and then the **Prompt library** tab.

The parameterizable prompts created by the Manual Testing Dept. are:

Positive Scenario Test Creator:

Generate up to 10 positive test cases in Romanian:

- 1 happy path (end-to-end) and 9 secondary flows (end-to-end)
- Mark title with "scenariu pozitiv"
- Add "pozitiv" to Labels; also add "happy-path" for the happy path case
- No duplicates
- Max 50 steps per case
- Each step = a user action (click, input, navigation, tab)
- Include application/system and user + role in login steps
- All steps must have expected behavior

Negative Scenario Test Creator:

Generate up to 10 negative test cases in Romanian:

- Focus on invalid, unexpected, or incorrect input and blocked end-to-end flows
- Group cases by element/feature
- Mark title with "scenariu negativ" and add "negativ" to Labels
- No duplicates
- Max 50 steps per case
- Each step = a user action (click, input, navigation, tab)
- Include application/system and user + role in login steps
- All steps must have expected behavior

Non-functional Test Case Generator

Generate 1 non-functional test case for each of the following: Performance, Security, Usability, Compatibility.

- Mark title with "scenariu non-functional"
- Add "non-functional" to Labels
- No duplicates
- Max 50 steps per case
- Each step = a user action (click, input, navigation, tab)
- Include application/system and user + role in login steps

- All steps must have expected behavior
- Output must be in Romanian

UI/UX Non-functional Test Generator

Generate up to 5 non-functional test cases in Romanian focused on User Interface and User Experience.

- Mark title with "scenariu non-functional"
- Add "non-functional" to Labels
- No duplicates
- Max 50 steps per case
- Each step = a user action (click, input, navigation, tab)
- Include application/system and user + role in login steps
- All steps must have expected behavior

IMPORTANT! Prompts can be edited by the user in the test scenario generation process to obtain results as close as possible to expectations.

AI-Generated Test Scenarios Evaluation Guide

Purpose of the guide

This guide establishes the steps and evaluation criteria used by testers for the test scenarios generated with Test Manager - UiPath, so that:

- the responsibility for the tests' quality remains with the tester;
- the generated test scenarios are treated as proposals, not final deliverables;
- the testing quality is maintained at the level of the organization's standards.

Basic principles

- **Test Manager UiPath = assistant, not author**

The tester is the owner of the test scenario and decides the final form.

- **Critical validation, not passive acceptance**

Test Manager UiPath can make logical, contextual, or terminological mistakes — the test scenario must be read "like a text written by an intelligent intern, but without practical experience".

- **Contextual verification**

Test Manager UiPath does not know all the constraints of the application, environment, or internal flows; the user must ensure they provide it with the most appropriate and complete context by adding relevant supplementary documents and/or updating the prompt.

- **Final responsibility belongs to the tester**

Only the tester decides on the quality and usefulness of a generated scenario and submits the final test scenario into Jira.

The evaluation of test scenarios is carried out in the following order:

1. Initial evaluation - details here
2. Detailed evaluation - details here

Initial evaluation of tests generated with Test Manager

Immediately after generating the test scenarios, before pressing the **Create tests** button, the user visually goes through all the generated scenarios successively, quickly answering the following questions:

No	Question
1	Were the types of tests requested via the prompt generated (e.g., happy-path)?
2	Does the test have the Description and Expected behaviour fields filled in?
3	In the Description field, is the user's action clearly highlighted?
4	In the Expected behaviour field, is the result of the action clearly highlighted?
5	Are the steps in the test logical and do they follow a real usage flow?
6	Is the test relevant, does it cover at least one requirement from the US for which it was generated?
7	Is the test repetitive, redundant with another test generated for the User Story?
8	Was the information from the attached documents taken into consideration?
9	Does the test respect the naming rule and any labels mentioned in the prompt?
10	Can the test be executed manually independently?

1. Depending on the answer to the questions (YES/NO/Partial), the user decides which tests they can use by checking the corresponding checkbox and which tests they will not use at all, leaving the checkbox unchecked. To create the selected tests, follow the steps here.
2. If the user decides that the result of the test generation is unsatisfactory, and does not select any generated test for creation/use, they will press the **Regenerate** button, then follow the detailed steps in Generating test scenarios, starting with the **Provide additional context** window, where they can add new relevant documents and/or further down the flow they can customize/modify the prompt to generate usable tests.

The initial evaluation stage will always end by checking the checkbox of the generated tests considered usable. The flow continues with Saving test scenarios in Test Manager and sending them to JIRA - QA - Confluence.

Detailed evaluation of tests generated with Test Manager

After saving the tests in Test Manager, the user manually checks the following criteria and refines the test scenarios, if necessary:

Criterion	What you check	Examples of signals
A. Relevance	Tests an existing functionality, a valid requirement, or a real user flow.	The scenario describes a function that does not exist in the application.
B. Logical correctness	The steps lead to a valid result; they are not inverted or impossible.	The scenario asks for "login before accessing the login page".
C. Testing value	The scenario brings additional coverage, including edge cases.	The scenario duplicates an already existing test.
D. Ethics and UX (if applicable)	The scenario does not contain ambiguities or unethical assumptions in the user description.	Uses unrealistic or unpermitted personal data.

Refining the test scenarios involves modifications consisting of:

- Adding some steps
- Eliminating some steps
- Modifying the order of some steps
- Updating some steps through a clearer, simpler language
- Adjusting test data and prerequisites.

This will also be done by going through some additional questions compared to the ones above:

No	Question
1	Do the scenarios cover all the preconditions defined in the user story?
2	Are all involved roles mentioned (user, support roles, involved systems)?
3	Do the scenarios validate all the acceptance criteria from the user story?
4	Are the scenarios complete (no essential steps or relevant scenarios are missing)?

The flow continues with Steps to follow in Jira - QA - Confluence

Saving test scenarios in Test Manager and sending them to JIRA

After completing the initial evaluation stage of the generated tests, the user presses the **Create tests** button and the generated tests are saved in Test Manager and automatically sent to Jira.

The saved tests will be found in the Test Cases section on the main page of the requirement.

At the same time, the test scenarios are automatically sent to JIRA, in the Xray Test Repository, in the Drafturi Test Manager subfolder within the folder of the SCRUM team for which the test scenarios were created.

Further on, the user will go through the detailed evaluation stage, described here.

Steps to follow in Jira

Steps to follow in Jira

- To quickly identify in Jira each test scenario created with Test Manager for a certain User Story, enter the Jira URL of the test scenario in the JIRA Address bar. We find this URL in Test Manager -> Requirements -> Access US from the list -> Access Test Case from the Overview tab of the US -> Access Details tab of the Test Case.
- **IMPORTANT!** The user enters each test scenario generated by them with Test Manager and saves their own user as Assignee. In the Reporter field, the value 'Jira Automation' must always remain.
- Edit the test scenario and save the following updates:
 1. in the Link issues section, select the value "tests" in the Linked issues field and add the tested US in the Issue field.
 2. Update the priority of the test scenario, according to 04. LX Priorities (*) - QA - Confluence.
 3. Move the test scenarios from the Drafturi Test Manager subfolder to the corresponding subfolder within the SCRUM team's parent folder.

After completing all the steps above, the user will continue with the as-is steps / workflow provided by the Manual Testing Workflow, starting with 05. Validating LXs\Test Scenarios () - QA - Confluence. **NOTE!** If it is found that a test created by Test Manager and automatically arrived in Jira in the Drafturi Test Manager subfolder will not be used, it will be transitioned to the 'Archived' status and will be moved to the Arhivate subfolder within the Scrum team's parent folder, according to Archiving LXs () - QA - Confluence.

02. Creating tests in Xray (*)

Naming the LXs

- The title will contain in its name the name of the project team\tested project.
- An LX title will have "positive scenario", "negative scenario", "regression scenario", "smoke scenario", "non-functional scenario" in its name to be able to differentiate them.
- If UI fields or elements are tested, it is recommended to use the name "field validation" or "UI element validation".
- If business flows are tested, it is recommended to use the flow name in the name.
- If an LX continues another LX in the same flow, it is recommended to use the numbering of the LXs:
 - Individual Client Opening - Creatio - Non-resident individual client without card with high risk degree (D-E) - positive scenario 1

- Individual Client Opening - Creatio - Non-resident individual client without card with high risk degree (D-E) - positive scenario 1 - 1
- Individual Client Opening - Creatio - Non-resident individual client without card with high risk degree (D-E) - positive scenario 1 - 2

Rules for writing test scenarios:

- ALL test scenarios must have the LABEL: *US number* to make it easier to add multiple LXs to the Sub Test Execution (01. Adding LXs to Sub Test Execution-Xray Workflow (*) - QA - Confluence) and a label with the type of the created scenario.
- A test scenario MUST NOT exceed 50 steps.
- If a scenario exceeds this limit, the tester must analyze and evaluate the possibility of splitting it into multiple smaller scenarios, so that each scenario remains clear, manageable, and specific to a distinct functionality.
- Scenarios must be concise, avoid redundant steps, and focus on efficiently testing a functionality.
- If a scenario continues another scenario, the “Call Test” function will be used.
- Scenarios should ideally be written following the business flow, and the execution of test cases must be thought of as having a logical sequence.
- Scenarios should not verify the same thing multiple times; "exhaustive" type testing will be limited as much as possible.
- If a business flow requires multiple scenario combinations to be covered (e.g., users with different roles for different screens\application functionalities), using the most representative test data combinations will be attempted as much as possible, thus covering as many important cases as possible - but not all possible combinations to limit the testing effort.

The structure of the test scenarios is as follows:

Action	Represents the concrete step the tester must perform in the application. This is a clear, precise, and easy-to-follow instruction. It must accurately describe the user's action.	Enter the amount in the „Transfer value” field.	 Yes
Data	Specifies the information required to execute the step. It can contain specific values, test conditions, or missing data.	100 RON / Insufficient balance	 No
Expected Result	Describes the correct behavior of the application, according to the specifications, after executing the step. The expected result must be clear, observable, and measurable.	The system displays the message „Transfer successfully completed”.	 Yes

For adding LXs in X-ray, the following steps are respected:

Step 1: Access the Xray Library from the Jira project and enter the Xray Test Repository section.

Step 2: Search for the folder with the name of the scrum you are part of.

Step 3: Depending on the tested functionality or application section, identify the corresponding subfolder.

– If it does not exist, create a new subfolder following the steps from the chapter Creating folders/subfolders in Scrum projects - QA - Confluence.

Step 4: Right-click on the subfolder where you want to upload the LX and select the Create Test Here option.

Step 5: A pop-up will open in which some essential fields for creating the test must be filled in.

Step 6: The following fields will be filled in:

General Section

- **Project:** Xray Library (LX)
- **Issue Type:** Test
- **Summary:** ScrumName–USName–scenario type
- **Influenced applications:** Filled with the information from the US
- **Description:** -
- **Reporter:** Filled automatically with the name of the person who created the LX
- **Assignee:** Assign to me
- **Priority:** A value is chosen depending on the criteria described in chapter 04. LX Priorities (*) - QA - Confluence
- **Environment:** -
- **Labels:** USnumber, scenario type (Example: SCRT-6538, positive)
- **Test Repository Path:** The location in the Xray library where the LX is created
- **Epic Link:** -
- **Attachment:** -

Link Issues Section

- **Linked Issues:** Tests
- **Issue:** US number

After filling in all the fields mentioned above, the LX looks like this:

Instructions for adding test steps:

1. Manual addition of steps
 - if there are steps from another LX that can be used, use the Call test function, if not, add steps manually according to the steps below:
 - Add Step -> New
 - manually fill in the sections: Action, Data and Expected Result

NOTE! In case a different view of the added test steps is desired, the following method can be used:

1. Add the first test step and press the Steps Dialog button.
2. A new window is displayed where the test steps are viewed differently.
3. After entering all the steps, click the Save button and return to the initial page.

Archiving LXs (*)

All LXs that are no longer used or relevant must be transitioned to the Archived status to keep track and maintain the traceability of the testing process.

Each team uploads ALL LXs with the Archived status into the Archive subfolder within each folder bearing the scrum team's name. If the subfolder does not exist, one is created following the steps from the chapter Creating folders/subfolders in Scrum projects - QA - Confluence.

Reasons why an LX can be transitioned to Archived status:

- The LX is no longer used in the current workflow
- The associated requirement or functionality was removed from the project scope
- A new version exists that completely replaces it
- It can no longer be updated or maintained in a relevant form for the project
- It was identified as a duplicate of another already active LX
- Business changes have made the LX irrelevant
- LXs generated with Test Manager that are either no longer necessary or have been compiled into another LX and no longer require use

We recommend that the status of an LX be changed to Archived only after going through all the necessary checks.

LXs should be kept updated as much as possible, and archiving should be used only when the LX can no longer be used to a significant extent.

Instead of deletion, the Archived status allows keeping the historical record and maintaining traceability in the testing process.

Call Test (*)

When is Call Test used

- You have a set of steps that repeat in many scenarios (e.g.: Login, Logout, navigation to a certain menu)
- You have created “building blocks” (basic) scenarios and want to include them in more complex ones
- You want to maintain consistency between scenarios and avoid mistakes through copy/pasting
- If a functionality is modified -> you only update the base scenario and all scenarios that “call” it are automatically updated
- When you want to reduce maintenance time and have cleaner, shorter, and easier-to-read scenarios

Steps for adding:

- Add Step -> Call Test

- in the newly opened pop-up, add the number of the LX you want to reuse
-

01. Test Scenarios: rules and structure (*)

1. Purpose of LXs

The LXs document the test scenarios in the Xray Library, ensuring the traceability and correct validation of the User Stories (US).

Each LX covers a specific functionality and can be of the positive, negative, regression, UI/UX, smoke or non-functional type.

2. Organization in the Xray Library.

The detailed working procedure can be seen in chapter 05.
Working with Xray Library (Xray Test Repository) (*) - QA - Confluence

- The LXs are created in the Xray Test Repository, in the corresponding folder of the SCRUM team.
- Each functionality has its own subfolder; LXs are not added directly into the main folder.
- To create a new subfolder:
 1. Select the SCRUM folder.
 2. Press the „+” button.
 3. Rename the New folder with the name of the tested functionality or section.
- In each folder named with the scrum team's name, there must be a Regression Tests, Smoke Tests, Archived, TestManager Drafts subfolder and folders named after functionalities/sections.

3. Naming rules

General format:

ScrumName - USName - scenario type

Scenario types:

- positive scenario
- negative scenario
- regression scenario
- smoke scenario
- non-functional scenario
- field validation / UI elements validation

If a flow has multiple parts → it is numbered: Part1, Part2 or 1-1, 1-2.

4. Mandatory structure of scenarios

Each LX must include all three columns:

What the tester does in the application	With what data / conditions	What correct result must appear
---	-----------------------------	---------------------------------

Rules:

- The Data field is optional, but recommended for certain steps.

- Maximum 50 steps per LX → if it is exceeded, it is split into multiple LXs.
- Avoid redundant steps; follow the logic of the business flow.
- Use Call Test to reuse existing scenarios. For more details about the Call Test function, check the chapter Call Test - QA - Confluence.

5. Rules and best practices

- All LXs must have the label with the US number and the scenario type.
- Smoke and Regression scenarios are kept in dedicated folders and contain the smoke/regression label.

01. Positive LXs (*)

Positive LXs or positive scenarios must contain tests that verify if the application works correctly when it is used according to expectations (the US), with valid data.

A positive scenario in testing is a test flow in which the user respects the application rules and introduces valid data, and the system works according to expectations.

In other words, positive scenarios verify if the application does what it is supposed to when it is used correctly.

The title of the positive scenario must be of the format: ScrumName-USName-positive scenario

Label: positive, *USno*

Note: All created LXs must be linked to the US.

Simple examples:

- Login: the user enters the correct username and password → the system allows authentication.
- Online payment: the user enters valid card data and has sufficient funds → the payment is processed successfully.
- Form completion: all mandatory fields are filled in correctly → the form is submitted without errors.

Usually, positive scenarios are the first ones to be written and tested, because they verify the basic functionality and confirm that the application "works" under normal conditions.

Complete details about how to create LXs can be found in the chapter here 02. Creating tests in Xray (*) - QA - Confluence.

02. Negative LXs (*)

Negative LXs or negative scenarios verify how the application handles unexpected situations or invalid data. The goal is to ensure that the application does not behave incorrectly or crash.

A negative scenario in testing is a test flow in which the user does not respect the application rules or enters invalid data, to verify how the system handles errors.

Practically, negative scenarios confirm that the application does not allow incorrect actions and displays corresponding error messages.

The title of the negative scenario must be of the format: ScrumName-USName-negative scenario

Label: negative, *USno*

Note: All created LXs must be linked to the US.

Simple examples:

- Login: the user enters the wrong password → the system displays the message "Invalid username or password".
- Online payment: the user enters an expired card → the system blocks the transaction and displays the error.
- Form completion: the user leaves the mandatory fields blank → the system does not allow submission and displays a warning message.

Negative scenarios are just as important as positive ones, because they ensure the fact that the application is robust and does not "crash" when the user makes a mistake or tries to break the rules.

Complete details about how to create LXs can be found in the chapter here 02. Creating tests in Xray (*) - QA - Confluence.

03. Regression LXs (*)

Regression LXs or regression scenarios are carried out after modifications in the application (bug fix, new feature) to verify if the existing functionalities continue to work correctly. The goal is to prevent the appearance of new bugs in the already developed functionalities.

Regression scenarios:

- Regression scenarios must be added to the "Regression tests" folder within each SCRUM folder.
- The title of the regression scenario must be of the format: ScrumName-USName-regression scenario
- Label: regression, *USno*
- Note: All created LXs must be linked to the US.

For more details on how regression tests are defined, they can be found in chapter 02. Defining Regression Test Scenarios (-) - QA - Confluence.

Complete details about how to create LXs can be found in the chapter here 02. Creating tests in Xray (*) - QA - Confluence.

04. UI/UX LXs (*)

UI LXs or UI scenarios refer to verifying the visual and interactive aspect of an application: colors, buttons, layout.

UX LXs or UX scenarios refer to the overall user experience: how easy, how intuitive, and how pleasant it is to use the application.

Their goal is to make sure the interface is attractive (UI) and that navigating the application is done easily and efficiently (UX).

The title of the UI/UX scenario must be of the format: ScrumName-USName-field validation or ScrumName-USName-UI/UX elements validation

Label: UI/UX, *USno*

Note: All created LXs must be linked to the US.

Complete details about how to create LXs can be found in the chapter here 02. Creating tests in Xray (*) - QA - Confluence.

When do we do UI/UX Testing?

- After implementing a new design or functionality.
- Before a major release.
- After feedback from real users.
- Within regression testing for visual or interactive components.

Concrete examples in UI/UX testing

1	Design consistency	Displaying on various sizes and resolutions	The UI adapts without overlapping or disappearing; fonts and buttons remain readable
2	Adaptability to orientations/resizing	Changing screen orientation/window resizing	Elements rearrange correctly, without loss of content or layout
3	Interactions and gestures	Click, hover, swipe, drag & drop, etc.	All respond correctly and intuitively, without delays or unexpected behaviors
4	UI Performance	Response to actions, animation speed, fluid scroll	The interface remains fluid even on slow devices
5	State management	Application in background, screen lock, refresh	The entered data and position in the flow are kept
6	Behavior in varied networks	Lack of connection, weak network, reconnection	Clear messages, offline actions saved, recovery without losses
7	Visual feedback and confirmations	Executed actions: save, error, success	The user receives clear feedback (message, visual change, animation)
8	Ergonomics	Accessibility with one hand or just keyboard	Important elements are easily accessible and large enough for easy interaction
9	Accessibility	Screen readers, dark mode, large fonts	Readable text, easy navigation, full support for assistive technologies

1	Design consistency	Displaying on various sizes and resolutions	The UI adapts without overlapping or disappearing; fonts and buttons remain readable
10	Permissions and notifications	Access requests and behavior if refused	Explicit messages, the application works correctly even without permissions
11	Behavior in special conditions	Battery saving mode, multitasking, zoom, bright light	The UI remains stable and readable, without major interruptions
12	Simplicity and clarity in flows	Forms, checkout, usual operations	Minimum steps, useful validations, clean design without useless elements

05. Smoke LXs (*)

Smoke tests represent a specific type of tests\LXs. For more details here: 02. Defining Smoke tests - QA - Confluence

Smoke scenarios:

- Smoke scenarios must be added to the "Smoke tests" folder within each SCRUM folder.
- Label: smoke, *USno*
- More details here 01. Smoke Tests Workflow - QA - Confluence

Complete details about how to create LXs can be found in the chapter here 02. Creating tests in Xray (*) - QA - Confluence.

Recommendations:

For an LX that goes through screen windows - recommendation to have 1 LX for each screen; if there are screens accessed using another LX - call test to the LXs needed to reach the screen

For flow LXs 1 complex happy path LX

1 failed login LX

1 or more LXs with rights for specific roles - for obvious screens after login

1 successful login LX - extract login steps from existing LXs, create a separate login LX to be called by the other tests

It is recommended to use the call test function as often as possible where possible.

06. Non-functional LXs (*)

Non-functional LXs verify the performance, stability, security, and usability of the application, not the actual functionality.

The goal is to ensure that the application works correctly and efficiently under real conditions of use, providing a good user experience.

A non-functional scenario is a test flow that evaluates the behavior of the application beyond the basic functions — for example, how fast the system responds, how stable it remains under load, or if data is properly protected.

Practically, non-functional scenarios confirm that the application:

- responds in an acceptable time,
- can support a large number of simultaneous users,
- is safe in terms of access and data,
- is easy to use and intuitive.

The title of the non-functional scenario must be of the format: ScrumName-USName-non functional scenario for scenarios that cover performance, security, Usability, Reliability, Compatibility, Compliance, Configuration requirements, etc.

- Label: non-functional, *USno*

Note: All created LXs must be linked to the US.

Simple examples:

- Performance: the system responds in a maximum of 2 seconds when 100 users simultaneously access the main page.
- Security: the unauthenticated user tries to access an internal page → the system redirects to the login screen.
- Stability: the application remains functional after 12 hours of continuous use without errors.
- Usability: buttons and error messages are clear and intuitive, so the user can quickly understand how to correct a mistake.

Non-functional scenarios are just as important as functional ones, because they ensure that the application not only "works", but works well, fast, and safely under real conditions of use.

Complete details about how to create LXs can be found in the chapter here 02. Creating tests in Xray (*) - QA - Confluence.

04. LX Priorities (*)

Smoke tests will always have Urgent priority, for more details here 02. Defining Smoke tests - QA - Confluence

Regression Tests will have another way to set priorities 04. Regression tests priority (*)

General prioritization criteria

Impact upon customer	How serious it is if the functionality doesn't work. Ex: cannot access money.
Frequency of use	How often the functionality is used. Ex: login = daily.
Technical risk	How unstable or sensitive the tested area is.
Blockage for other functions	If it affects other modules. Ex: login blocks the whole application.

Practical recommendations:

- Urgent = Functionalities that affect access to or the safety of money.
- High = Important functionalities, but where the error does not lead to direct losses.
- Normal = the majority of secondary functionalities, testable later.
- Low = Rare, unused or experimental functionalities.
- Not a Priority = The default priority with which an LX is created. It is mandatory to be changed.

Priority mapping, test types, and examples

The examples below are generally applicable, with exceptions listed below the table

Urgent	Positive	Critical for business, blocking	Authentication in the mobile app with SMS code (2FA)
	Negative	Security, legal risk	Attempt to access with multiple invalid data → account blocked
	Regression	Essential functionalities recently modified	Confirming an interbank transfer works incorrectly
High	Positive	Important functionalities, but not blocking	Opening a savings account
	Negative	Critical validations, but without immediate risk	Transfer to an invalid IBAN must be refused
	Regression	Usual behaviors with bug history	Displaying the balance after login
Normal	Positive	Standard functionalities, without risk	Updating the address in the client profile

Urgent	Positive	Critical for business, blocking	Authentication in the mobile app with SMS code (2FA)
	Negative	UI validations, error messages	Entering a special character in an unpermitted field
	Regression	Stable areas, with low impact	Navigating between application tabs (e.g.: "Accounts" → "Payments")
Low	Positive	Rare or auxiliary functionalities	Saving a custom theme in the banking application
	Negative	Rare scenarios, without impact	Simultaneous opening of the application on 2 devices
	Regression	Almost unmodified functionalities	Viewing the history of an old payment from 2019

05. Validating LXs\Test Scenarios (*)

Validating test scenarios is the process by which it is verified if the test scenarios are correct, complete, and relevant in relation to the requirements of the application or system to be tested.

What is a test scenario?

A test scenario describes a general testing situation that validates a certain usage flow or functionality of the application. More information here 02. Manual Test Scenarios Definition Workflow (*)

What does validating scenarios mean?

Validating test scenarios means ensuring that:

1. They cover all the requirements defined in the functional specifications.
2. They are logical and achievable – meaning they can actually be run by testers.
3. They reflect the real use of the application by the user.
4. They correctly identify positive and negative conditions (e.g.: login with wrong data).
5. They are clear and without ambiguities, so that anyone who reads them can understand them.

01. What does validating LXs\test scenarios mean? (*)

Validating test scenarios = verifying the fact that the scenarios:

- test the correct requirements,
- are complete and clear,
- can be executed without ambiguities,
- have correct and verifiable expected results.

A Product Owner (PO), Application Owner (or IT Business Analyst\Developer) or Real Beneficiary can return the LXs to the tester after validation for several reasons, to ensure that the tests correctly cover the established requirements and standards. Here are some common reasons:

Incomplete coverage of business flows

- If the PO\Application Owner\Real Beneficiary notices that certain business flows or functionalities are not covered by the LX(s), they can ask for their completion or modification.
- If the tests do not include critical scenarios or important use cases, the PO\Application Owner\Real Beneficiary can request their inclusion to ensure complete coverage.

Discrepancies between requirements\flows and LXs

- If the tests seem to reflect an erroneous interpretation of the business flows, the PO\Application Owner\Real Beneficiary can request their correction to better align with expectations.
- If the expected results from the LXs do not correspond to the established expected results, the PO\Application Owner\Real Beneficiary can request their revision.

Lack of clarity

- If the steps described in the LXs are not clear enough or are ambiguous, the PO\Application Owner\Real Beneficiary can ask for clarifications or their rewording to avoid confusions in execution.

- The PO\Application Owner/Real Beneficiary can request more details in the expected results section if these are not precise enough to determine the success or failure of a test.
- LXs should ideally be written following the business flow, and the execution of test cases must be thought of as having a logical sequence. If the PO\Application Owner/Real Beneficiary notices that the logical and normal business flow is not respected then they can request their revision.

Repetitive tests or tests that verify the same thing multiple times

- LXs should not contain steps that repeat in other LXs. If the steps repeat, they must be called using the "Call Test" function from Xray to be able to maintain a logical structure of the tests. This working method will also be useful for test automation. For this reason, the PO\Application Owner/Real Beneficiary can send the LXs back for revision and rewriting.
- LXs should not verify the same thing multiple times; limiting the "exhaustive" type testing detailed below will be attempted as much as possible. For this reason, the PO/Application Owner/Beneficiary can send the LXs back for revision and rewriting.

Lack of negative scenarios and edge cases

- If the tests do not include negative scenarios (for example, invalid inputs or unforeseen behaviors), the PO\Application Owner/Real Beneficiary can request their addition to verify the results of the tested applications and the displaying of errors.
- In case limit situations or rarer cases are not covered, the PO\Application Owner/Real Beneficiary can ask for a revision to include these scenarios, essential for ensuring quality.

Alignment with Recent Modifications

- If recent modifications were made in the requirements or business flows, and the tests were not updated to reflect these changes, the PO\Application Owner/Real Beneficiary can ask for the revalidation or updating of the tests.

For any of the reasons above, the PO\Application Owner/Real Beneficiary can send the LXs received from Testers back for rewriting.

The reason why the test cases are sent back will have to be mentioned either as a comment in the LX.

02. Validating LXs\test scenarios workflow (*)

After creating the test scenarios, the LXs must be added in the Description field of the Defining Scenarios sub-task.

To avoid keeping information outside Jira, the validation of scenarios will be done in the following way.

- The test scenarios\LXs are created according to 02. Identifying and creating tests in Xray (*) - QA - Confluence
 - the Links within each LX "tests\Tested by" are added to the tested User Stories
- The "Defining Scenarios" Sub-task is created according to 06. Create Defining Scenarios sub-task (*)
 - in the "Defining Scenarios" Sub-Task, all the LXs testing the User Story will be added in the "Description" field
- The LXs are created and remain in the "Defining" status until the validation process concludes

- when the LXs are considered complete from the Tester's point of view, they will add a comment in the "Defining Scenarios" Sub-Task for validation
 - the comment will have the form "please validate the LXs in the Description"
 - the users involved in the normal validation of the LXs will be added with @:
 - mandatory validator: Product Owner and Testing Team Lead
 - the PO has the responsibility to further hold the scenario presentation meeting to the Application Owner and real Beneficiary
 - the team must be consulted for the complete list
- The defining scenarios sub-task will remain in "In Progress" until a response is received (at least from the PO)
 - comment from the validators "Test scenarios\LXs OK"
 - any request to update\modify the LXs from the validators will be done in Jira in comments
 - the scenario validation process ends when all the LXs have been modified according to the modification requests from the Jira comments on the sub-task
 - at the end of the scenario validation process, the LXs will be transitioned into the "Finalized Scenarios" status and the "Defining Scenarios" Sub-Task can be closed with the "Closed" status
- In case it is found that the US is in a status past the testing phase and the validation was not received, the problem is escalated to the PM for unblocking.
- Examples of reasons why the LXs\test scenarios can be sent for revision can be found here 01. What does validating LXs\test scenarios mean? (*).
 - To avoid revising the LXs it is recommended to cover the points in the list as a minimum necessity
 - The list must be treated as an orientation guide, each team will have specificities not covered by the list

03. To whom are the scenarios sent for validation (*)

The LXs (or scenarios) are usually sent for validation to the Product Owner, Application Owner or real Beneficiary. It was agreed that it is the Product Owner's responsibility to send the scenarios to the Application Owner/real Beneficiary as well if they consider input is necessary for validating the scenarios.

Generally (with a few exceptions) the Product Owner and the Application Owner are the same person so the scenarios (LXs) will be sent for validation to the same person.

LXs can also be sent for validation to BAs, IT representatives or even the entire SCRUM\Waterfall team.

User Stories contain the "Influenced Applications" field - these are the applications that will be tested.

To be able to identify the Application Owner for the tested application, the following columns from the List of applications and system owners must be checked

- K (Functional ownership)
- L (Functional owner)

The list of tested applications below are the applications that have been historically tested within the teams/projects (based on the "Influenced applications" field in Jira for User Stories within the projects)

Team\Project	Product Owner	Tested Applications
AA (Waterfall Upgrade)	N\A	
Flow Credite	Andreea Lupulescu	Credit Flow
Pj. Externe (Creatio Pensii)	N\A	IBS, T24, [Creatio] Flux Pensii, LibraBank, [Sysmart] Collection, [Arggo] Aplicatie Brokeri
SCRUM Admin. Credite	Andreea Lupulescu	IBS, T24, IBK, [Sysmart] Credite, Portal evaluatori externi, Sysmart
SCRUM Alte Divizii	PO	IBS, T24, IBK, Libra Business Suite, LibraBank, Libra Online, Aplicatia de pontaj, OpenKM, Mediatel, Portal evaluatori externi, [Arggo] Aplicatie Brokeri, Sysmart, [Sysmart] Valorificari Bunuri, [Sysmart] Collection
SCRUM Analiza Credite	Alexandru Marghidanu	IBS, T24, IBK, OpenKM, [Sysmart] Credite, Libra Business Suite, Sysmart, Creatio, CRM MS Dynamics (old?), RECOM (?old)
SCRUM Carduri	Florin Draghici	IBS, T24, IBK, Libra Journey, LibraPay, Libra Online, OpenKM, LibraBank, IBS WebAPI, IBS card API, Serverul de autorizare, API Banking, Mediator
SCRUM Conformitate	Adrian Tilinca	IBS, T24, IBK, OpenKM, Temenos FCM, Libra Home
SCRUM CP	Simona Volintiru	IBS, T24, OpenKM, [Sysmart] Popriri, [Creatio] Flux Pensii, Libra Business Suite, LibraBank
SCRUM Creatio Deschidere Client	Cosmin Cazan	IBS, T24, [Creatio] Deschidere Clienti, [Creatio] CRM

Team\Project	Product Owner	Tested Applications
SCRUM CRM	Catalina Oprea	[Creatio] CRM, [Creatio] Credite (old), CRM MS Dynamics (old)
SCRUM Decontari	Ramona Barbu	IBS, T24, IBK, FCM, LibraBank, OpenKM
SCRUM Digital	Adelina Peta	IBS, T24, IBK, MBK, Libra Business Suite, Libra Online, Sysmart
SCRUM Digital HUB	Eduard Weiss	IBS, T24, IBK, API Banking, IBS WebAPI, Mediator, WSO2 API GatewayExtern, API Investigator, LibraBank, [Arggo] Aplicatie Brokeri, [Sysmart] Investigator, RT
SCRUM Digitalizare fluxuri clienti	Cosmin Cazan	IBS, T24, IBK, [Creatio] Deschidere Clienti, [Creatio] CRM, IBBO Clienti, OpenKM, RT
SCRUM IBK/MBK	Cristina Costovici, Silviu Babescu	IBS, IBK, MBK, IMBK WebAPI, Libra Journey, IBBO Clienti, Libra Online, IdentityServer, LibraBank, IBS card API, LibraPay, OpenKM, API Banking
SCRUM Libra Journey	Iulia Balasan	IBS, IBK, Libra Journey, IBS card API, LibraBank, LibraPay
SCRUM LibraPay	Eduard Weiss	IBS, IBK, Libra Journey, LibraPay, Dashboard LibraPay (comercianti), Serverul de autorizare
SCRUM Operatiuni	Cristina Tanasescu	IBS, T24, IBK, OpenKM, FCM, LibraBank, IBBO Clienti
TIPS (Decontari)		
TPH (Waterfall Upgrade)	Ramona Barbu	

Examples of Application Owner for the most frequently tested applications below (the complete list of applications and Owners is here: List of applications and system owners)

In the IBS and T24 cases, the Application Owner will be the representative of the specific module from the Functional ownership column (application ownership division)

Application	Functional ownership (application ownership division)	Functional owner (Application Owner)
IBS	Director Divizia Operatiuni Decontari - Sef departament/VP coordonator Analiza financiara - Director Div. Analiza Credite Corporate Administrare contracte - Director Div. Administrare Credite RECOM - Director Divizia Operatiuni ARHIVA - Director CP ePopriri - Director CP AML - Director Div. Conformitate CARDS - Cards Management - Director Div. Carduri RISK - Director Div. Adm. Riscurilor	Director Divizia Operatiuni

Application	Functional ownership (application ownership division)	Functional owner (Application Owner)
	Monitorizare clienti si workout - Director Div Monitorizare Clienti si Workout	
T24	Director Divizia Operatiuni Owner modul FT - Director Divizie Operatiuni Owner modul LD - Director Div. Administrare Credite Owner modul PD - Director Div. Administrare Credite Owner modul FX - interbancar - Director Div. Trezorerie	Director Divizia Operatiuni
Libra Internet Banking (IBK)	Director eBusiness	Director eBusiness - Alexandru Bită
Libra Mobile Banking (MBK)	Director eBusiness	Director eBusiness - Alexandru Bită
Libra Journey	Director Divizia Carduri	Director Divizia Carduri
LibraHome	Director Divizia eBusiness	Alexandru Bită
Libraonline	Director Divizia Marketing	Octavian Hera

Application	Functional ownership (application ownership division)	Functional owner (Application Owner)
LIBRAPAY	Director Divizia eBusiness	Alexandru Bită
OpenKM Document Management	Director Divizia Managementul Calitatii	Simona Volintiru (sau Georgian Stoica)
Creatio CRM, Pensii, Deschidere Client	Divizia Managementul Calitatii Proceselor	Marghidanu Alexandru (sau Georgian Stoica)
Sysmart - Audit	Director Divizia Audit Intern	Madalina Soare
Sysmart - Collection	Director Divizia Colectare Debite	Dragos Vilcu
Sysmart - Debitori	Director Divizia Administrare Credite	Emilia Nicolae
Sysmart - InvestigatorVD	Director Divizie Analiza Credite	Irina Sita
Sysmart - Popriri	Director Centrul de Procesare	Cornelia Vieriu
Sysmart - Urmare Credite	Director Divizia Analiza Credite	Irina Sita

06. Create Defining Scenarios sub-task (*)

All newly created User Stories will have this sub-task created.

However, if there is a need for this sub-task to be created manually, the steps below can be followed.

This sub-task will be used for logging the time spent on writing test scenarios and for validating the created test scenarios.

1. Creating a subtask

To open this type of sub-task, the steps below will be followed:

STEP 1: We enter the User Story(US) that was assigned to us.

STEP 2: We scroll on the US until we reach the Sub-Tasks section and press the + button on the right side.

STEP 3: The Create Subtask pop-up opens: *scrum abbreviation-US number*.

STEP 4: We fill in the following fields as follows:

Field	Instruction/ Value to be filled in
Issue Type	Sub-Task
Summary	Defining Scenarios
Description	-
Assignee	Assign to me
Sub-task type	Testing None
Original Estimate	Time estimate for the effort put into writing scenarios
Remaining Estimate	-
Labels	-
Attachment	-
Story Points estimate	-
Access ticket	-

STEP 5: Press the Create button and verify if our sub-task was added in the Sub-Tasks section of the taken US.

STEP 6: Access the previously created sub-task.

The Defining scenarios sub-task is created to help the team know the stage we are at with our activity on the US that was assigned to us. In this sub-task you have the obligation to log your time spent creating test scenarios.

2. Subtask work flow

Upon creating the subtask, its status will be TO DO. The tester has the obligation to change the status from TO DO → IN PROGRESS when they start writing the scenarios in X-ray or generating them with TestManager. The status will remain IN PROGRESS until validation is received from the PO/Application Owner. After validating the scenarios, the subtask status must be modified to DONE.

07. Adding LX to the Defining Scenarios Subtask (*)

Steps for adding the LX on the defining scenarios subtask:

Step 1: In case there is no "Defining Scenarios" subtask on the US being worked on, the subtask is created according to the steps here.

Step 2: The LX is created in the X-ray Library according to the steps here.

Step 3: Return to the "Defining Scenarios" subtask and add the previously created LX/LXs in the "Description" field.

Step 4: After adding the LXs, follow the steps described here.

02. Manual Test Scenarios Definition Workflow (*)

The workflow describes the complete process of defining, documenting and validating test scenarios (LXs) related to a User Story (US), as well as the management mode of the Defining scenarios sub-task in Jira.

The main purpose is to ensure:

- traceability between the requirements from the US and the related tests;
- a clear record of the time spent for writing and validating the scenarios;
- a unitary structure for all the scenarios created in the Xray Library.

The scenario definition process includes the following stages:

1. Creating the "Defining scenarios" sub-task following the steps in the chapter 06. Create Defining Scenarios sub-task (*) - QA - Confluence

The sub-task is created automatically for each new US. If it is missing, it is created manually and is used for logging time and for adding the related LXs.

2. Creating the LXs in the Xray Library following the steps in the chapter 02. Creating tests in Xray (*) - QA - Confluence

The tester identifies the flow that must be tested and creates the scenarios (positive, negative, regression, smoke, UI/UX etc.) respecting the naming rules and structure (Action, Data, Expected Result).

All LXs are saved in the corresponding subfolder from Xray and are labeled with the US number.

3. Adding the LXs on the "Defining scenarios" sub-task following the steps in the chapter 07. Adding LX to the Defining Scenarios Subtask (*) - QA - Confluence

After creation, the LXs are inserted in the Description field of the sub-task, to allow their visibility and validation by the PO together with the Real Beneficiary and Application Owner.

4. Logging time following the steps in the chapter 10. LogTime - QA - Confluence

The tester has the obligation to log daily the time spent for writing and validating the scenarios, in the "Defining scenarios" sub-task.

5. Validating scenarios. The working procedure can be found in chapter 02. Validating LXs\test scenarios workflow (*) - QA - Confluence

The scenarios are validated by the PO together with the Real Beneficiary and Application Owner. After validation, the sub-task is passed to the DONE status.

6. Updating flows and reusing tests

If the flow is already documented in the Xray Library, Call Test is used to avoid duplicating steps. If it doesn't exist, a new flow is documented and added to the library for subsequent reuse. For more details on how the Call test function is used, check the Call Test - QA - Confluence chapter.

This workflow ensures coherence between the scenario documentation and the testing process, contributing to better planning, transparency and control of the QA activity.

02. Executing Test Scenarios Workflow (*)

When the Tester transitions a User Story from the Ready for testing status to In testing, a pop-up will open in which 2 fields must be obligatorily filled out:

Testing type: has a default value. From the dropdown list, the type of testing to follow on the User Story will be selected:

- QM - testing is carried out by the Testers from the QM division
- Business - testing is carried out by business testers.
- The Testing Type field is visible after completion in the Requirement section (the last field) and can be edited through the Edit option.

Testing responsible: will be filled out with the name of the Tester (QM or business) who will test the requirement. As a rule, the person making the transition and testing the requirement will also fill out the Testing responsible field.

→ If the value in the "Testing type" field is changed, then the "Testing responsible" field becomes mandatory and the value in it is deleted (the user must manually add the testing responsible)

→ The Testing responsible field is visible in the People section, under Documentation responsible.

- Upon creating User Stories, 4 types of Sub Test Executions for testing will automatically be created, which will be seen in the Sub Tasks section:
 - UAT Testing - positive, negative, UI/UX, non-functional and regression LXs will be executed here
 - Smoke PRL Testing
 - PRL Testing - only Happy path LXs (at least 1) + bug retesting (discovered in UAT) with Major, Blocker severity will be executed here (Retesting all Bugs found in UAT is not mandatory)
 - Regression PRL Testing

The order of running tests in Prelive is the following: Smoke PRL Testing → PRL Testing → Regression PRL Testing

It is no longer necessary to attach evidence for any type of testing: LX execution, UAT or Prelive testing, SCRUM or Waterfall testing, including in fixing bug retesting; evidence in bug reporting remains.

After finalizing the Sub Test Execution testing (Smoke PRL Testing 03. Smoke Tests Execution (), *Regression PRL Testing* 04. *Regression Tests Execution* (), PRL Testing) the tester will transition the User Story in Jira into the Ready for Release status.

If multiple LXs were executed in the UAT test environment, but in PRL it is decided that some of them should not be executed, they are NOT added to the PRL Testing Sub-Test Execution.

The test scenario execution process includes the following stages:

1. Creating Sub Test Execution

For each User Story, a Sub Test Execution is created, either automatically, or manually if it was not generated.

2. Adding LXs on Sub Test Execution

The test scenarios (LXs) relevant for the respective testing are added into the Sub Test Execution. These LXs can be added using search options by number, name, filters or JQL - more details here. Only the LXs that actually need to be tested in each testing phase are selected.

3. Executing scenarios

The tester runs every step from the added scenarios, marking the result of each step: Pass, Fail, Blocked, Aborted etc. In case of fail, bugs are reported and evidence is added.

4. Logging time

During execution, the tester fills out in the Sub Test Execution the estimated time and the actual time consumed for each testing activity. The tester has the obligation to log their time spent executing the scenarios daily.

00. Create Sub test execution and Transition (*)

In case the User Story is older and the Sub-Test Execution was not opened automatically, follow the steps below:

During Sprint Planning meetings, it is mandatory to estimate the testing hours within the Sub-Test Execution - you can check them here

To fill in the time destined for testing, the Tester will access the Edit option on the Sub Test Execution and will modify the Original Estimate field with the estimated time for testing:

1. Enter the User Story in the Sub task section and select the "+" option
2. A pop up opens where we must fill in the fields as follows:

Field	Instruction/ Value to be filled in
Issue Type	Sub Test Execution
Summary	UAT/PRL/Smoke PRL/Regression PRL Testing
Description	-
Development environment	UAT/PRL
Reporter	Filled automatically with the name of the person opening the sub-test execution

Field	Instruction/ Value to be filled in
Assignee	Assign to me
Priority	Normal
Original Estimate	Time estimate for the effort put into testing
Environment	-
Attachment	-
Labels	-
Remaining Estimate	-

Create

Transition and statuses on sub test execution:

To Do: Execution has not started yet. The initial status when the Task was opened,

In progress: Execution is underway, the tester has started running the test scenario steps and it modifies automatically or is modified into this status by the tester.

Done: Execution was performed and finalized. The tester modifies into this status.

Aborted: Execution was interrupted and will not be finalized. A comment is left with the reason why the testing was put in this status.

01. Adding LXs to Sub Test Execution (*)

Steps for adding the LX on the Sub Test Execution in Jira

To add LXs on the Sub Test Execution, open it and press the Add Tests - Existing tests button, LXs can be added using 2 options:

1.1 "Select" Section - a list is displayed from where I can add the previously defined tests by typing the LX number or its name. It finishes by pressing the Add Selected button.

1.2. "Search" Section - a list of filters is displayed where we add a new filter: selecting the option "Filter(s) - "Label"

Project: Librarie Xray(LX)

Test Type: Manual

Contains text: can be searched by a specific word in the LX name from the Summary field

Folder: Select the folder/subfolder where the LX was saved in the Xray library

Label: the US number example- DS-6427, MBK-2827, etc

Then access the "Search" button

Adding can be done by selecting the desired LXs (Add selected) or selecting all displayed LXs (Add all)

2. "JQL" Section - a field that we fill with: issuetype = Test and labels = US Number example- DS-6427, MBK-2827, etc

Adding can be done by selecting the desired LXs (Add selected) or selecting all displayed LXs (Add all)

Running the test scenarios through Sub-test Execution is done after adding the LXs like in the steps above

- Open sub test Execution and press the Execute button (it is necessary for a person to be assigned to that issue for the Execute button to be available)
- The window where we find the test scenarios opens:

Rules for running scenarios are carried out according to the execution statuses defined below.

If the test step verification was not successfully completed, the FAIL status is set (Evidence is uploaded) and a Bug is opened according to 03. Reporting a Bug in Jira (*)

Within software testing, subtest execution statuses are essential to understand the progress and results of the tests. These statuses help QA teams to quickly identify problems and communicate efficiently about the state of the tests. Here is a detailed description of the five mentioned statuses:

Pass

- Indicates the tested functionality works as expected.

Fail

- The tested step signals a problem in the application that must be investigated and corrected, on which a Bug is opened according to 03. Reporting a Bug (*)
- The Fail status on the test step automatically changes the entire LX to Fail status,

Executing

- Indicates the test step is active and is in the process of evaluation.

Aborted

- The step to be tested is considered irrelevant for testing the User Story's functionality

Blocked

- Signals that the test cannot continue until the blocking problem is not resolved,
- The status can be used in the situation where a step from an LX cannot be run because a previous Bug exists on the flow (on other steps)
- The Blocked status on the test step automatically changes the entire LX to Blocked status, similar to the Fail status

02. UAT Testing (*)

The UAT Testing Sub-Test Execution will be automatically created in the Sub-Tasks section upon opening the User Story.

In case the US does not already have a created sub test execution, it is added according to 00. Create Sub test execution and Transition

To fill in the time destined for testing, the tester will access the Edit option from the UAT Testing section and modify the Original Estimate field from the initial value of 1 minute to the estimated time for testing.

Test execution in the UAT environment can begin after the test scenarios have been validated in Jira.

In the UAT test environment all LXs created for the US will be executed (Happy path, positive, negative, non functional, regression, UI/UX- separate Sub test execution is not created for each type of testing, all LXs are linked to the US)

On UAT Testing, it is mandatory that all steps from a scenario are executed and have the PASS status - steps in Aborted status are not accepted. In exceptional cases, if a step has an Aborted status, there must also be a justification on the respective step in the comments section.

- For validating test steps in UAT, the workflow described in the dedicated chapter is followed.
- In case of identifying Bugs during scenario execution in UAT, a BUG type Issue is opened following the steps here. The step on which it was identified being marked with FAIL status.
- After fixing the Bugs the tester has the obligation to retest them according to the workflow defined in 04. Retesting a bug fix (*). After the bug has been retested, the step on which it was identified is modified to PASS status.
- Bugs reported following UAT testing must have the "Development environment" field filled with the value "UAT". In exceptional cases, if the bug was not fixed, there must be a comment at the US level requesting conditional acceptance from the tester to the @PO with open bugs for promotion into Prelive

When the execution of tests in the UAT test environment and the retesting of Bugs is completed, the Demo meeting is held to obtain acceptance:

- The User Story status is modified to: Approval Required.
- Scrum Master sends an invitation for the Demo meeting to: Product owner, Application Owner and Project Beneficiary and other involved persons (developer, other people from business or involved in the project).
- The Demo meeting is held by the Tester (in their absence by the Product owner or Developer) respecting the steps from 04. DEMO Presentation in Agile\SCRUM with the purpose of obtaining acceptance.
- After obtaining Acceptance following the Demo meeting, the UAT Sub Test Execution status is modified from In Progress into Done. (the time spent in the Demo meeting is logged on the UAT Testing Sub Test Execution).

During Sprint Planning meetings, it is mandatory to estimate the testing hours within the Sub-Test Execution - you can check them here

To fill in the time destined for testing, the Tester will access the Edit option from UAT Testing and modify the Original Estimate field with the estimated time for testing.

If the LX is not relevant to be tested in the UAT test environment, it is not created and not added to the Sub Test-Execution.

Evidence is no longer attached on the LX steps from Sub test Execution.

03. Smoke Testing (*)

The Smoke Testing Sub-Test Execution will automatically be created in the Sub-Tasks section upon opening the User Story and will be run according to 03. Smoke Tests Execution

In case the US does not already have a created sub test execution, it is added according to 00. Create Sub test execution and Transition .

To fill in the time destined for testing, the tester will access the Edit option from the Smoke Testing section and modify the Original Estimate field from the initial value of 1 minute to the estimated time for testing.

Smoke Testing will be the first testing stage in the PRL test environment as follows:

- In the PRL test environment all LXs created in X-Ray in the dedicated folders of each Scrum with the name "Smoke Tests" will be executed.
- On each Epic/ICD there must be at least one executed "Smoke Tests" type Sub Test Execution.
- Smoke tests will be run after the deployment of each new build on Prelive.
- It is not run on each individual User Story.

In the situation where multiple developments (User Story; Task under an Epic or ICD) are delivered simultaneously, a single Smoke type testing will be performed.

- Smoke tests can be included in a single User Story\Task, in a single "PRL Smoke Testing" type sub-test execution
- In case Smoke testing is not performed on a User Story, the "PRL Smoke Testing" Sub Test Execution will be put in Aborted status, leaving a comment with the User Story number on which it was tested.

If a step from the Smoke testing failed, the following apply:

- Smoke testing is considered failed (Failed) — the purpose of Smoke testing is to verify if the environment and main functionalities are stable; if an LX is "Failed", detailed testing is not continued.
- In case of identifying Bugs during the execution of Smoke scenarios a BUG type Issue is opened following the steps here. The step on which it was identified being marked with FAIL.
- After the bug has been retested and the fix has been resolved, the step on which it was identified is modified to PASS status.
- Bugs reported following Smoke testing must have the "Smoke" label and the "Development environment" field filled with the value "Prelive". In exceptional cases, if the bug was not fixed there must be a comment at the US level requesting conditional acceptance from the tester to the @PO with open bugs for promotion into Live.
- Additional testing on that build in the prelive environment is interrupted, until the defect is fixed and Smoke testing is resumed.
- After fixing, the Smoke testing is resumed in full, to confirm the environment is stable again.

Only the LXs that will be run are added, LXs or test steps with Aborted status are not accepted.

Evidence is no longer attached on the LX steps from Sub test Execution.

04. PRL (Prelive) Testing (*)

The PRL Testing Sub-Test Execution will automatically be created in the Sub-Tasks section upon opening the User Story.

In case the US does not already have a created sub test execution, it is added according to 00. Create Sub test execution and Transition (*) .

To fill in the time destined for testing, the tester will access the Edit option from the PRL Testing section and modify the Original Estimate field from the initial value of 1 minute to the estimated time for testing.

Cloning the UAT Sub Test Execution and modifying the name is not accepted. In exceptional cases, if cloning is performed, the LXs that are not executed are obligatorily deleted and only those that will be executed are left on the Sub Test Execution.

In the PRL (Prelive) test environment the following will be executed:

- Happy path LXs (at least 1) - adding the LX/LXs is done as here.
- It is mandatory to retest bugs (discovered in UAT) with Major, Blocker severity, Bugs with other severity types are NOT retested
- For bugs identified in UAT, it is Mandatory to retest and check the check box - Retested in PRL
- In case of identifying Bugs during the execution of scenarios in PRL a BUG type Issue is opened following the steps here.
- The step on which it was identified being marked with FAIL status.
- Mandatory to fill in upon opening the BUG the PRL development environment.
- After fixing the Bugs the tester has the obligation to retest them according to the workflow defined in 04. Retesting a bug fix (*)
- After the bug has been retested and the fix has been resolved, the step on which it was identified is modified to PASS status.
- Bugs reported following PRL testing must have the "Development environment" field filled with the value "Prelive". In exceptional cases, if the bug was not fixed there must be a comment at the US level requesting conditional acceptance from the tester to the @PO with open bugs for promotion into Live
- At the end of testing, all steps on the scenarios added to the Sub test Execution must have the PASS status.

Testing on the Sub-Test Execution can only begin if the Smoke Testing task was performed and all steps have the PASS status.

Only the LXs that will be run are added, LXs or steps from LX with Aborted status are not accepted, in exceptional cases, if a step has an Aborted status, there must also be a justification on the respective step in the comments section.

If the LX is not relevant to be tested in the PRL test environment, it is not added to the Sub Test-Execution.

Evidence is no longer attached on the LX steps from Sub test Execution.

05. Regression Testing (*)

The Regression Testing Sub-Test Execution will automatically be created in the Sub-Tasks section upon opening the User Story and will be run according to 04. Regression Tests Execution (*)

In case the User Story does not already have a created sub-test execution, it will be added according to 00. Create Sub-Test Execution and Transition.

To fill in the time destined for testing, the tester will access the Edit option from the Regression Testing section and modify the Original Estimate field from the initial value of 1 minute to the estimated time for testing.

Regression Testing will represent the last testing stage in the PRL test environment, thus:

- In the PRL test environment the LXs created in X-Ray, in the dedicated folders of each Scrum, with the name "Regression Tests" will be executed.
- On each Epic/ICD there must be at least one executed "Regression Tests" type Sub Test Execution.
- Regression Tests will be run after finalizing the testing in PRL.
- It is not run on each individual User Story

In the situation where multiple developments (User Story; Task under an Epic or ICD) are delivered simultaneously, a single Regression type testing will be performed.

- Regression Tests can be included in a single User Story\Task, in a single sub-test execution named "PRL Regression Testing".
- In case Regression testing is not performed on a User Story, the "PRL Regression Testing" Sub-Test Execution will be put in Aborted status, adding a comment with the User Story number on which the testing was performed.
- In case of identifying Bugs during the execution of regression scenarios a BUG type Issue is opened following the steps here. The step on which it was identified being marked with FAIL status.
- If there is a test scenario with „Failed” status, there must be a reported Bug with the „regression” label and the "Development environment" field filled with the value "Prelive". In exceptional cases, if the bug was not fixed there must be a comment at the US level requesting conditional acceptance from the tester to the @PO with open bugs for promotion into Live.
- After fixing the Bugs the tester has the obligation to retest them according to the workflow defined in 04. Retesting a bug fix (*)
- After the bug has been retested, the step on which it was identified is modified to PASS status.

Only the LXs that will be run are added, LXs or test steps with Aborted status are not accepted

If the LX is not relevant to be tested in the PRL test environment, it is not added to the Sub Test-Execution

Evidence is no longer attached on the LX steps from Sub test Execution.

03. Bugs Workflow (*)

To identify a bug, the following steps are followed:

- The problem that appeared in the scenario validation, which does not coincide with the flow described in the specifications, is identified and tracked.
- Attempting to reproduce the bug by repeating the steps that led to the error's appearance. The goal is to observe if and when the bug is reproduced. If the problem continues to reproduce, a bug is opened in Jira.
- The bug is verified on multiple application versions, on multiple clients, and is tested under multiple conditions to confirm it reproduces in various situations.
- The steps followed, the exact details entered (client ID, what fields were filled, etc.) are noted according to the documentation 03. Reporting a Bug in Jira (*)
- After the identification and opening of the bug, the developer will take it over and handle its fixing. At the end, the developer will set the Status to "Resolve" and detail the cause in the "Cause" field before closing the bug.

More specifically, the bug goes through the following stages:

- **Bug reporting** – Jira is used for bug reporting and the fields on the bug reporting page are filled in according to 03. Reporting a Bug in Jira (*)
- **Bug investigation** – the responsible person is the assigned developer. They reproduce the bug, analyze it, estimate the effort for fixing it, and transmit to the tester if more details are necessary;
- **Bug fixing (Fix)** – the developer writes the remediation code;
- **Bug testing/verifying** – The responsible person is the one who opened the bug (reporter). In exceptional situations, the Product Owner can designate another person from the team to retest the bug. This is done by modifying the Assignee.

The person who opened the bug ensures that it has been fixed, retesting the affected scenarios and including regression testing, according to 04. Retesting a bug fix - QA - Confluence. If the bug persists, it is reopened.

01. What is a Bug (*)

A bug is an error, defect, or imperfection in the design, implementation, or operation of a software program or a technical system.

Bugs can appear for various reasons, such as:

1. Programming errors: Mistakes in the code written by developers.
2. Logic problems: The code works according to the instructions, but the implemented logic is incorrect.
3. Specification errors: Differences between what was requested to be implemented and what was actually created.
4. Unexpected conditions: The system or software encounters scenarios it was not designed to handle.
5. Interface errors: Integration problems between different modules or systems.

01. Types of bugs / "Category" Field (*)

When opening a bug in Jira, it is necessary to fill in the Category field to appropriately classify the identified bug.

Attention must be paid to modifying the initial Category with which the bug was opened. In case the developer changes the bug's category e.g. from Bug to Non-bug, it is indicated that the Tester notifies their Team Leader.

Each category of bug identified in Jira has a different behavior and can appear in different situations. Below you can find a synthesis of each option that can be selected in the bug Category field:

Bug category	Description	Who can set the category
BUG	It is a real error in the application that affects functionality, performance, or user experience. Ex: A form that does not submit correctly.	Tester Developer
Gap	It is a missing or unimplemented functionality, which should have existed according to the business process. It can appear due to incomplete communication between the business, analysis, development, and testing teams. It is not necessarily an error, but a lack of coverage in the requirement. Ex: In the MBK application, the user can view the balance, but does not have the option to download an account statement — although this function is essential for the user. Ex.: An approval process has 3 steps in reality, but the application completes only 2. It is discussed with the scrum team to confirm it is a real oversight and it is decided whether to create a new requirement or update the existing specifications in case the project deadline is not modified. The development and testing of the new requirement is planned in a future sprint.	Tester Developer
Settings	Problems caused by incorrect configurations or lack of configuration (e.g.: permissions, parameters, environment). Example: The application does not work correctly due to a wrong setting in the configuration file.	Tester Developer
Non-bug	Situation reported as a bug, but which proves to be intentional behavior or according to the specifications. Example: An error message that correctly appears in case of an invalid input.	Developer

Bug category	Description	Who can set the category
Incomplete specifications	The requirements or documentation do not provide enough information to correctly validate the application's behavior. Example: It is not clear how a function should behave in certain scenarios. More information is detailed in the dedicated page The bug from the Incomplete specifications category	Tester
Development during testing	The tested functionality is still under development, so the behavior can be unstable or incomplete. Example: A page is tested that does not have all the components integrated yet.	Tester
Modified technical solution	The application's behavior changed as a result of a technical decision, without this modification being added to the specifications.	Tester Developer
Duplicate	Bug already reported previously; it is a duplication of an existing error. Example: Two testers report the same problem at different times.	Tester Developer
Live behavior	Behavior already existing in the live version of the application, so the bug is not produced by a recent modification. Example: An error that also appears in live, not just on the test environment. For this type of bugs, it will be discussed in the scrum team if a new user story is opened.	Developer
Legacy	Inherited behavior from old versions of the application, which has not been corrected until now. More details about this bug category can be found here	Tester Developer
Improvement	It is not an error, but a suggestion for optimizing or improving the functionality or user experience.	Tester Developer

Bug category	Description	Who can set the category
Performance	The code works correctly, but moves slowly or consumes many resources. Ex: slow queries, inefficient loops. More details about this bug category are located here.	Tester Developer

02. Examples of bugs (*)

To give you a clearer idea of how a bug must be structured, you can find an example below.

Very important!

When detailing a bug, it is mandatory to correctly and completely fill in the fields containing the bug's details according to the complete documentation from section 03. Reporting a Bug in Jira (*). Among the most important fields to fill in so that the problem is understood as well as possible by the developer are:

- Bug detailing
- Steps to reproduce - in which it is mandatory to add:
 - Actual result - what happens in reality, the error identified at the moment the test is executed.
 - Expected result - the expected result. What is desired at the moment of executing the test, according to the requirement.
- Image with the identified error

03. Reporting a Bug in Jira (*)

In Jira, there are 2 methods for reporting a bug, given by the situations in which it is identified:

- Reporting a bug from the LX execution flow within a User Story. The workflow is documented here
- Reporting a bug within the Scrum project, without executing an LX. The workflow is documented here

For each bug category, it is mandatory that the information from the displayed pop-up be filled in according to the flow documented on the individual pages.

The time spent for opening the bug will be logged on the Testing sub test execution where the bug was identified (UAT/PRL/Smoke/Regression Testing). When the bug is retested, the time spent for retesting will be logged on the bug.

ATTENTION!

If multiple distinct problems are identified within the same flow, each bug must be opened separately in Jira. This working procedure helps with:

- correct prioritization of each problem that appeared
- allocating the bugs to multiple developers
- tracking progress and fixing each bug.

The easiest way we can tell that the identified bugs are distinct is by checking the steps to reproduce. If different steps must be followed to reproduce a bug compared to another problem in the same flow, a separate bug will be opened.

Bug reporting during the execution of an LX (*)

The most frequently encountered method of identifying and reporting a bug is the one in which the bug is discovered at the moment of executing an LX at the Sub Test Execution level within the User Story.

If the result on a step is not according to the Expected result, meaning it is Failed, a Bug will be opened:

- The bug is automatically linked to that LX and Test Execution.
- By pressing the + button next to the Defects field belonging to the test step

A pop-up will open which must be filled in with the details identified in the bug.

The displayed pop-up will be filled in according to the table below:

Field	Instruction/ Value to be filled in
Project	The Scrum where the bug was discovered. Even in situations where the bug is Legacy and will be linked to a general bugs Epic, the Project field will be filled with the scrum in which the testing was done.
Issue Type	BUG
Summary	Short and descriptive title for the bug
Assignee	The name of the developer who handled the development of the requirement (US). In case a developer to whom the bug should be assigned is not identified, Assignee will be filled with the Scrum Master's name, who will discuss the developer's prioritization of the bug in recurring meetings
Category	BUG
Reproducibility	There are 2 values in the dropdown list that are used: Always and Sometimes. Depending on the reproducibility of the discovered bug, we choose one of the 2 options. Always - This status is used when the bug can be reproduced accurately every time, following the same steps

Field	Instruction/ Value to be filled in
	Sometimes - This status is used when the bug appears only under certain conditions but not every time and may depend on variable factors.
Severity	The severity of a bug indicates the technical or functional impact of the problem on the application or users. Choosing the appropriate severity is essential for correct prioritization of bugs and efficient organization of resources. Trivial - Used for minor problems that have a negligible impact on functionality or user experience. The bug does not significantly affect the application's use and does not require an urgent fix. Usually, this type of defect is cosmetic (e.g., minor display errors or incorrect texts), does not interfere with the main flows of the application, and has no impact on performance. Minor - Used for problems that have a low impact on functionality or user experience. The bug does not prevent normal use of the application. Medium - Used for problems that have a moderate impact on functionality or user experience. The bug does not block normal use of the application, but can affect certain functionalities or create inconveniences for users. The problem is not critical, but requires correction to improve the application's performance. Major - Used for problems that significantly affect functionality, but do not completely block users. These are priorities for resolution, but the application remains usable under certain conditions. Used when a critical function does not work correctly, but workarounds exist, the impact on users is significant, but not completely blocking. Blocker - Used for critical problems that completely block the use of a function or the entire application. These bugs require immediate resolution. Used when an essential functionality for the application is completely non-functional, users cannot access or use the application at all, or the problem leads to data loss.

Field	Instruction/ Value to be filled in
Influenced applications	The information is taken from the user story, from the Requirement section → Influenced applications.
Development environment	Filled depending on the test environment in which the respective bug was found, UAT or PRL
Bug detailing	A detailed explanation of the problem.
Steps to reproduce	Step-by-step. Details on how the problem appears. Clear and ordered steps are added + if applicable, the link where the error was found, user, user role, etc. Also in this section must be included the Actual Result (what happens in reality) and Expected Result (what was supposed to happen)
Labels	The information is taken from the user story, from the Details section → Labels
Linked Issues	Relates to/blocks* → *Blocks. we use it when we have Blocker in the Severity field. The field is mandatory and must be filled in with the reference User Story.
Issue	US number
Sprint	-
Component/s	-
Attachment	It is mandatory to add a representative image where the incorrect functioning can be observed
Reporter	Filled automatically with the name of the person who identified the bug.

Bug reporting at Scrum/Jira Project level (-)

Reporting a bug directly from the Scrum project is done when the bug is not linked to either an LX or a User Story. In this case, the bug becomes the main source of information. It must be written as completely as possible so that the team can understand, reproduce, and treat the identified problem.

Attention!

Bugs that do not have an LX or User story will remain separately as items in the backlog and will require planning from the scrum team.

If it is decided that the bug should be transformed into a Change Request / User Story, the Tester will obligatorily open another bug. This will represent the initially opened bug, which was converted into a US. The new bug will be linked to the newly created User Story.

How to open a bug at the Scrum/Jira Project level:

From any page of the Scrum where the bug was identified, by pressing the Create button from the Menu bar:

A pop-up will open containing the fields that must be filled in:

Field	Instruction/ Value to be filled in
Project	The Scrum where the bug was discovered. Even in situations where the bug is Legacy and will be linked to a general bugs Epic, the Project field will be filled with the scrum in which the testing was done.
Issue Type	BUG
Summary	Short and descriptive title for the bug
Assignee	In case a developer to whom the bug should be assigned is not identified, Assignee will be filled with the Scrum Master's name, who will discuss the developer's prioritization of the bug in recurring meetings
Category	A value from the drop-down list is selected, depending on the type of identified bug
Reproducibility	There are 2 values in the dropdown list that are used: Always and Sometimes. Depending on the reproducibility of the discovered bug, we choose one of the 2 options. Always - This status is used when the bug can be reproduced accurately every time, following the same steps Sometimes - This status is used when the bug appears only under certain conditions but not every time and may depend on variable factors.
Severity	The severity of a bug indicates the technical or functional impact of the problem on the application or users. Choosing the appropriate severity is essential for correct prioritization of bugs and efficient organization of resources.

Field	Instruction/ Value to be filled in
	Trivial - Used for minor problems that have a negligible impact on functionality or user experience. The bug does not significantly affect the application's use and does not require an urgent fix. Usually, this type of defect is cosmetic (e.g., minor display errors or incorrect texts), does not interfere with the main flows of the application, and has no impact on performance. Minor - Used for problems that have a low impact on functionality or user experience. The bug does not prevent normal use of the application. Medium - Used for problems that have a moderate impact on functionality or user experience. The bug does not block normal use of the application, but can affect certain functionalities or create inconveniences for users. The problem is not critical, but requires correction to improve the application's performance. Major - Used for problems that significantly affect functionality, but do not completely block users. These are priorities for resolution, but the application remains usable under certain conditions. Used when a critical function does not work correctly, but workarounds exist, the impact on users is significant, but not completely blocking. Blocker - Used for critical problems that completely block the use of a function or the entire application. These bugs require immediate resolution. Used when an essential functionality for the application is completely non-functional, users cannot access or use the application at all, or the problem leads to data loss.
Influenced applications	The application in which the bug was identified
Development environment	The test environment where the respective bug was found, UAT or PRL
Bug detailing	A detailed explanation of the problem.

Field	Instruction/ Value to be filled in
Steps to reproduce	Step-by-step. Details on how the problem appears. Clear and ordered steps are added + if applicable, the link where the error was found, user, user role, etc. Also in this section must be included the Actual Result (what happens in reality) and Expected Result (what was supposed to happen)
Labels	Labels related to the identified problem are noted, to classify the bug as correctly as possible.
Epic Link	In case there is no Epic in the scrum for the bugs that are not linked to a User Story, the identified bug is linked to the Legacy bugs Epic of the Testing team https://jira.libra.bank/browse/STAM-161
Linked Issues	-
Issue	Left blank because it was not identified during the testing of a User Story
Sprint	-
Component/s	-
Attachment	It is mandatory to add a representative image where the incorrect functioning can be observed
Reporter	Filled automatically with the name of the person who identified the bug.

04. Bug Categories (*)

Bugs can be classified depending on their nature, impact, moment of identification, or cause.

Also, bugs identified in the tested applications can be opened differently in Jira, depending on the moment of their identification and their typology.

The most frequently encountered types of bugs met during the activity are:

- Performance Bug
- Improvement Bug
- Legacy Bug
- Regression Bug

- Smoke Bug
- Incomplete specifications Bug
- Ui/UX Bug

In each dedicated chapter, you will learn what each type of bug represents and you will find out techniques for opening them.

04. Retesting a bug fix (*)

After the developer has confirmed fixing a bug in the test environment, the Tester has the obligation to retest the bug and verify if the previously signaled problem has been fixed.

What aspects must be considered when retesting a bug:

- Follow the “Steps to reproduce” detailed in the bug;
- Choose the appropriate test environment for retesting the bug, depending on what was noted in the “Development environment” section;
- Verify if the bug is still reproduced after the fix, following the execution of all reproduction steps.

Depending on the result following the bug retesting, the steps below are followed:

The bug was fixed:

- It is confirmed that the testing result complies with the specifications;
- If relevant, a short negative testing is done to ensure no unforeseen aspects appear;
- The bug's status is modified from In testing to Finalized.
- In case the defect was opened based on an execution: the step's status is modified from Fail to Pass.

The bug is still reproduced:

- The bug's status is modified to Re-open;
- A comment is left on the bug attaching evidence that the bug is still reproduced;
- The bug will be taken over by the developer for fixing.

Retesting bugs in Prelive:

During Prelive testing, the Tester has the obligation to retest the bugs identified in UAT testing, which had Major or Blocker severity.

To validate that Major or Blocker bugs have been retested in PRL, the Tester will manually check the "Was it retested in Prelive?" checkbox, checking the "Yes" value.

The Was it retested in Prelive? checkbox:

- Is available at the BUG issue type level;
- Can be accessed by pressing the Edit option;
- The Tester will select the "Yes" value after retesting the bug in PRL;
- It is no longer necessary to add images in case the bug does not reproduce in PRL.

If the bug with Major/Blocker severity identified in UAT is reproduced when retested in PRL:

- It is reopened by selecting the "Re-open" status;
- A new printscreen is added, in which it is mentioned that the bug is reproduced in PRL;
- The bug will be taken over by the developer for fixing.

In exceptional cases where the Product Owner decides that a bug identified in UAT or PRL will not be fixed:

- it is mandatory for the Tester to request confirmation from the Product Owner that the bug will not be treated in the test environment where it was identified.
- The Tester will leave a comment on the User Story like "the bug/the following bugs were not tested in UAT/PRL. I request acceptance for continuing testing in the next test environment / transitioning into Ready for release."
- The Product Owner will return with a comment giving their OK.

Note!

The time spent for retesting the bug will be logged on the Bug

05. Jira Bugs Workflow (*)

Open

The bug was identified and reported by the Tester, but has not been taken over by the developer.

In progress

The developer has taken over the bug and it is being worked on. The responsible person begins to investigate the cause of the bug.

Ready for testing

This status is set by the developer when they have fixed the reported bug.

The developer must select the correct category in which the bug falls, choosing one of the existing values in the dropdown list: bug, change request, settings, non-bug, incomplete specifications, development during testing, technical solution change, duplicate.

Category details:

- Bug = development error
- Settings = will be selected when the development didn't work due to an unstarted service, unallocated rights
- Non-bug = an issue that was opened as a bug, although this was normal behavior
- Incomplete specifications = when the dysfunctionality is due to not treating the respective case in the specifications document
- Development during testing = when the dysfunctionality appeared due to development while testing
- Technical solution change = when the dysfunctionality appeared due to changing the technical solution, following the testing results
- Duplicate = the same dysfunctionality has already been signaled

In addition, the developer must detail the cause, in the Cause field, before closing the bug.

In testing The tester takes over the bug for retesting after the fix. The tester must ensure that the request categorization is correct. Otherwise, they modify it before finalizing the request.

The tester confirms that the bug is resolved and that the solution works correctly. Treating and resolving the bug according to expectations will be confirmed by pressing the “Finalized” button.

The final status of the bug will be FINALIZED

If the bug was not fixed following the checks performed by the Tester, they will return the bug to the developer by pressing the Re-open button. The developer will take over the bug for fixing.

Other statuses:

If the developer finds that the description of the encountered error is incomplete, the requirement will be sent back to the Tester by pressing the "Return" button. The next status will be Returned.

In the situation above, the tester who reported the bug will bring the necessary additions and will send the bug to the developer by pressing “Respond”. The next status will be Responded. The bug will be treated again starting from the In progress status.

On Hold can be set when the bug is put on hold until finding a solution

Cancelled can be set when the bug is canceled for various reasons.

The transitions on statuses are highlighted in the diagram below:

04. Incidents Workflow (*)

The tester tests the incidents for which they are the Reporter or which are assigned to them by the Product Owner of the team they are part of.

What tasks must be created for Incidents:

- **Defining scenarios (sub-task)** - assignee to the Tester who will define the test scenarios. Test scenarios for Incidents do not need to be validated by the Product Owner.
- **UAT Testing (sub-test execution)** - the task is mandatory;
 - assignee to the Tester who will test in UAT;
 - if multiple testers will test on the same Incident - each tester will make their separate sub-test execution.
- **PRL Testing (sub-test execution)** - the task is mandatory;
 - assignee to the tester who will test in PRL;
 - if multiple testers will test on the same Incident - each tester will make their separate sub-test execution.

For testing incidents, both in the UAT test environment and in PRL, evidence will NOT be added following the performed testing.

It is recommended that Incidents NOT be estimated in Story Points. In case the testing is lengthy, it can be decided in the scrum team that the Incident be transformed into a User Story.

Resolving incidents can be done directly on Live under the following conditions:

- major impact on the business, where resolution time is essential
- there is no test environment or data for replication, where the problems appear only in production and cannot be replicated on the test environment
- if it is a punctual correction (workaround), where a correction patch already validated on a test environment exists (the specific case has been encountered before), this can be applied directly on live until the cause is resolved (which may take time).

Under other conditions, major / urgent incidents are resolved on PRL, validated and applied on live.

00. What is an Incident? (*)

An incident represents any unplanned interruption of an IT service or reduction of its quality. The incident is identified in Live (not on the test environments) by a user, a Tester, a developer or a monitoring system.

Examples of incidents:

- The server is down.
- The application does not respond.
- The service performance is very low.
- Users cannot authenticate.
- Online payment failed for users.

- The displayed data is incorrect or incomplete.

At the moment of identifying an Incident, a ticket is opened in Jira, of the Incident type. The incident is officially recognized by the team as being real and affecting the service and the users.

Process of managing an IT incident in Jira:

1. **Reporting** – Someone observes the problem and creates a ticket in Jira.
2. **Registration** – The incident is opened in Jira.
3. **Classification and prioritization** – The severity and impact are established.
4. **Assignment** – The incident is assigned to a developer or the support team.
5. **Investigation and remediation** – The cause is identified and the solution is applied.
6. **Testing / confirmation** – It is verified if the service works again.
7. **Resolution and closure** – It is confirmed that the incident has been fixed and it is closed.

01. Defining Test Scenarios for Incidents (*)

The same steps are followed as in the documentation 02. Manual Test Scenarios Definition Workflow (*)

Atypical situations for defining test scenarios for an Incident:

- An attempt is made to identify an LX suitable for the flow that will be tested.
- In case an already created LX is not identified, a new one is created. It is recommended to create new LXs only if the scenario described in the ticket does not already have a written LX - the minimum requirement is 1 positive LX.
- The created LX will have to contain the "Incident" Label to be able to differentiate between an LX resulting from testing an Incident and one resulting from testing a User Story.
- The LX will be linked to <https://jira.libra.bank/browse/STAM-157>
- The newly created LX will be added into an existing subfolder that describes the tested flow.
- Otherwise, and if there is no subfolder with the tested flow within the Incident, a new subfolder "Incidents Scenarios" will be created.

Scenarios for an Incident do not require validation by the Product Owner.

02. Executing Test Scenarios for Incidents (*)

The test scenarios for Incidents will be executed in both test environments, UAT and PRL, by adding the LXs in the Sub Test Execution according to the workflow from chapter 02. Executing Test Scenarios Workflow (*)

- **UAT Testing** – a Sub-test execution will be created - the sub-task is mandatory;
 - assignee to the Tester who will test in UAT;
 - if multiple testers will test on the same Incident - each tester will make their separate sub-test execution.
 - if it is found that the Incident has not been fixed in UAT, a comment is left mentioning this fact and the Incident is reopened for fixing.
 - for the situation above, an image is added to demonstrate that the Incident is still reproduced.

- **PRL Testing** - a Sub-test execution will be created - the sub-task is mandatory;
 - assignee to the tester who will test in PRL;
 - if multiple testers will test on the same Incident - each tester will make their separate sub-test execution.
 - if it is found that the Incident has not been fixed in UAT, a comment is left mentioning this fact and the Incident is reopened for fixing.
 - for the situation above, an image is added to demonstrate that the Incident is still reproduced.

For testing incidents, both in the UAT test environment and in PRL, evidence will NOT be added on the execution of the LX steps following the performed testing.

For Incidents, the Demo meeting is not held.

03. Jira Incidents Workflow (-)

The Incident is no longer reproduced and has been resolved correctly:

- The tester modifies the status from Requires confirmation into Ready for release.

The Incident is still reproduced:

- The Incident's status is modified to Re-open;
- A comment is left on the Incident attaching evidence that the Incident is still reproduced;
- The Incident will be taken over by the developer for fixing.

The workflow for treating an Incident is found below:

Reporting an Incident (*)

For any dysfunctionality observed in Live, an Incident will be initiated in production in the Production project in Jira, not a bug.

The Incident will not be reported by the Tester, but at the request of the team's Product Owner, they can open the Incident in Jira.

05. Working with Xray Library (Xray Test Repository) (*)

X-ray is a testing Add-on (submodule/tool), integrated in Jira. The purpose is to create a library of updated scenarios that can be reused in future testing.

Accessing the Xray Library

The user logs into Jira by accessing the link: <https://jira.libra.bank> . Then writes xray in the Search field and selects Xray Library (LX) from the Projects section of the drop-down list, accessing Xray Test Repository.

In the Xray Test Repository, test scenarios (LXs) are created in two ways:

- automatically, following the instructions presented in 01. A AI test scenario generation workflow - UIPath (draft) - QA - Confluence .
- manually, following the instructions presented in 02. Identifying and creating tests in Xray - QA - Confluence.

Regardless of the method of creating test scenarios, the storing/depositing method in the Xray Test Repository will be respected, according to Rules\Structure for the Xray Library (*) - QA - Confluence.

Creating folder/subfolders in Scrum projects

How to add a new subfolder

1. Select the parent folder with the scrum's name and press the “+” icon at the bottom of the page.
2. Modify the name of the sub-folder from “New folder” to the name of the flow, functionality or the name of the section from the tested application.

to add a mention that they cannot delete folder/subfolder - must ask TL for support.

Rules\Structure for the Xray Library (*)

In the Xray Test Repository, tests are kept organized in dedicated subfolders opened within the Scrum team's parent folder. The opening of these subfolders is presented here.

The types of subfolders that must be found within each Scrum team's parent folder are grouped as follows:

- flows/functionalities/sections subfolders
- subfolders containing smoke tests
- subfolders containing regression tests
- subfolders containing tests automatically generated with UIPath Test Manager
- subfolders containing archived tests

Rule! Tests are always created and kept in the dedicated subfolder, and not in the parent folder. !!!

These tests, which are erroneously found in the parent folder, will be selected (multiple selection via Ctrl+left mouse click) and will be moved, most simply, through the drag&drop method, into the subfolder dedicated to the respective targeted area/flow/functionality:

!!! In the same way presented above, tests generated automatically with Test Manager - UIPath will be moved, from the Test Manager Drafts subfolder to the subfolder dedicated to the respective area/flow/functionality within the parent folder of a Scrum team.

It is the responsibility of the testers to constantly ensure that there are no tests saved/created directly in the parent folder of the Scrum team in which they carry out their activity.

Flows/functionalities/sections subfolders

This type of subfolders are created / will be created (as the case may be), immediately under the parent folder of the Scrum team, according to the steps here.

It is recommended that the tester:

1. searches in the team's confluence area (example Scrum Wallet at Libra Journey (Wallet) - Libra Journey (Wallet) - Confluence, in the image below); if it exists, subfolders are created based on the structure
2. If the area from step 1 does not exist, the subfolder structure and subfolder naming should be established within the team, together with the BA/PO.

The subfolders are named after the name of the flow, functionality or section of the tested application.

Within these subfolders are created / added (moved) only tests with a status different from Archived, which test the flow, functionality or section from the subfolder's name.

Smoke Tests Subfolder

This type of subfolder is created / will be created automatically, immediately under the parent folder of the Scrum team, according to the steps here.

Within these subfolders are created / added (moved) only smoke tests with a status different from Archived. Smoke tests are defined here.

Regression tests subfolders

This type of subfolders are created / will be created automatically, immediately under the parent folder of the Scrum team, according to the steps here.

The subfolders are named Regression tests.

Within these subfolders are created / added (moved) only regression tests with a status different from Archived. Regression tests are defined here.

IMPORTANT!

within this subfolder it is necessary to create subfolders based on the structure from here.

Test Manager Drafts Subfolder

This type of subfolder is created / will be created automatically, immediately under the parent folder of the Scrum team.

1. Within these subfolders, the tests generated in UIPath Test Manager are automatically added according to the workflow here.
2. After their update, if applicable (eliminating/adding/updating steps, modifying steps order, adding labels, etc.), these tests will be manually moved by the user into the corresponding subfolder from the Scrum team's parent folder.

Archived tests subfolders

This type of subfolders are created / will be created (as the case may be), within the parent folder of the Scrum team, according to the steps here.

The subfolders are named Archived.

Within these subfolders are added (moved) only tests with Archived status. Details about archiving tests can be found here.

Filtering LXs in Xray

There are situations where:

- upon manual initiation of a test (LX), filling in the field was omitted / the Test Repository Path field was saved with erroneous values
- after initiation, it was erroneously moved into a subfolder
- the user wants the quick identification of a certain test

For the quick identification of the sought tests (and where it is required according to the rules here, their movement into the correct subfolder), the filtering tool will be used:

- the user will position themselves on the All folder and press the Filter button
- For the fastest / most exact identification, the Advanced option will be used, where the user creates their best filter/query; searches the test by a keyword from the name (summary) and the name of the one who created the test and the calendar date/period in which it was created and test status; practically any additional filter leads to a faster identification of the sought test:

example: type = Test and Summary ~ rating AND Reporter = firstname.lastname AND CreatedDate <= now() AND Status = "Finalized scenarios" ,

02. Regression Tests Workflow (*)

Regression testing is a type of software testing that aims to verify whether recent changes in the code (such as adding new features, fixing errors, or modifying requirements) have not negatively affected the existing functionalities of the application. Thus, it ensures that the software continues to function as expected after updates.

Main characteristics of regression scenarios:

- They cover critical flows and essential functionalities to ensure the stability of the application after modifications.
- They are periodically updated to reflect the real state of the application and to eliminate tests that are no longer up-to-date.
- They have a clear structure and correct labeling (label „Regression”, organized by flows).
- Any type of positive, negative, and UI/UX tests can become a good candidate for regression.

Steps for defining regression scenarios:

1. Each team must define its own regression scenarios, depending on the specifics of the tested application.
2. Before creating new scenarios, check the Xray folders if there are already written tests that can be reused. Depending on the situation, we have 4 options:
 - **Variant 1:** Each team goes through the folders in Xray containing tests marked with the Regression label and evaluates:
 - If the Test covers the complete flow → it is moved to the subfolder corresponding to the flow from the regression tests
 - If the test can be completely executed → it is kept
 - If there are differences between the test and the current flow (AS IS) → additions are made
 - If the test partially covers a flow, the test is cloned and the steps are deleted or necessary additions are made to correctly reflect the regression flow.
 - If the test is no longer up-to-date, its status is changed to "Archived"
 - **Variant 2:** Check in the Xray library by the flow name and if it can be used
 - Add label: "regression"
 - Set Priority = set according to 04. Regression tests priority (*)
 - **Variant 3:** if you find nothing relevant, create the scenario from scratch, with the same rules:
 - label= regression, Priority= set according to 04. Regression tests priority (*)
 - **Variant 4:** Upon completing the testing of a new development at the US/EPIC level
 - it is evaluated if the steps from the already tested LX are relevant for a new regression testing
 - All steps are relevant: add the regression label and move to the dedicated folder.
 - Certain steps are relevant: the LX is cloned and the steps are updated (added/deleted).

- label= regression, Priority= set according to 04. Regression tests priority (*)

Eligible types of tests for regression

- Positive tests
- Negative tests
- UI/UX tests
- Any type of test covering critical functionalities can become a candidate for a Regression Test

Running regression tests * Test environment restoration (UAT/PRL)

- Major Release (live launch of an important project/new application)
- Minor Release (live launch of a new development, Epic/ICD)
- Major code changes, library updates, optimizations, system integration

Below is information about the execution of regression tests:

Situation	Description	Execute?	When?
Test environment restoration	Test environment restoration: UAT and PRL	Yes - Partial regression on Urgent and High priority LXs	After finishing the test environment restoration, after the successful execution of Smoke tests
Major Release	Release (Launch in live) for an important project with impact in multiple systems/applications or a new application (e.g.: Flow, TPH, AA)	Yes - Complete regression	After deploying all requirements in PRL, after the successful execution of Smoke tests
Minor Release	Release (Launch in live) of a new project development (e.g.: Implementation of an Epic/ICD)	Yes - Partial Regression (running should take maximum 12h)	After deploying all requirements in PRL, after the successful execution of Smoke tests
Major changes to existing code	Refactoring of the calculation method for commissions	No - Partial Regression (running should take maximum 12h)	Upon request (or included in the testing after an environment restoration)

Situation	Description	Execute?	When?
Update of libraries / external dependencies	Frontend framework update (React, Angular, etc.)	No - Partial Regression (running should take maximum 12h)	Upon request (or included in the testing after an environment restoration)
Performance optimization	Reducing response time for API requests	No - Partial Regression (running should take maximum 12h)	Upon request (or included in the testing after an environment restoration)
Modifications in backend / system integration	Integration of a new anti-fraud system	No - Partial Regression (running should take maximum 12h)	Upon request (or included in the testing after an environment restoration)

Activity reporting:

All activity defining regression scenarios will be reported in Jira in ATQ-47 - Jira project doesn't exist or you don't have permission to view it.

For the actual testing, a Sub Test Execution is opened, which will be generated automatically along with the other three and will bear the name: PRL Regression Testing.

In case a US does not require regression testing, this Sub Test Execution will be marked as Aborted.

Other important aspects:

- Not all USs will have regression tests
- Regression testing is done at the build/deployment/US level

Validation of Test cases (LXs)

- Validation of regression LXs by the PO is not mandatory
- The PO can ask for the revision of test scenarios, adding test scenarios or adding new business flows

Automation

- after they are validated, the regression test cases (LXs) become candidates for automation and enter the automation process; see "procedure for taking over tests to automation" (WIP)
- Priority for automation will have to be dictated by the manual testing team that worked on the LXs
 - one must consider how much the automation of LXs helps, taking into account the manual execution time of the tests
 - how often a manual test repeats

Maintenance of business flows, Test cases and the Regression suite

- Regression test cases will enter a continuous maintenance circuit - they will have to be reviewed periodically by the Testers
- As the application\project advances, new implementations will always come, and the test cases will suffer modifications
 - as new implementations appear, the testers assigned to the project will have to evaluate the impact on the existing regression suite
 - modifications can occur following functional changes
 - modifications can occur following technical limitations or changes
 - some tests can become irrelevant or redundant and can be closed\removed from the suite

01. What are Regression Tests? (*)

Regression tests consist of re-executing functional and non-functional tests to verify if functionalities that worked previously continue to function correctly after modifications (bug-fixes, improvements, optimizations, etc.). They are essential for ensuring the continuous stability and quality of the application over time.

Purpose of regression tests

- **Verification of existing functionalities:** Ensures that recent modifications have not negatively affected the already implemented functionalities.
- **Detection of collateral effects:** Identifies defects accidentally introduced into other areas of the application.
- **Maintaining application stability:** Contributes to delivering a stable and predictable product in production.
- **Validating integration between systems:** Confirms that interactions between applications or modules remain functional after modifications.

Why are Regression tests important?

- Ensures the application's stability after code modifications, launches, or environment restorations, preventing defects in already existing functionalities.
- Detects collateral effects of new developments or refactorings, even in apparently unaffected areas.
- Validates integration between systems in case of projects with broad impact (e.g.: Flow, TPH, AA), where a modification can affect multiple applications.
- Optimizes the testing effort by reusing existing tests and running them automatically, where possible.
- Allows flexible and efficient testing, adapted to the context: complete regression for major releases, partial regression for punctual modifications or Urgent + High priorities.
- Supports launch decisions in production, providing confidence that the application works correctly after each development stage or technical intervention.

What do Regression tests usually cover?

- The existing functionalities of the application, to verify if they continue to function correctly after code modifications, deploy, or environment restoration.

- Critical business flows, including those that cross multiple applications or systems (e.g.: Flow, TPH, AA) (account opening, user authentication and authorization, scheduled and recurring payments, currency exchange, balance inquiry, etc.)
- Areas directly or indirectly affected by new developments, refactorings, optimizations, or integration of external systems.
- Test cases with high priority (Urgent and High), especially in partial regressions.
- Interactions between modules and systems, to prevent collateral effects in dependent applications.
- The application's behavior in positive, negative, and UI/UX scenarios, depending on the identified risks.

02. Defining Regression Test Scenarios (-)

The tests included in the Regression package must meet the following criteria:

- The name of the regression LX will contain in the title the Scrum team, the name of the flow/functionality and "Regression scenario"
- It is labeled and organized appropriately - It has the regression label, is added in the correct folder/subfolder and is easily identified in Xray.
- LXs must be maintained, checked and updated
- Covers critical business flows - The test validates essential functionalities such as: payments, account opening, transfers, card administration, credits, savings, etc.
- Is current and relevant - The test reflects the current state of the application (as-is) and can be fully executed, without blockages or outdated steps.
- Is reusable and stable - The test has already been successfully executed previously and can be run repeatedly without frequent modifications.
- Is clearly documented - Includes explicit testing steps, relevant test data and well-defined expectations.

Recommendations:

-For an LX that goes through windows/screens - recommendation to have 1 LX for each screen (all menus/tabs must be gone through in a screen); if there are screens that are accessed using another LX - call test to the LXs needed to reach the screen

-For flow LXs 1 complex LX

-1 successful login LX - extract login steps from existing LXs, make a separate login LX to be called by the other tests

-1 or more LXs with rights for specific roles - for the screens/menus/tabs highlighted in the displayed page, after login

-Negative and UI/UX testing LXs

It is recommended to use the call test function as often as possible where possible.

Recommendations for regression test independence

1. **One test = one clear regression objective**

- Do not combine multiple complex checks in a single scenario (e.g. „client data modification + payment initiation + statement generation”).
- Scenarios must be separated: „client data modification test”, „payment initiation test”, „statement generation test”.

2. **No dependencies between tests**

- A regression test must not assume another test has already run (e.g. the transfer test does not assume the login test has run).
- Each test does its own setup (e.g. login, creating test data), using call test for common steps.

3. **Isolated and reproducible test data**

- Avoid using the same test data for multiple scenarios (e.g. the same bank account for all tests).
- Use automatically generated data or standardized sets, to prevent conflicts and ensure repeatability.

4. **Minimize complex integration dependencies**

- Regression tests must validate clear functionalities and flows, not cover all possible integrations in a single scenario.
- Example: an interbank transfer test validates only the transfer, not the SMS notification or integration with other systems, unless these are the objective of the test.

5. **Run tests in parallel**

- Write tests so they can be executed simultaneously, without blocking each other. If they cannot be run in parallel, there are hidden dependencies that must be eliminated.

6. **Avoid „chain” type scenarios**

- Do not create very long regression scenarios, with multiple steps dependent on each other.
- Complex end-to-end tests can exist separately, but regression must be granular and easy to diagnose.

General recommendations for regression:

- Cover critical flows and high-risk areas.
- Keep tests clear, independent, and easy to maintain.
- Do not go into margin details or complex validations, unless relevant to regression.
- Periodically review the regression package to eliminate outdated tests and add relevant scenarios.

Recommendations for organizing regression tests

- Tests must be added in Xray in the Scrum folder - "Regression tests" subfolder with an opened subfolder for each flow
- Must have label = regression
- Set Priority = set according to 04. Regression tests priority (-)

What to include in a regression test

- Verification of existing functionalities after code changes, deploy, or environment restore.

- Coverage of critical banking flows: account opening, transfers, payments, card administration, credits, savings, currency exchange, balance inquiry, generating statements, etc.
- Validation of integrations and interactions between systems (e.g.: ANAF, Credit Bureau, anti-fraud, internal modules).
- Testing both positive scenarios (happy path) and negative ones (e.g.: invalid data, missing permissions).
- Checking application stability in all relevant environments.
- Covering recent modifications and high-risk areas, including functionalities directly or indirectly affected by new developments, refactoring or optimization.

Orientation Guide for regression tests:

No.	Tested Aspect	Description	Concrete Example / Testing Steps	Expected Result
1	Coverage of critical business flows	Tests essential functionalities for business continuity	Payment initiation, account opening, interbank transfer	Prevents major defects that can affect clients or daily operations
2	Currency and relevance	The test reflects the current state of the app and can be fully executed	Test updated after the last release	Ensures correct validation of existing functionalities after changes
3	Independence	The test does not depend on running other tests and has its own setup	Client data modification test	Allows parallel running and rapid identification of failure cause
4	Isolated and reproducible test data	Uses dedicated, auto-generated data or standardized sets	Unique test account for each execution	Avoids conflicts and ensures execution repeatability
5	Coverage of positive and negative scenarios	Includes both success and error flows relevant to regression	Successful transfer and transfer with invalid data	Ensures detection of both functional and validation defects
6	Correct priority	Priority reflects the risk and impact of the tested functionality	Test for critical function = Urgent/High	Optimizes execution time and focuses on high-risk areas

No.	Tested Aspect	Description	Concrete Example / Testing Steps	Expected Result
7	Stability across all environments	Has the same behavior and results in all test environments	Balance view test	Ensures consistency and reliability in execution
8	Clear documentation	Includes explicit steps, relevant test data, well-defined expectations	Test with detailed steps and clear data	Eases test maintenance and updating
9	Correct organization and labeling	Has „Regression” label, placed in the right folder/subfolder	Test with correct label and folder	Facilitates rapid identification and selection of tests for regression execution

01. Initial Elicitation of Business Flows from the Product Owner (-)

Laura is looking at this '

Guide for Creating Business Flows for a Regression Suite

The role of the Product Owner (PO) is crucial to ensure that all critical business flows are correctly represented in the regression test suite. The purpose of this guide is to provide the necessary steps to define and create efficient business flows that cover all essential scenarios so that the Testing department can create a Regression suite and be able to maintain product quality.

A business flow is a sequence of actions that a user or a system goes through to achieve a specific goal within the application or project. It is a representation of how users interact with your product to reach a certain result.

Business Flows are essential for regression testing, because they reflect how the product's functionalities are used in real life. They ensure that the modifications brought to the application do not negatively affect the existing functionalities.

After the business flows have been finished and documented, it would be ideal for them to be listed in a structured form so that the same structure can be transposed into a similar structure of folders and test cases in Jira Xray.

The list will have to be sent to the Testing team within the project, together with any written documentation or support material for the described flows.

Suggestions for the initial documentation of Business Flows

Identifying Business Flows The first step is identifying the business flows that are essential for the functioning of the product\project.

- **Product Analysis:** Find out what functionalities are available, what are the main objectives of the application, and how it is structured from a technical and functional point of view.
- **Knowing the Users:** Identify and understand the user categories and roles (personas) that use the application. What needs and goals do they have? What tasks are they trying to accomplish by using the product? What permissions do they have?
- **Reviewing Requirements and Documentation.** Review the documentation and\or requirements to understand which functionalities are essential for the application (or project) and what expectations exist regarding performance and security. Examine the user stories and the backlog to identify the business flows that are already described in the requirements. These offer a direct perspective on the steps the users will follow.
- **Mapping Business Processes.** Use process mapping techniques, such as BPMN (Business Process Model and Notation), to create diagrams describing the specific steps in each business flow. These diagrams should include all actions, decisions, and interaction points with the system. During mapping, pay special attention to decision points, branches, and critical interaction points, because these are essential for an efficient business flow.
- **Collaborating with Stakeholders.** Collaborate with stakeholders, including the development team, the testing team, and end users, to understand the existing business flows and identify new ones. Discussions with them can reveal flows you didn't initially consider. Organize work sessions to define and validate business flows. In these workshops, gather feedback and ensure that the proposed flows correctly reflect the business requirements and the users.

- **Validating Flows.** Test and validate business flows with real scenarios, meaning how users would use the application in everyday life. This step helps to ensure that the identified flows are realistic and cover all possible cases.

The final goal is creating a structure of flows like in the table below, the goal is not the complete documentation of the flows, existing documentation will be used as much as possible.

Priority	Flow Name	Details	Documentation	Roles
1	Name Flow1	Here it will be filled with details about Flow1; important actions of the user and the intermediate or final expected result	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 1	Product Owner, Business Analysts
1	Name Flow2	Here it will be filled with details about Flow2; important actions of the user and the intermediate or final expected result	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 2	Product Owner, Business Analysts
2	Name Flow3	Here it will be filled with details about Flow3; important actions of the user and the intermediate or final expected result	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 3	Product Owner, Business Analysts
3	Name Flow4	Here it will be filled with details about Flow4; important actions of the user and the intermediate or final expected result	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 4	Product Owner, Business Analysts

Example of listing Creatio CRM business flows below

- allocating and reallocating clients (contact and LegalEntity) with different criteria:
 - active account (allocation attempt by user from the branch with the owner, allocation by superior from the same branch with the owner, allocation user and superior from another branch different from the owner's, allocation and reallocation for user from headquarters for superior of the owner)
 - customer status (allocation attempt by user from the branch with the owner, allocation by superior from the same branch with the owner, allocation user and superior from another branch different from the owner's, allocation and reallocation for user from headquarters for superior of the owner)
 - related account

- ownership status active
- + activity/opportunity registrations with ownership deadline calculation according to implemented rules
- scenarios for related account with inactive ownership status
- for opportunities - job competencies

For starters, non-functional type flows and requirements (performance, accessibility, compatibility, security) will be captured separately and will not be included in the regression tests, regression suites will be considered strictly functional.

The priority of the flows will correspond directly to the priority of the tests that emerge from the flows. More details here: 03. Regression Tests Priority

Running all tests resulting from the business flows for priority levels 1 + 2 + 3 will mean running a Complete Regression.

A partial regression can be the execution of regression tests with tests of a certain priority or a regression on a certain functional\or code area of an application.

Documenting Business Flows

For the test cases resulting from the business flows to be written as well as possible, a detailed description of the business flows and any documentation material is needed.

- **Diagrams:** Diagrams can be used (and it is recommended to use them) so that the business flows are visible. These diagrams should illustrate the main steps, decision points, and possible results. Where using diagrams is not possible, a detailed description of the flows can be used. Already existing diagrams will be used as much as possible.
- **Detailed Description:** For each business flow, clearly document each step, including user actions, expectations, and check points. Make sure the documentation is detailed enough for the testing team to be able to transform it into test cases.
- **Using User Story Scenarios:** Link business flows to user stories and functional requirements to ensure that all critical aspects are covered.

Collaborating with the Testing and Development Team

Documenting business flows must be considered a team effort and for this, communication with the project team must be constant until the final form of the flows is reached.

- **Validation Workshops:** work sessions can be organized with the testing and development team to review the proposed business flows. This helps identify potential gaps and ensures everyone has the same understanding of the flows.
- **Continuous Feedback:** Encourage continuous feedback from the testing team during the creation and updating of business flows, to ensure they can create efficient test cases.

Monitoring and Maintaining Business Flows

Once the flows have been created and documented, they enter a continuous maintenance cycle. New functionalities, changes in roles and permissions, or maybe even the disappearance of some conditions or entire flows means that the documentation and tests will have to be adapted.

- **Periodic Reviews:** Business flows must be reviewed periodically to reflect the modifications brought to the product. Make sure any change in the product is updated in the business flows documentation.
- **Adapting to Changes:** As the application evolves, ensure that the business flows are updated to include new functionalities or to reflect the modifications of the existing ones.
- If a User Story or a specific requirement is known to affect the regression business flows, this is good to be mentioned and communicated to the Testing team in the project.

Communication and Traceability

- **Ensuring Traceability:** Each business flow should be traceable to the requirements and/or user stories. This helps validate that the regression tests cover all critical requirements.
- **Efficient Communication:** Good communication must be maintained with all stakeholders to ensure that the business flows are correctly understood and implemented in the regression tests.

Validating Tests After the business flows have been transmitted to the Testing team on the project and after the tests have been written by the Testing team, they are sent for validation to the Product Owner (and to the Application Owner - where applicable).

This validation is important to ensure that the tests cover the business flows and the expected results following the running of the tests as well as possible.

The better the tests cover the business flows, the earlier important defects can be caught.

The validation process will be similar to the one for User Stories. If an LX is not considered complete or does not cover the need of the business flow, it can be sent back for revision with the appropriate mentions.

The reasons why test cases can be sent back for revision to the Tester are documented here [Scenario validation - QA - Confluence](#).

02. Evaluating existing tests for Regression (*)

Evaluating regression tests refers to how we analyze the efficiency, coverage, and impact of regression tests within the quality assurance process.

This activity can be an independent/autonomous one - in case the business flows are not documented (or there is no clarification from the PO regarding flows and priorities), the activity can be started internally.

The tests chosen for regression should include:

- Frequently used functionalities
- Modules affected by recent modifications
- Complex and integration cases
- Tests that have frequently failed previously or covered recent defects

The selection of regression scenarios is done based on:

- the risk of affecting the existing code,
- the implementation's complexity,
- the dependency on critical modules (e.g.: login, payments, dashboard),
- the history of previous defects or instabilities in the respective area.
- environment restoration
- major release
- minor release

Reusing test cases - Initial analysis of already existing scenarios

- Verify if there are already tests that cover the described business flow
- Reusing already existing tests will be attempted as much as possible. For this purpose, any folder\test repository that might contain test cases (LXs) about the business flows for an application or project will be analyzed. Generally, the evaluation\analysis will be done within the same project but reusing some scenarios from other projects is not excluded if they are considered suitable.
- the tests that can be reused can be:
 - already written tests with the purpose of a regression test - tests that have the "regression" label
 - already written tests for testing User Stories\Epics based on new functionalities\implementations.
- Some business flows might already be covered by one or more already existing LXs. Those tests must be reused as much as possible. The tests will be moved into a new folder and can be included in the Regression Suite.
 - If a business flow is already covered by 1 or more LXs these will be reused for the purpose of Regression Testing.
 - If LXs already exist for a business flow they must be analyzed\evaluated and in case they are not up to date they will have to be edited with the new implementations.

- If one or more LXs are already written and they represent prerequisites for the business flow for which the initial analysis is done, the LXs will be included using the "Call Test" functionality

03. How to create regression tests in Jira/Xray(*)

The tests are created like any other scenario according to 02. Identifying and creating tests in Xray - QA - Confluence

The tests must be added in the SCRUM of each team in a dedicated subfolder named "Regression tests"

The regression tests need Labels = regression

The regression tests will have Priority set depending on The regression tests priority

Regression is performed only for those User Stories that introduce modifications with direct or indirect impact on the existing functionalities or at the PO's request. - to review if it is needed here and not at execution

For the User Stories that isolate completely new functionalities, without impact on the existing areas, coverage through regression is not necessary, but the performed tests can be taken into account for a future regression - to review if it is needed here and not at execution

04. Regression tests priority (*)

Prioritizing regression tests when creating LXs

Business flows must be prioritized according to multiple factors among which

- frequency of use
- impact on the business
- their complexity
- coverage of the most important actions and purposes of the application
- importance for the client
- importance for the project

Critical flows, which are frequently used and have a major impact on users, should be tested with priority.

The proposal is using a priority scale based on 3 levels: Urgent(Critical), High, Normal(Medium),

Priority: URGENT

- Definition: Essential tests for the basic functionality of the application, which must run immediately to prevent major blockages.
- Includes: Critical flows (happy path), UI + backend interactions, simple security checks, visual integrity.

Priority: HIGH

- Definition: Important tests for complex scenarios or with major impact, which can affect the stability or security of the system.
- Includes: Alternative flows, data validations, roles and permissions, systemic errors, high volume performance, simultaneous transactions, audit, backup, migration.

Priority: NORMAL

- Definition: Useful tests for ensuring a complete and coherent experience, but which do not directly affect critical functionality.
- Includes: Complete navigation, interactions between modules, messages and notifications, rollback, compatibility, accessibility, general performance.

More details and examples 05. Orientation guide for regression test priorities (*)

05. Orientation guide for regression test priorities (*)

Aspect / tested situation	Description	LX Priority	Regression / smoke	Functional / Non-functional	Comments / Necessary actions
Main flow	Tests the basic complete "happy path" flow	Urgent	Smoke / Regression	Functional	Critical for essential functionality
Critical flows regression testing	Re-testing of flows that have a major impact in the system	Urgent	Regression	Functional	Stability check after deploy
UI + backend/API testing	Verifies if the UI and backend communicate correctly	Urgent	Smoke / Regression	Functional	Validates communication between components
Complete navigation through important screens	Goes through all important screens of the application; Goes through the application without complex actions – just click/tap on each screen	Urgent	Smoke / Regression	Functional	Verifies UI integrity and that no errors appear
Simple security testing	Login, logout, session expiration, invalid passwords	Urgent	Smoke / Regression	Non-Functional	Prevents unauthorized access
Alternative flows and exceptions	Tests error scenarios, invalid	High	Regression	Functional	Critical for stability and robustness

Aspect / tested situation	Description	LX Priority	Regression / smoke	Functional / Non-functional	Comments / Necessary actions
	data, unexpected conditions				
Persistent data validation	Confirms that modifications are saved correctly in the system	High	Regression	Functional	Essential for data correctness
Role-based access testing	Confirms restrictions and permissions on roles	High	Regression	Functional	Critical for rights management
System errors behavior testing	Verifies timeouts, unexpected errors, disconnections	High	Regression	Functional	Prevents blockages or data loss
Test data completeness testing	Realistic, varied, mocked data	High	Regression	Functional	Ensures test reproducibility
Secondary flows regression testing	Re-testing flows with medium impact	High	Regression	Functional	For ensuring general quality
External systems integration testing	Verifies communication with third-party systems (e.g.: payments API)	High	Regression	Functional	Critical if external dependencies exist
Multi-user flows testing	Simulating simultaneous concurrent access	High	Regression	Non-Functional	Prevents conflicts and blockages

Aspect / tested situation	Description	LX Priority	Regression / smoke	Functional / Non-functional	Comments / Necessary actions
Large volume data flows testing	Tests behavior with large data volumes	High	Regression	Non-Functional	Ensures stability at high load
Parallel transactions flows testing	Correct management of simultaneous transactions	High	Regression	Non-Functional	Prevents data corruption or loss
Complex validations flows testing	Verifies complex business rules	High	Regression	Functional	Confirms respect for rules and constraints
Offline / reconnect flows testing	Behavior on connection loss and recovery	High	Regression	Non-Functional	Prevents issues in unstable network conditions
Advanced security flows testing	Penetration tests, SQL injection, XSS etc.	High	Regression	Non-Functional	Protection against cyber attacks
Audit and logging flows testing	Verifies the correctness of generated logs	High	Regression	Functional	Ensures traceability and compliance with norms
Backup and restore flows testing	Ensures data integrity in recovery situations	High	Regression	Functional	Preparation for critical situations
Data migration flows testing	Validation of migration correctness	High	Regression	Functional	Maintains continuity and data integrity
Complete navigation	Goes through all screens involved	Normal	Regression	Functional	Verifies UI integrity for all

Aspect / tested situation	Description	LX Priority	Regression / smoke	Functional / Non-functional	Comments / Necessary actions
through all screens	in flows, without errors				application screens
Interactions between modules	Verifies communication between modules (e.g.: notifications, reports)	Normal	Regression	Functional	Important for system cohesion
Performance and response testing	Verifies response times and general performance	Normal	Regression	Non-Functional	Important for user experience
Messages and feedback validation	Clear and useful messages in the flow	Normal	Regression	Functional	For a good user experience
Duration and complexity testing	Optimized flows for fast running	Normal	Regression	Functional	For maintaining pipeline efficiency
Rollback/undo flows testing	Verifies action cancellation functionality	Normal	Regression	Functional	Ensures data recovery and error prevention
Notification and alert flows testing	Validating correct sending of notifications (email, SMS, push)	Normal	Regression	Functional	Maintains efficient communication with the user
Compatibility flows testing	Verifies functioning on diverse OSs, browsers and devices	Normal	Regression	Non-Functional	Extends user coverage

Aspect / tested situation	Description	LX Priority	Regression / smoke	Functional / Non-functional	Comments / Necessary actions
Accessibility flows testing	Complies with norms for users with disabilities	Normal	Regression	Non-Functional	Increases inclusion and complies with legislation

03. Regression Tests for new implementations (*)

Change and impact analysis (Impact Analysis)

- Clearly identify the new requirements or changes in the existing requirements (for example, functional documentation, diagrams, information in the User Story and/or Confluence).
- Check with the team or Product Owner if the business flows are not already documented and prioritized - for more details 01. Initial Elicitation of Business Flows from the Product Owner (-)
- Perform an impact analysis to identify the functionalities that could be affected by the implemented changes.
 - Verify the system components that are linked to the recent modifications.
 - Identify all dependent areas that could be affected (e.g.: modules, databases, user interfaces, APIs).
- Prioritize the functionalities that must be tested depending on the degree of risk and criticality - if priorities are not already made.

Test Case - LX

Before starting to create new test cases, first verify that there are no already created LXs: 02. Evaluating existing tests for Regression (*)

If LXs that can be reused for the purpose of the tested User Story are not found, then new LXs will be created.

- Regardless of whether it is an existing test case that will be reused or a new one, the LX will be added in the Xray structure within the project in the "Regression Tests" subfolder of each Scrum team.
- The Regression LXs will receive the "regression" label.
- Influenced Application field - the application must be added here, like for any other LX.
- The Priority field will be used for mapping to the 3 priorities of business flows (by default when a new LX is created it will have Priority set to "Normal")
 - Test Case LX priority = Urgent - Priority 1
 - Test Case LX priority = High - Priority 2
 - Test Case LX priority = Normal - Priority 3
- The LX's title will contain the name of the Business Flow - this can be found at the User Story level,
 - in the Summary field (the name of the User Story) or in the User Story field from the requirement
 - if the business flow cannot be found it is best to ask the Product Owner or the team

Regression test cases will enter a continuous maintenance circuit.

As the application\project advances new implementations will always come and the test cases will be updated by the tester.

Maintenance of Test cases and the Regression suite

- as new implementations appear, the testers assigned to the project (helped by PO and BA) will have to evaluate the impact on the existing regression suite

- modifications can occur following functional changes
- modifications can occur following technical limitations or changes
- some tests can become irrelevant or redundant and can be closed\removed from the suite
- some User Stories require new regression tests and for this they must be validated by the PO

04. Regression Tests Execution (*)

When is it performed?

1. Mandatory - After each UAT or PRL restore based on the restore calendar it is done on the Epic in progress
2. Mandatory - In the PRL phase of the Epic (project) (a broader regression of the application on areas that are usually prone to problems, important flows, after the successful running of the Smoke tests)
3. Following various technical upgrades that require a regression testing - where DEV requires this validation
4. At the request of the PO or Scrum team for any other changes that require regression testing

more details here 06. Regression Testing Workflow (*)

Regression testing consists of re-executing functional and non-functional tests after code modifications, to verify if things that worked before still work correctly.

Regression tests will be run as follows after the restore calendar

- at pre-established time intervals - every 3 months, after restore

?

How long is it tested? * The testing period varies depending on the type of testing performed: total or partial (not to exceed 12h)

- The specific testing period will depend on
 - how large the regression test suite is,
 - which tests are priority,
 - which tests are chosen for regression etc.

In what order is it tested? Happy path LX → Smoke Testing → Regression testing → Retesting critical Bugs

The tests will be executed depending on priority - priority is best to come from the Product Owner 01. Initial Elicitation of Business Flows from the Product Owner (-)

If there is no priority from the Product Owner, the Testing team in the project will try as much as possible to prioritize the test cases - the team's opinion, especially the Business Analysts', will be asked as much as possible.

If regression testing is not run on a User Story, the PRL Regression testing Sub Test Execution will be closed by setting the Aborted status.

How is it tested? * A sub-test execution is created - the sub task execution will be named PRL Regression Testing

- The LXs are added in the sub-task execution - (Add existing Tests - JQL) the filter below can be used - example for the IMBK project,
 - type = Test and labels="regresie" and issue in testRepositoryFolderTests(LX, "Scrum MBK", "true") and status !=ScenariiFinalizate
- (Optional) A test plan is created for the regression period for the project
- (Optional) From the Test plan it is added from the "Test Execution" section

It is no longer necessary to attach evidence for any type of testing: LX execution, UAT or Prelive testing, SCRUM or Waterfall testing, including in bug fixes retesting; evidence in bug reporting remains.

Where is it tested? Regression tests will be run on both the UAT environment and the PRL environment.

Who tests? Initially the Testing team\Testers who are assigned to the project.

- Manual testers: will run the tests without the automated label; type = Test and labels="regresie" and priority = High and "Aplicatii influentate" = MBK and status !=ScenariiFinalizate and label != automated
- Auto testers:
 - will run the tests with the automated label; type = Test and labels="regresie" and priority = Normal and "Aplicatii influentate" = MBK and status !=ScenariiFinalizate and label = automated or
 - will send the automatic test run report to the manual testing team - it must exist clearly
- Long-term plan: a dedicated team that will run the tests - this team will have a rotation-based component

Bugs

Bugs will be linked to the LX at the time of execution at the steps in the LX - otherwise they do not appear in reporting

How is progress tracked? Within the project the Xray Reports section - Test Executions Reports or Test Plans Report

Guide for the Tester - Best practices for Regression Test Cases (-)

Regression testing ensures that the modifications brought to the application did not introduce defects in the existing functionalities. To create a robust regression test suite based on business flows, a tester must follow a well-defined series of steps. This guide offers a structured approach to write efficient regression tests.

Analyze the Documentation: Start by reviewing the relevant documentation, including business requirements, functional specifications, and business flow diagrams. Understand how each flow should work and what objectives it pursues. The flows will be listed, detailed, and documented by the Product Owner and BAs.

Collaborate with BA and PO: Discuss with the Business Analyst and the Product Owner to clarify any ambiguities and to gain the best possible understanding of the critical business flows.

Establish the Key Points: In each flow, identify the critical interaction points, such as data validation, transaction processes, or other essential actions.

Business flows can be received from the Product Owner in the format below.

Based on the received structure, the test structure in Xray can be created.

Based on the priorities received from the Product Owner, the priority of the test cases can be established. The initial effort is estimated - ideally, the Scrum Master\PM for the SCRUM project will be notified and time will be allocated for this activity.

1	Name Flow1	Here it will be filled with details about Flow1	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 1
2	Name Flow2	Here it will be filled with details about Flow2	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 2
3	Name Flow3	Here it will be filled with details about Flow3	Diagrams, written documentation (Confluence or any other type), Jira user stories or epics, printscreens etc for Flow 3

Reusing test cases - Initial analysis of already existing scenarios * Verify if there are already tests that cover the described business flow

- Reusing already existing tests will be attempted as much as possible. For this purpose, any folder\test repository that might contain test cases (LXs) about the business flows will be analyzed. In principle within the same project but reusing some scenarios from other projects is not excluded if they are considered suitable.
- the tests that can be reused can be
 - already written tests with the purpose of regression testing
 - already written tests for testing User Stories\Epics based on new functionalities\implementations.
- Some business flows might already be covered by one or more already existing LXs. Those tests must be reused as much as possible. The tests can be copied\cloned\edited and\or moved to a new folder and can be included in the Regression Suite.
 - If a business flow is already covered by 1 or more LXs these will be reused for the purpose of Regression Testing.
 - If LXs already exist for a business flow they must be analyzed\evaluated and in case they are not up-to-date they will have to be edited with the new implementations.
- If one or more LXs are already written and they represent prerequisites for the business flow for which the initial analysis is done, the LXs will be included using the "Call Test" functionality

Prerequisites

- Prerequisites must be noted and documented - they can be added in the "Description" section for the LX in Xray.
- The prerequisites will include all the preceding steps to be able to start the LX itself and can be written in the form of free text - or a Confluence URL

- If there are prerequisites already written in the form of LXs, these will be included in the LX steps using the "Call Test" function

Defining New Testing Scenarios

- **Create Test Scenarios for Main Flows:** For each business flow, define testing scenarios that cover the main path (happy path) and all possible branches (edge cases). Scenarios should reflect the different ways users can interact with the application.
- **Including Negative Scenarios:** Do not limit yourself only to success flows; also create negative testing scenarios to verify how the system handles errors or unexpected behaviors.
- The Test Case (LX) name will contain the name of the tested business flow (or at least a part of it)

General Best Practices

- **Group Test Cases into Suites:** Organize tests into logical suites that reflect the structure of the business flows.
- Each suite should correspond to a specific business flow. The structure will be like the one here 01. Initial Elicitation of Business Flows from the Product Owner (-)
- The LXs must cover the entire business flow, not just partially. Multiple LXs executed in a specific order can cover a business flow.
- a business flow must be covered by at least one test case - LX.
- The LXs that repeat (e.g. login into the application, navigation to a certain section of the application) will be called in the test using the Xray "Call Test" functionality
- Negative scenarios must also be evaluated and added
- Test cases should not contain steps that repeat in other test cases. If steps repeat, they must be called using the "Call Test" function from Xray
- The LXs will have to be written as descriptively as possible - the final goal is direct automation based on the LXs.
- The test steps must be clear and specific and ideally if the action has a result, the expected result must be written.
- The test cases should ideally be written following the business flow, and the execution of test cases must be thought of as having a logical sequence.
- Test Cases should not verify the same thing multiple times; limiting the "exhaustive" type testing will be attempted as much as possible
- If a business flow requires multiple combinations of scenarios to be covered (e.g. users with different roles for different screens\application functionalities) using the most representative test data combinations will be attempted as much as possible thus covering as many important cases as possible - but not all possible combinations to limit the testing effort
- Regression LXs must receive the "regression" tag and Priority for the test case will be set depending on the priority of the Business flow
- The manual test execution effort must be estimated (to be seen where it can be introduced)
- The LXs will be added in the Regression Tests subfolder of each Scrum team

- If it is not found, the folder of the application\SCRUM team will be created
- Sub-folders will be created under the SCRUM project that will respect 1:1 the business flows structure received from the PO
- Regardless of whether it is an existing test case that will be reused or a new one, the LX will be added in the Xray structure within the project.
- The Regression LXs will receive the "regression" label
- Influenced Application field - the application must be added here, like for any other LX
- The Priority field will be used for mapping to the 3 priorities of business flows (by default when a new LX is created it will have Priority set to "Normal")
 - Test Case LX priority = Urgent - Priority 1
 - Test Case LX priority = High - Priority 2
 - Test Case LX priority = Normal - Priority 3

Scenarios validation (optional for regression tests) (-)

A Product Owner (PO) can return the test cases to the tester after validation for several reasons, to ensure that the tests correctly cover the established requirements and standards. Here are some common reasons

For any of the reasons below, the PO can send the test cases received from Testers back for rewriting.

The reason why the test cases are sent back will have to be mentioned, either as a comment in the LX, or by email following the evaluation (procedure already existing for User Stories)

Prerequisites

- Prerequisites must be noted and documented - they can be added in the "Description" section for the LX in Xray.
- The prerequisites will include all the preceding steps to be able to start the LX itself and can be written in the form of free text - or a Confluence URL
- If there are prerequisites already written in the form of LXs these will be included in the LX steps using the "Call Test" function
- If the PO notices that prerequisites are not noted where it is known they are needed, for this reason the PO can send test cases back for revision and rewriting.

Incomplete Coverage of business flows

- **Lack of certain Testing Scenarios:** If the PO notices that certain business flows or functionalities are not covered by the test cases, they can request their completion or modification.
- **Omitting Critical Flows:** If the tests do not include critical scenarios or important use cases, the PO can request their inclusion to ensure complete coverage.

Discrepancies between Requirements\Flows and Tests

- **Wrong Interpretation of Requirements:** If the tests seem to reflect a wrong interpretation of the business flows, the PO can request their correction to align better with the expectations.

- **Differences between Expected Results and Specifications:** If the expected results from the test cases do not correspond to the established expected results, the PO can request the revision of the tests.

Lack of Clarity

- **Unclear or Ambiguous Instructions:** If the steps described in the test cases are not clear enough or are ambiguous, the PO can ask for clarifications or their rewording to avoid confusions in execution.
- **Insufficiently Detailed Expected Results:** The PO can request more details in the expected results section if these are not precise enough to determine the success or failure of a test.
- **Test Case with too many steps:** If the test case has way too many steps and becomes hard to follow, the PO can ask for their rewriting and/or splitting a test case into several.
- **Test cases do not respect the logical order in the business flow:** The test cases ideally should be written following the business flow, and the execution of the test cases must be thought of as having a logical sequence. If the PO notices that the normal business flow is not respected then they can ask for a revision

Repetitive Tests or tests that verify the same thing multiple times

- **Test Cases have steps that repeat in other Test Cases:** Test cases should not contain steps that repeat in other test cases. If steps repeat, they must be called using the "Call Test" function from Xray to be able to maintain a logical structure of the tests. This workflow will be useful for test automation as well. For this reason the PO can send test cases back for revision and rewriting.
- **Test Cases should not verify the same thing multiple times;** limiting the "exhaustive" type testing detailed below will be attempted as much as possible. For this reason the PO can send test cases back for revision and rewriting.

Lack of Negative Scenarios and Edge Cases

- **Omitting Negative Tests:** If the tests do not include negative scenarios (for example, invalid inputs or unforeseen behaviors), the PO can request adding them to verify the results of the tested applications and error displaying.
- **Lack of Edge Cases:** In case limit situations or rarer cases are not covered, the PO can ask for a revision to include these scenarios, essential for quality assurance.

Discrepancies with Business Priorities

- **Inadequate prioritization:** If the tests do not correctly reflect the business priorities, meaning they do not focus on the most critical functionalities (according to the business flow priorities), the PO can ask for the realignment of the tests with the established priorities.
- **Ignoring the Impact on the Final User:** If the tests do not consider the impact on the final user experience, the PO can request adjustments to ensure that the tests reflect the way real users will interact with the application.

The Need for Alignment with Recent Modifications

- **Modifications in Requirements or Business Flows:** If recent modifications were made in the requirements or business flows, and the tests were not updated to reflect these changes, the PO can ask for the revalidation or update of the tests.

Ensuring Compliance with Quality Standards

- **Respecting Quality Standards:** If the tests do not respect the quality standards or the internal processes established by the team, the PO can ask for modifications to align with these standards.
- The PO can request the revision of the tests to ensure that all completeness criteria are met, including all relevant use cases and business flows.

Feedback from other team members

- **Suggestions from other team members:** If the PO receives additional feedback from other team members suggesting changes or improvements to the tests, they can return the test cases to include these suggestions.
 - **Alignment with Client Expectations:** If the tests are not aligned with the expectations of the clients or final users, the PO can request adjustments to better reflect their needs and requirements.
-

07. Agile\SCRUM Workflow (*)

Agile is a flexible approach for software development, which encourages frequent deliveries, collaboration between teams, and rapid adaptation to changes. Scrum is an Agile framework that organizes work into short sprints, usually of 2 weeks, with clear objectives and validated deliverables.

Agile/Scrum working mode (in the team's practice)

2-week Sprints

- Each sprint begins with a planning (Sprint Planning) where the tasks to be carried out are established.
- Daily, a Daily Stand-up is organized, where each member communicates their progress, blockages, and the plan for the day.
- At the end of the sprint:
 - Sprint Retrospective – what went well and what can be improved is discussed.

Tools used:

- Jira for tasks and backlog.
- Confluence for documentation.
- Xray for test management.

Tester's specific tasks:

1. Requirements and specifications analysis

- Understanding business requirements and project documentation (e.g.: user stories, functional specifications).
- Identifying critical or unclear areas that require additional clarifications.

2. Testing planning

- Defining testing strategies for new or existing functionalities.
- Estimating the testing effort and defining the necessary types of tests (functional, regression, usability etc.).

3. Writing test scenarios and cases

- Creating test scenarios based on requirements (Test Cases).
- Using a clear structure like Action / Data / Expected Result.
- Organizing tests in tools like Jira + Xray.

More details here: 01. Defining Test Scenarios Workflow - QA - Confluence

4. Manual test execution

- Running tests on various environments, browsers, or devices.
- Verifying the functionalities' behavior according to requirements.
- Retesting defects after fixing (retesting).

5. Reporting and tracking defects

- Reporting bugs in Jira.
- Ensuring the defect description is clear and contains reproduction steps, obtained and expected results, screenshots/logs.
- Collaborating with developers for clarifications and prioritization.

More details here: 03. Bugs Workflow - QA - Confluence

6. Validating fixes and regression

- Verifying the correct resolution of defects (bug fix validation).
- Executing regression tests to confirm that the modifications did not affect other functionalities.

More details here: 06. Regression Testing Workflow - QA - Confluence

7. Participating in Agile ceremonies (where applicable)

- Participating in daily, sprint planning, analysis (grooming), and retrospectives.
- Providing feedback related to testing coverage and identified functional risks.

8. Documenting and updating tests

- Updating test cases when modifications appear in the application.
- Organizing and maintaining an updated and relevant test library.

9. UI and UX testing

- Verifying the interface's intuitiveness and the application's behavior in unforeseen scenarios.
- Identifying potential blockages in the user experience.

More details here: 04. UI\UX LXs - QA - Confluence

10. Collaboration and communication

- Active communication with the development team, BA, PO, SM etc.
- Clarifying requirements or expected behavior for complex functionalities.

For User Stories

When the US is taken into testing, the "Testing Responsible" field must be filled in.

- **Defining scenarios (sub-task)** - assignee to the Tester who will define the scenarios
- **UAT Testing (sub-test execution)** -
 - assignee to the Tester who will test in UAT
 - if multiple testers will test on the same User Story - each tester will create a separate sub-test execution
- **PRL Testing (sub-test execution)** * assignee to the tester who will test in PRL
 - if multiple testers will test on the same User Story - each tester will create a separate sub-test execution

00. What is Agile\SCRUM?

Agile Methodology (SCRUM framework) – A project management methodology characterized by sequentiality and taking over the project deliverables in successive iterations. Project specifications are defined in the form of user stories. The project is organized in sprints that have a defined scope and deliverable.

Within a Scrum team, each member has a well-defined role, but collaborates closely to deliver functional and valuable products.

Roles: * Product Owner – represents the person responsible for defining releases and the detailed implementation roadmap, for defining implementation deadlines and achieving them, as well as prioritizing the Scrum team's backlog (user stories and incidents) to facilitate the implementation of requirements, while maintaining the conceptual and technical integrity of the deliverables. Most often, the activities are carried out in collaboration with the Business Analyst.

- **Scrum Master** – represents the person holding the Project Manager function within the Process Quality Management Division who ensures that the Scrum methodology is correctly applied, that the Definition of Ready and Definition of Done criteria are respected, that the Business Analyst and Product Owner define user stories and set acceptance criteria, creates the Sprint Backlog, Release Plan, creates and manages the project plan. Monitors progress and escalates impediments when necessary and ensures that Scrum ceremonies take place, achieve their purpose, and respect the Scrum methodology.
- **Business Analyst** – represents the person holding this function within the Process Quality Management division or in other divisions, who deals with the analysis of requirements registered in Jira and the drafting of functional specifications, which must reflect the business need and provide all the details necessary for the technical analysis, development, and testing of the requirement.
- **Tester** – represents the person holding this function within the Process Quality Management division, but also the person from the Business divisions, who defines the test scenarios based on the functional specifications and who tests the developments to ensure implementation according to business requirements.
- **Developer** – represents the person from the IT Division allocated to implement the requirement in the IT applications.

Also within the Scrum methodology, ceremonies are essential meetings that structure the team's work and support transparency and adaptation.

Ceremonies:

- **Daily Sprint** – the daily 15-minute ceremony during which each SCRUM team member answers the 3 questions: What did I do yesterday? What will I do today? What impediments do I have?
- **Sprint Retrospective** – the ceremony in which the team analyzes the progress during the sprint, brings proposals to improve the working mode.
- **Planning** – the ceremony in which the Scrum team assumes user stories for the next period depending on the team's availability or velocity and details the User Stories into subtasks which they estimate in minutes/hours.

- **Analysis/Grooming** - the ceremony in which the detailed specifications of the user stories are presented, so that all aspects are clarified. During this meeting, the technical solution for each individual user story is also identified.
- **Planning Poker** – the technique of estimating user stories in terms of complexity, in Story points, by comparison with user stories already implemented by the team. To see how estimation is done in Story Points, you can go [here](#).

01. What is a User Story?

Within a Scrum team, a User Story is a short, clear, and informal description of a functionality desired by the user expressed from their perspective, to highlight the value it brings.

Definition:

User Story = a functional requirement written from the end user's perspective, which expresses what they want to achieve and why.

It is a core element of the Product Backlog and is used by the Scrum team to plan, develop, and deliver relevant functionalities in a Sprint.

02. Ceremonies in Agile\SCRUM

Within the Agile Scrum methodology, there are 4 main ceremonies (also called "Scrum events") that structure the team's activity:

Below you can find a clear list of each ceremony and its explanation:

1. Sprint Planning

What it is:

The session where the team decides which User Stories they will work on in the next sprint and how they will be implemented.

Purpose:

- Defining the Sprint Goal
- Selecting the backlog items that can be completed
- Detailing the necessary technical activities

Recommended duration:

Up to 60 minutes for a 2-week Sprint

2. Sprint Retrospective

What it is:

The team's meeting to reflect on the working mode and identify improvements for the next sprint.

Purpose:

- What went well?
- What didn't go well?
- What can we improve?

Recommended duration:

Up to 60 minutes for a 2-week Sprint

Note: Most of the time, the Sprint Planning and Sprint Retrospective meetings are held together at the end of the sprint.

3. Daily Scrum (Daily Stand-up)**What it is:**

The daily 15-minute meeting where team members synchronize their activity.

Purpose:

Answering the 3 questions:

- What did I do yesterday?
- What am I doing today?
- What impediments do I have?

Recommended duration:

Exactly 15 minutes

4. Analysis meeting (Backlog Grooming / Refinement)**What it is:**

Periodic activity (not an official ceremony in the Scrum Guide) where the team clarifies, details, and prioritizes the backlog items.

Purpose:

- Clarifying requirements
- Estimating User Stories
- Preparing the backlog for the future sprint

Recommended frequency:

Up to 3 hours for a 2-week Sprint

03. Components of a User Story**1. Title (Summary)**

A concise and clear description of the functionality or requirement that the user story addresses.

2. Description

Detailed explanation of the user's need, often expressed through a clear formula:

"As a [user type], I want to [functionality] so that I can [benefit]"

3. Acceptance Criteria

The set of precise and verifiable conditions that must be met for the user story to be considered complete and accepted.

4. Priority

The level of importance of the user story for business or users, which influences the order in which it will be developed.

5. Estimate

The effort needed to implement the user story, jointly estimated by developers and testers, usually expressed in story points.

6. Additional information (Attachments/Notes)

Support materials such as documentation, sketches, mockups, or useful links that facilitate understanding and implementing the requirement.

04. The role of a Tester in Agile\SCRUM

The tester has the role of validating that the developments were made according to the specifications, by going through the test scenarios, respecting the Agile/Scrum working mode. In Scrum teams where this role is not covered, the responsibilities will be ensured by the business resource allocated to that team.

Responsibilities: * provides input for team consolidation;

- clarifies the user stories that will be presented in the analysis/grooming meeting, together with the Business Analyst, to understand the requirement to be tested and to contribute to completing the specifications, if necessary, before development begins;
- identifies the prerequisites necessary for testing and requests team support to find them easily;
- provides estimations in Story Points;
- reports daily in Jira the time allocated for testing/defining scenarios/bugs;
- assumes user stories they will test in the sprint, depending on the testing capacity, estimating the necessary hours for testing in the DEV2/UAT test environment and in PreLive as well as the number of hours for defining test scenarios;
- defines the scenarios and tests the assumed user stories;
- validates the test scenarios with the Product Owner and the Application Owner;
- reports bugs, if any, and ensures their remediation;
- identifies impediments and implements actions to reduce associated risks;
- updates the list of impediments;
- updates the status of tasks and User Stories;
- presents the Demo for User Stories to the Product Owner or Application Owner then modifies them into the Approval Required status;
- discusses impediments encountered by individual members and identifies solutions to motivate the team;
- agrees on concrete actions to improve the activity in the next sprint.

Sprint execution:

- Start development -> while IT develops, the tester works on defining the test scenarios in Jira (Xray) (with IT and BA support) and organizing the prerequisites. The test scenarios are sent to the PO for validation (+ team in CC + other application owners, if applicable).
 - IT testing -> will be done with a minimum set of scenarios (what IT considers should be validated).
 - Code review -> after IT testing is finished (and before business testing starts), the patch is sent for code review. After completing the code review, the patch is moved to UAT and business testing begins.
 - Bugs identified in testing are opened in Jira and bug fixing is done in the development environment.
 - After bug fixing, corrections are moved to UAT and retested, and if all test scenarios are successfully run at the end in UAT, a demo is made to the PO or Application Owner to obtain acceptance. The PO will track the validation of the Definition of Done.
- *** In the Review (Demo) meeting, the PO can invite other stakeholders who are directly impacted.
- The PO will transition the status in Jira on the accepted USs to Final Test status.
 - The initial code review is reviewed and the final patch is created (the same patch that will reach Live) and moved to Prelive.
 - Testing in Prelive (only smoke test -> testing the most important functionalities with the purpose of validating the good functioning of the application).
 - The tester will transition the User Story in Jira into Ready for Release status.
 - The Scrum Master runs through the mandatory checklist before implementation in Live.
 - Go Live planning by IT.
 - After implementation in production, IT will change the status on the US in JIRA from Ready for release to Live, no later than the day after go live.

01. Defining Test Scenarios workflow in Agile\SCRUM

Stage	Description
Understanding the requirements	You carefully read the user story and the related acceptance criteria
Clarifications with the team	You discuss with the BA, PO or developers to understand the functionality
Identifying testing areas	You establish what functionalities and behaviors must be verified
Creating test scenarios	You write detailed test scenarios, according to the structure: Action/Data/Expected result.

Stage	Description
Organization in Xray	You upload the scenarios into the corresponding folder from the Xray Test Repository
Internal review	You ask for scenario validation from the PO/Application owner.
Preparation for execution	You ensure the scenarios are clear, correct and ready for testing

02. Executing Test Scenarios workflow in Agile\SCRUM

Stage	Description
Taking over test scenarios	You identify the prepared scenarios for the user stories included in the Sprint
Creating test execution	You create a Sub-test Execution in Jira for the tested story
Running scenarios	You run step by step each defined test scenario
Updating test status	You mark each test as Pass, Fail, Executing, Aborted or Blocked, as appropriate
Defect reporting	For any step that fails, you create a bug and link it to the respective step.
Progress tracking	You update the testing progress in Jira.
Retesting & closure	After fixing a defect, you retest it and update the test status.

03. Bug Reporting in Agile\SCRUM

Stage	Description
Identifying the bug	Observing an unexpected or erroneous behavior
Collecting information	Gathering steps, environment, data, evidence for the clear reproduction of the problem
Creating bug in Jira	Filling in essential fields + relevant attachments

Stage	Description
Associating with user story	Linking to the story/test for traceability
Communication	Informing the developer and tracking the status
Retesting & closure	Verifying after the fix and closing if the bug is resolved

04. DEMO Presentation in Agile\SCRUM

The DEMO meeting is a presentation of the results of running tests in the UAT test environment, with the purpose of receiving acceptance from the Product Owner and Beneficiary.

The DEMO meeting is held:

- in the case of Scrum projects → within the sprints, after the completion of testing in the UAT business test environment - with a minimum of 2 days before the sprint ends, so that the Tester has enough time for testing in Prelive (smoke test)

The Scrum Master is the one who sends the invitation to the demo meeting, after the testing in the UAT test environment has been finalized.

To hold demo meetings, users and clients are prepared in advance, presenting the most relevant tested cases and any other case requested by the Product Owner or the Project Owner, alongside the evidence of the already tested scenarios.

At the end of the demo meeting, the Product Owner and the Beneficiary will determine whether they grant acceptance (transitioning the User Story to the Final test status) or refuse and the User Story returns to the In testing status, where testing is resumed.

DEMO Checklist

The tester is responsible for going through the list below and noting the problems\risks encountered.

If they notice risks for any of the points on the list, the tester must notify their Team Lead to officially flag the risks to the Product Owner via email.

1. Verify the validation of the test scenarios

- Make sure there is validation on the test scenarios from the PO.

2. Verify that there are no open bugs

- Avoid situations where unvalidated functionalities or bugs block acceptance.
- If there are bugs, the tester makes a list of bugs ordered by severity, with blockers being the priority.
- The list will be sent to the Team Leader and the identified risks will be mentioned.

3. Confirm Acceptance Criteria

- Acceptance criteria are part of a user story and all must be met, tested, and validated to be presented in the demo.

- Acceptance criteria are part of DoD (Definition of Done) and if at least one of the criteria cannot be presented in the demo, then the risk must be flagged to the TL.
- If there are no acceptance criteria, contact the Product Owner to establish together which flows will be presented.
- Confirm with the Product Owner that the flow you will present is relevant and is the one expected by the Beneficiary.
- In case the Product Owner is not available, it is recommended to present a happy path in the Demo.
- Make sure there are no ambiguities in the functional requirements and if there are, clarifications will be requested from the PO/BA.
- Confirm the existence of updated documentation (specifications, user stories, mockups).

To provide a clear picture of the DEMO content and the aspects that require attention, either a request for flow validation will be sent by email (for projects of low complexity), or a DEMO Preview meeting will be requested (for projects of high complexity), where what will be covered and how it meets business needs will be presented.

The request will be sent in advance to set expectations and obtain confirmation of the scenarios and flows that will be presented in the demo.

4. Test data

- Do not use manipulated test data that requires manual intervention - the normal business flow must be followed.
- Confirm that the test data is clean, relevant. The test data presented in the DEMO must be data as close to production as possible, without improvisations (e.g.: manipulation in the database).
- Make sure you have test data from the business that is relevant for the tested developments. To replicate the test data received from the business, IT colleagues will be called upon for the necessary imports from the Live environment to the test environment.
- Business test data:
 - CUI/Client Id/CNP/IBAN/credit reports/users (rights)/financial documents (ANAF, CRC, Just etc).

The tester has the obligation to verify through a quick test that the aspects below function properly:

- **Environment and Configuration**
 - Ensure all external services and APIs are functional.
 - All integrated systems that are part of the environment you will present on are functional. (ex: IBS, T24, OpenKM, Sysmart).
- **Critical Functionalities**
 - Test all main flows (happy path) that will be presented.
 - Verify error scenarios and displayed messages (no unexpected errors should appear).
 - Ensure all buttons, links, and UI actions work.
- **Performance and Stability**

- Run the app on relevant devices and browsers.
- Check response times for key functions.
- Monitor if crashes or freezes occur.
- **Visual Aspect and UX**
 - Confirm that the design meets the requirements (colors, fonts, layout).
 - Check responsive design (desktop, mobile).
- **Data and Security**
 - Ensure displayed data is correct and updated.
 - Verify that no sensitive data is visible.
 - Test authentication and authorization (correct roles).

DEMO Preview

Depending on the size of the project for which the DEMO meeting must be held, the tester / testing team has the responsibility to validate the deliverables together with the PO.

The DEMO Preview will be held / transmitted (depending on the points described below) one day before the official DEMO meeting.

The tester/testing team together with the Team Leader will establish the direction to go:

1. In the case of projects with low complexity (e.g.: 1-2 USs to be presented in the Demo), the tester will only send an information email requesting validation of the mentioned flows, according to the working procedure described here.
2. In the case of projects with high complexity (especially when there is a large number of User Stories to be presented), a DEMO Preview meeting will be requested to validate the deliverables before the official demo, according to the working procedure described here.
3. If major problems are identified for holding the demo meeting, the tester will notify the Team Lead, and the latter will send a notification to the PO with the identified risks, signaling that these could affect the demo's quality.

1. Flow validation - DEMO Preview e-mail

In the case of projects with low complexity (e.g.: 1-2 USs to be presented in the Demo), the tester will only send an information email requesting the validation of the mentioned flows - see the proposed structure of the information email below.

What the request email contains:

- **To:** Product Owner, Business Analyst, Beneficiary, Tester, Testing TL, Testing Department Head, QM Division Director
- **Content:** * ♦ User Stories included in the demo
 - ♦ Relevant test scenarios
 - ♦ Known limitations and uncovered behaviors

- ◆ Unresolved bugs, if any, with the related impact
- **Request feedback:** participants are invited to send observations before the DEMO meeting.

Example request for validation of flows presented in the official DEMO:

Subject: Validation of flows presented in the DEMO meeting

To: Product Owner, Beneficiary

CC: Business Analyst, Tester, Testing TL, Testing Department Head, QM Division Director

From: Scrum Master/Testing Team Lead/Tester

Hello,

We wish to inform you regarding the flows that will be included in the DEMO meeting *tomorrow (or DEMO date), to ensure transparency and alignment before the presentation. During the demo the following will be addressed:

- ◆ User Stories included in the demo (mention the USs)
- ◆ Relevant test scenarios (previously validated with PO/BA) (mention the scenarios)
- ◆ Known limitations and uncovered behaviors (mention what these are)
- ◆ Unresolved bugs, if any, with the related impact (mention what these are)

This communication aims to give you a clear picture of the demo's content and the aspects that need attention.

Please send us any observations or points to discuss before the meeting, if applicable. If, following this email, we receive no feedback regarding the above, we will consider that we have met the acceptance criteria.

Thank you!

2. Flow validation - DEMO Preview meeting

In the case of highly complex projects, a DEMO Preview meeting will be requested to validate deliverables before the official demo, especially when there are a large number of User Stories to be presented. The main purpose of this meeting is to establish the main flows and scenarios that will be presented in the official demo.

What the request email contains:

- Organizational details: date, time, estimated duration, location (Teams link)
- Participants: Product Owner, Business Analyst, Beneficiary, Tester, Testing TL, Testing Department Head, QM Division Director
- Meeting objectives:
 - Presenting the user stories included in the Demo.
 - Validating the flows/cases presented in the Demo.
 - Discussing limitations and uncovered behaviors
 - Identifying and discussing unresolved bugs and their impact

- Clear agenda for structuring the discussion
- Request feedback: participants are invited to send observations before the meeting.

Example DEMO Preview request:

Subject: DEMO Preview meeting invitation

To: Product Owner, Beneficiary

CC: Business Analyst, Tester, Testing Department Head, QM Division Director

From: Scrum Master/Testing Team Lead/Tester

Date: [Meeting date]

Time: [Meeting time]

Estimated duration: 30 minutes

Location: [Teams Link]

Hello,

We invite you to the DEMO Preview meeting where we will present and validate the following aspects related to the current deliverables:

- ◆ User Stories included in the demo
- ◆ Relevant test scenarios (previously validated with PO/BA)
- ◆ Known limitations and uncovered behaviors
- ◆ Unresolved bugs, if any, with the related impact

The purpose of this meeting is to establish the main flows and scenarios that will be presented in the official demo.

Agenda:

1. Presenting the user stories
2. Detailing the validated scenarios
3. Discussing limitations and bugs
4. Presenting a main flow/happy path and presented scenarios.

Please send us any observations or points to discuss before the meeting, if applicable.

Thank you!

Best practices for DEMO

- Understand the specific nature of the Scrum, the roles, and the specific challenges to adapt the demonstration efficiently.
- Use relevant examples, real flows and scenarios that demonstrate how the product solves their specific problems.

- If there are epics that contain a large number of user stories, they will be grouped and presented as a package, where possible.
- If test data is accessible to other users as well, notify the people (especially the Scrum team and QM team) who might access and corrupt them, to avoid accessing them.
- Notify the development team not to make modifications/deployments on the demo environment at least 1h before the demo.
- Ensure backup (screenshots or video) in case of live problems.
- Note any known limitations and temporary solutions.
- Confirm with the team what will be shown and what will be avoided.
- Managing questions during the DEMO meeting:
 - In the case of business-related questions -> redirect to BA
 - In the case of questions related to included/excluded developments in the package -> redirect to PO
 - In the case of technical questions -> redirect to DEV
 - In the case of questions related to testing scenarios -> mention which scenarios were validated and checked with PO/BA in the DEMO Preview meeting.
- Prepare the presentation according to the recommended Demo structure.

Frequent mistakes during a testing session/demo

- Trying to cover too much content: Do not try to test or present all functionalities in a single session. Focus on business scenarios and not on technical ones.
- Lack of materials or test data: Prepare all necessary resources (test data, accounts, configured environments) before the demo, to avoid blockages. Do not rely just on tools, ensure you have a Plan B (e.g.: test data/backup prerequisites/print-screens/video-screens) in case of technical errors.

Standard structure for DEMO presentation

For an efficient presentation of newly developed flows, it is recommended that the DEMO meeting have the following structure:

1. Agenda / DEMO Content (2-3 min)

Responsible: Scrum Master if participating, otherwise PO

- Introduction / Purpose of the demo.
- Presentation of the epic or ICD in Jira, according to the list of User Stories the Scrum Master sent in advance to set the Demo meeting.

2. General business overview of the implemented project (2-3 min)

Responsible: PO

- Objective of the functionality.
- What the presented project solves from a business perspective.

3. Presentation of Relevant Scenarios

Responsible: Tester

For each user story:

- Title and short description / US Presentation.
- The main flows that will be covered.
- Scenario name + purpose.
- Main steps (happy path flow).
- Behavior in case of error (if applicable).
- Validations previously made with stakeholders.

Rule for questions:

- Questions are accepted/encouraged after the presentation of each flow, to maintain coherence.
- Questions that might derail the presentation are noted and discussed at the end.

4. Conclusions and final questions (5–10 min)

Responsible: PO

- Recap of presented functionalities.
- Known limitations (if they exist).
- What's next in the roadmap.
- General questions or remaining unclarities.

5. Closing

Responsible: Scrum Master if participating, otherwise PO

- Thanks for participating.
- Confirmation of acceptance (if applicable).
- Next steps (e.g., Go/No-Go, feedback, delivery in prelive).

Definition of Done - informative

To access the latest version of the Definition of Done document, access [Definition of Done](#).

Definition of Done USER STORY:	
IT tests have been run and the expected result was achieved	
Code review	
Technical specifications finalized	

Definition of Done USER STORY:	
Live patch prepared on the prelive environment	
All critical bugs fixed (if applicable)	
Business testing acceptance obtained	
Business acceptance criteria met	
Manuals and procedures approved for the functionality (if applicable)	
Board/Executive Comm/VP/etc approval for functionality approval (new or modified)	
Identification of all divisions impacted by the new project's functionalities	
FIMA ok needed	
Stress test	
Ok for testing (actual testing or "ok" on test scenarios)	
Demo and obtaining acceptance	
Dependencies on other goLives	
Reminder to the impacted division to finalize flows before the GO LIVE date	
Message interval stop work in applications	
Establish deployment steps order in production	
Internal informations (branch/impacted divisions/division heads)	
Client informations (collaboration with Marketing if applicable)	
Manual/Procedures/product sheet	

Definition of Done USER STORY:	
Impact in data structure/reporting	
Provide support for clients (for projects with client impact)	
Post-implementation - Smoke test	

Document validation responsables	
Name	Division

How do we hold a demo?

This course, "How do we hold a DEMO?", is dedicated to understanding and applying the best practices for organizing and presenting a DEMO meeting within Agile projects. The DEMO represents the key moment of the sprint, where the team makes its progress visible, validates deliverables, and obtains the necessary feedback for final acceptance.

During the course you will learn:

- The role and importance of the DEMO in the Agile process.
- The essential checklist for preparing a session without surprises.
- The difference between a DEMO Preview and an official DEMO.
- The recommended structure for a clear and efficient presentation.
- Best practices for planning, execution, and communication.
- Techniques for managing emotions and verbal presentation (e.g. "Traffic Light Technique").

The goal is to help you deliver a professional, well-organized, and business-oriented DEMO that ensures transparency, clarity, and collaboration.

Video Resources

To view the full recorded course, access the link below:

► Video Course – How do we hold a DEMO?

How do we hold a DEMO.mp4

Useful links

- Coursera – Effective Communication : <https://www.coursera.org/learn/wharton-communication-skills>
- Udemy – Public Speaking Essentials : <https://www.udemy.com/course/public-speaking-presentations/>

- LinkedIn Learning – Communication Foundations : <https://www.linkedin.com/learning/communication-foundations>
- TED Talks – Public Speaking Tips : https://www.ted.com/playlists/226/before_public_speaking

Estimates in Agile\SCRUM (Story Points Estimation)

How user stories are estimated in story points:

1. A baseline reference is chosen:

- Each team refers to the reference user stories*, which you can find here

2. Complexity is analyzed:

- The total effort necessary for completion is considered, including:
 - complexity
 - duration
 - uncertainty

3. The modified Fibonacci sequence is used:

- The values are: 1, 2, 3, 5, 8, 13 SP

4. The estimation is done as a team (Planning Poker)

- Each member of the team votes on the complexity of the discussed User Story, without the PO, BA and SM
- In case there are differences in the estimation, those who awarded the highest and the lowest scores will justify and explain the implementation method to the team. A revote is held, until the team agrees on the score, and the information regarding the Story points estimation is filled out in JIRA.
- In case the team's estimation for a US exceeds 20 SP, the possibility of it being split into multiple USs of a smaller size is analyzed, so that the completion of any new resulting US does not exceed the duration of a sprint
- Remember that the estimation must include both the testing effort and the development effort

Example:

	Defining scenarios (in minutes\hours)	UAT Testing (in minutes\hours)	PRL Testing (in minutes\hours)	SP x 1.5h\1SP total
US 1 story point	30m	45m	15m	1.5h
US 2 story point	1h	1.5h	0.5h	3h
US 3 story point	1h	2.5h	1h	4.5h

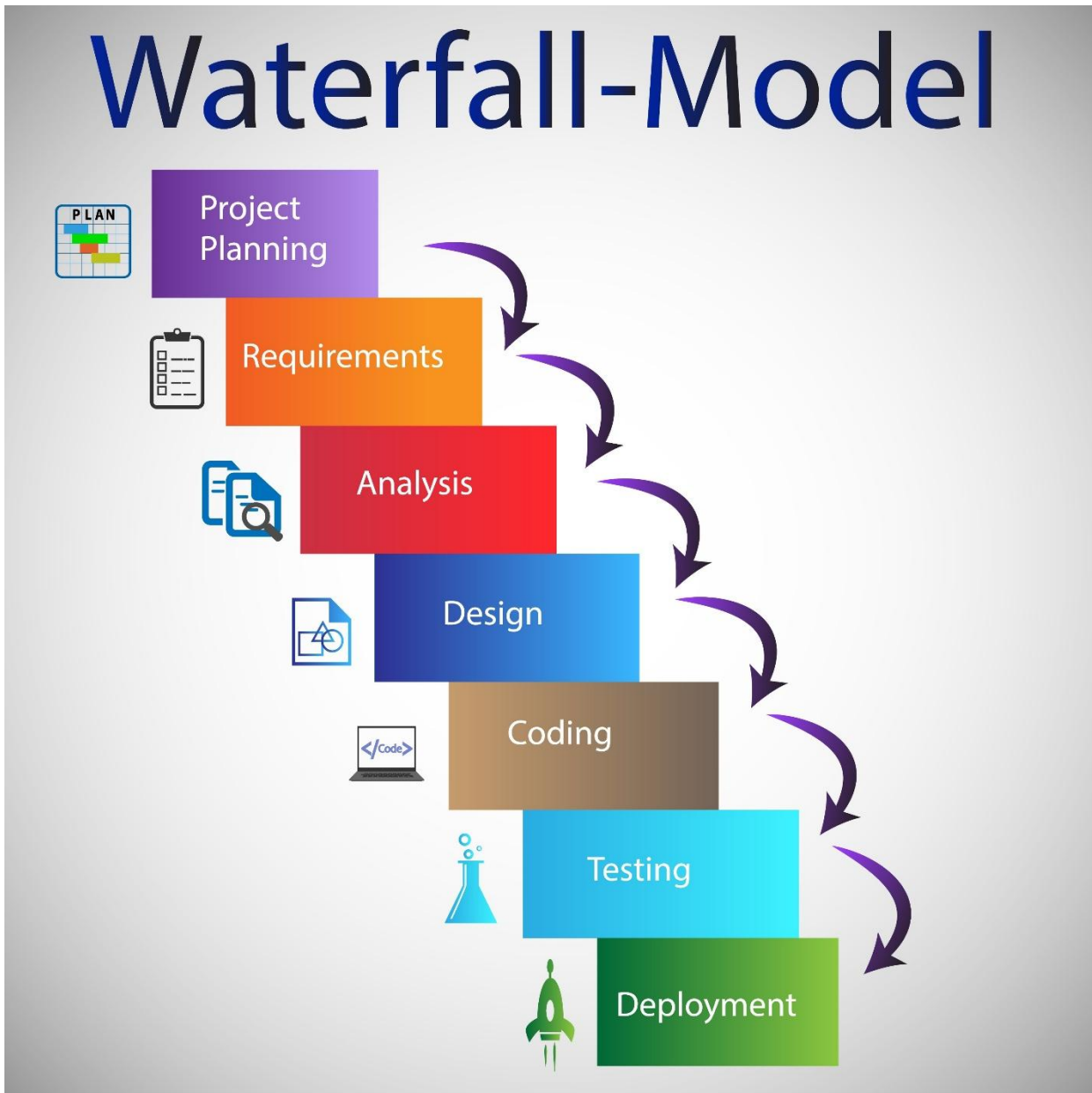
	Defining scenarios (in minutes\hours)	UAT Testing (in minutes\hours)	PRL Testing (in minutes\hours)	SP x 1.5h\1SP total
US 5 story point	2h	4h	1.5h	7.5h
US 8 story point	3h	7h	2h	12h
US 13 story point	5h	11h	3.5h	19.5h
US 20 story point (ideally must be split or re-estimated into smaller USs)	8h	16h	6h	30h

A US is estimated with 20 story points only with VP and division director agreement.

IMPORTANT!!! * These values are purely indicative, each team adjusts its estimates based on experience.

- Story points also reflect uncertainty – an 8 SP story can take longer if problems arise.
- Ideally, user stories should be small enough that they can be completed in one sprint.

08. Waterfall Workflow (*)



Shutterstock

Explore

The Waterfall (cascade) approach is chosen for a project when the requirements, context, and objectives fit a predictable, linear, and well-planned process.

Situations in which the use of Waterfall is decided:

- Requirements are clear and stable, well-defined and known from the beginning
- The project scope is predictable, without major uncertainties
- Detailed planning is possible from the beginning by phases/stages, deliverables and clear deadlines
- Validation is done at the end, the final product can be tested only after the complete finalization of the development

Marking in Jira the fact that a project is approached via Waterfall is done by selecting the Waterfall value in the Methodology field within the Epics that make up the project.

How does Waterfall work?

The process begins with the complete collection and definition of the requirements gathered under one or more Epics in Jira, followed by design, implementation, testing, delivery and, finally, maintenance. Each phase (Release) has clear objectives, detailed through tasks that take the form of User Stories in Jira and produces documentation that serves as a basis for the next stage.

Main characteristics

- **Sequential phases:** each phase (Release) begins only after the completion of the previous one.
- **Rigorous planning:** all requirements and work steps are defined before the development begins.
- **Detailed documentation:** each stage is documented for clarity and control.
- **Rigidity:** changes are difficult to manage after a phase is concluded.

Differences between Waterfall and Agile/Scrum methodology

Criterion	Waterfall	Agile / Scrum
Project types	Internal processes, IT infrastructure, compliance projects	Digital applications, automations, innovations
Planning	Sequential – fixed stages (analysis → development → testing → implementation)	Iterative – short sprints (2 weeks), continuous adaptation
Requirements	Fixed, completely defined at the beginning	Flexible, adjusted along the way after feedback and regulations
Business involvement	Limited – business participates at the beginning and at the end	Permanent – Product Owner + internal users involved in every sprint
Delivery	Single – complete product at the end of the project	Incremental – gradual deliveries of functional modules
Control & documentation	Emphasis on planning, budget and complete documentation	Emphasis on value, collaboration and minimal documentation

00. What is Waterfall? (*)

Waterfall Methodology – Project management methodology characterized by linearity and the chaining of several successive stages (analysis, development, testing and implementation). In the case of projects implemented applying the Waterfall methodology, the specifications are complex, the scope of the project is strict, defined at the beginning of the project, based on which the tasks leading to the project completion deadline are planned and estimated.

The project team is formed of:

- **Owner** - Is the person who requests the project or benefits from its implementation, representing the interests of the end user (if applicable)
- **Project Manager** - Is the person who must ensure a good progress of the project implementation. Also, facilitates the understanding and correct application of project management methodologies within the bank.
- **Business Analyst** – represents the person holding this function within the Process Quality Management division or in other divisions, who deals with the analysis of requirements registered in Jira and the drafting of functional specifications, which must reflect the business need and provide all the details necessary for the technical analysis, development, and testing of the requirement
- **Tester** – represents the person holding this function within the Process Quality Management division, but also the person from the Business divisions, who defines the test scenarios based on the functional specifications and who tests the developments to ensure implementation according to business requirements.
- **Developer** – represents the person from the IT Division allocated to implement the requirement in the IT applications.
- **Business resources** - Is the team responsible for the correctness of the requirement implementation, according to the specifications and within the planned deadline (together with the rest of the project team).

Projects carried out applying the Waterfall Methodology are implemented sequentially, in well-defined stages, planned at the beginning of the project by the Project manager and validated by the entire project team.

01. What is a Task in Waterfall? (*)

Within the Waterfall methodology for project management, a task represents a specific and delimited activity, a deliverable that must be achieved to advance the project towards its objectives.

These are opened in Jira in the form of User Stories within the project's Epics.

More concretely:

- A task is a clearly defined unit of work, with a well-established purpose - it is represented by the User Story within Jira
- In the Waterfall model, the project is divided into sequential phases / epics, and each epic is composed of multiple tasks (User Story).
- The tasks (User Story) are finalized in a strict order, without passing to the next phase before the completion of the User Stories from the previous phase.

Characteristics:

- They are well planned and scheduled.
- They have clear deadlines.
- Responsibilities are assigned to the team.
- The results of each task (User Story) contribute to the completion of the respective phase / epic.

02. Testing Process in Waterfall (*)

In Waterfall, the testing of User Stories begins after the code has been completely developed. Tests are not done in parallel with development, but only after all the requirements composing an epic are developed.

2.1 Planning Phase

- the tester analyzes the functional and technical specifications
- The following are defined in the project team:
 - Testing strategy
 - Test Plan
 - Test environments and necessary data
 - Involved resources

2.2 Writing test scenarios

- The test scenarios are written after the completion of the design phase (in which the general structure of the system, the design, how the components and modules are organized and how they communicate with each other are defined.)

2.3 Executing Tests

- The product is tested after the complete finalization of the development

2.4 Reporting defects & validating fixes

- Bugs are registered in Jira
- after their fixing, they are retested and validated

03. The role of a tester in Waterfall projects (*)

Waterfall stages and the tester's role

Phase	Description	Tester Role
 Requirements analysis	Collection and detailed definition of functional and non-functional requirements	Participates in reviews Contributes with testability requirements Initially estimates (high-level) the testing effort
 System design	Application architecture, data models, UI/UX structure	Participates in reviews

Phase	Description	Tester Role
		Analyzes design documents for testing planning and creates the first tests Details/refines the testing effort estimation
 Implementation	The dev team writes the code based on specifications	Writes the test scenarios
 Testing	The testing team executes the tests on the finalized build	Executes functional, regression, UI, integration, system testing etc.
 Launch	The application is delivered to production (LIVE)	Validates the final version through a smoke testing
 Maintenance	Post-launch fixes and improvements	Tests incidents, regressions, hotfixes, change requests

01. Defining test scenarios in Waterfall (*)

In Waterfall, test scenarios are defined after the analysis and design documentation has been approved, meaning:

- after the delivery of the Business Requirements Document (BRD),
- and the Functional Specification Document (FSD) / Technical Design Document (TDD).

Thus, testers work on fixed and approved requirements, without the risk of them changing during testing.

Test scenarios are built based on:

- functional and non-functional requirements (from the BRD and FSD),
- flow diagrams, screen mock-ups, technical design documents,
- business rules and use cases.

Defining the scenarios is done according to the general working mode, described here.

02. Executing Test Scenarios in Waterfall (*)

In Waterfall projects, test execution takes place after the complete finalization of the development phase.

The execution of the test scenarios is done according to the general working mode, described here.

03. Bug Reporting in Waterfall (*)

(This section was left blank in the source text)

04. Waterfall Estimates - Projects\ICDs (*)

Estimations in Waterfall projects are done in two stages:

1. In the requirements analysis stage, the tester participates in reviews, requests clarifications on the requirements and requests a check list with main points to track in testing the requirements, then makes an initial (high-level) estimation of their testing, in hours/days.

Elements taken into account in the estimation:

- testing experience on similar requirements
- novelty of the development/application
- complexity of the requirements
- project team feedback

2. During the design stage, based on the requirements documented in the specifications.

Here they will use the Testing Estimation Form .xlsx, which helps in providing more detailed and accurate testing effort estimations (for filling it out, it is indicated to be saved locally and then renamed for the purpose of the estimation)

- Title - Testing Estimates (Project Name, ICD or project)

For situations when the provided estimates for the Testing effort are contested by other roles, be they business or technical, the purpose of this form (and filling it out) is to support the estimated testing effort with details.

Usage instructions:

Estimates with values in hours will be entered only in columns B,C,D.

The activities (rows) marked with yellow are mandatory, the estimation is not considered complete without values for these activities.

The activities that are not marked with yellow can be considered optional but it is recommended that values be passed for them if it is known they will take place (based on past experience or information from project responsables).

Whole values or half hours can be used; Examples 1 or 0.5.

An attempt will be made as much as possible to pass values as exact as possible for each estimate without exaggerating plus or minus.

- **Optimistic Estimate (column B)** - The best scenario, in which everything goes well; the shortest time for the respective activity; there is no impediment, no delay to be able to start and finish the activity
- **Pessimistic Estimate (column D)** - The worst scenario, in which the most problems appear; the longest time for the respective activity; many impediments, many delays to be able to start and finish

the activity; here probabilities\past experience can be taken into account. The values for this estimate must always be higher than those for the Optimistic estimate.

- **Most likely estimate (Column C)** - The most realistic estimate based on experience; The values for this estimate must always be higher than those for the Optimistic estimate and lower than the Pessimistic estimate.
- **Realized time (column F)** - upon finalizing testing and completing all Testing activities (in all environments) the real time realized for activities will be passed.
- **Estimates observations:** reasons\extra information will be added that can support the estimate for the activity.

The Final Estimate that will be communicated is the value from cell E22.

09. Mobile Testing Workflow (-)

You can view the installation mode depending on the Scrum below:

Instalare MBK

 Libra Wallet– Installation and testing builds guide

List of projects involving mobile testing:

- Scrum Mobile Banking (MBK)
- Scrum Wallet

For any new build\release we must ensure that the mobile testing covers all important variants. The testing must be performed at least on the devices and OSs below:

- iOS (any version)
- Android (any version)
- Huawei (any version)

For major builds\releases the variants below must be additionally included: In progress

- iOS beta How to Opt-in to Apple Developer Beta Software on iPhone and iPad - MacRumors
- Android beta <https://www.google.com/android/beta#devices>

Below is an orientation guide for the tests that can be covered for Mobile Testing on the Libra applications

Functional testing

- **Authentication and authorization:**
 - Tests login mechanisms (user/password, multi-factor authentication (MFA), biometric authentication – fingerprint, Face ID).
 - Verifies the functionality of the password reset and account activation option.
- **Financial transactions:**
 - Verifies the execution of payments between accounts (internal and external).
 - Tests transfers, bill payments, recurring payments and sending money.

- Ensures that confirmation messages are correctly sent after each transaction.
- Tests the display of the current balance, transaction history and account statements.
- Tests functionalities such as card blocking/unblocking, requesting a new card, changing the PIN.
- Verifies the functioning of push alerts for banking notifications (payments, transfers, account limits).
- Tests the user profile modification functions, security options and language preferences.
- Verifies if the interface elements are displayed correctly and function according to expectations (buttons, menus, forms).
- Ensures that navigating in the application is fluid and without errors.
- Verifies if all main functions of the application work correctly (e.g. registration, authentication, transactions).
- If the application interacts with the hardware (camera, GPS, accelerometer), verify these functionalities.

Security testing

- **Secure authentication:**
 - Verifies if passwords are protected, if the application imposes a secure password policy and if multi-factor authentication (MFA) is implemented.
 - Tests lockout mechanisms after multiple failed authentication attempts.
- **Data protection:**
 - Verifies if sensitive data is encrypted both in transit and at rest.
 - Ensures that sensitive data is not kept on the device (e.g. information about bank accounts or cards).
 - Verifies if the application allows screenshots for sensitive banking data.
- **Common vulnerabilities:**
 - Ensures the application uses secure sessions and correctly expires the inactive session.

Usability and accessibility testing

- Ensures that all touch gestures (swipe, zoom, long press, etc.) work correctly and offer adequate feedback to the users.
- Tests for the consistency of design elements, texts, buttons and the overall experience on diverse devices and OS versions.

Permissions and notifications management

- Ensures that the application correctly requests, manages and revokes permissions (e.g. camera, location, microphone). Verifies if the application works correctly if permissions are refused.
- Tests push notifications, local notifications and background ones under different conditions (locked screen, application in background, etc.).

01. Defining Mobile Testing Test Scenarios (-)

Mobile test scenarios for an internet banking application represent structured sets of steps and conditions that verify the functionality, security, performance and user experience on mobile devices (IOS, Android and Huawei). Scenarios for testing on Android and IOS devices will be created for all types of testing (current User Story testing; smoke testing; regression testing), and will be used in both test environments (UAT and PRL). Scenarios for testing on Huawei devices will be created only once, as this is limited to validating smoke tests in the PRL environment. These ensure that the mobile banking application works correctly in diverse real situations, including authentication, transactions, navigation, notifications and account management, on multiple types and versions of devices and operating systems.

The scenarios are made according to the general working mode 01. How to generate test scenarios with AI - UiPath Test Manager (*)

The LX name must also contain the mobile device, according to the example in the image:

The process of defining scenarios for testing on mobile devices includes the following stages:

- **Creating the „Defining scenarios” sub-task:**

The sub-task is automatically created for each new User Story. If it is missing, it is manually added and is used for logging time and attaching the LXs corresponding to the scenarios.

- **Creating the LXs in the Xray Library:**

The tester identifies the mobile flows that must be tested (positive, negative, regression, UI/UX, device/OS compatibility etc.) and creates the scenarios respecting the structure: Action, Data, Expected Result.

Within the same test scenario, the tester will obligatorily verify both the Romanian language page of the application, as well as the corresponding English language page. The purpose is not to verify the functionality (behavior which should be the same in both pages/languages), but the language displayed on the corresponding pages and identifying possible insertions of Romanian/English in inappropriate pages. In case there are USs/CRs that specifically contain text additions/modifications, these texts will be specified in the expected results of the test steps (RO variant + EN variant).

The LXs are saved in the corresponding subfolder and a label with the User Story number is added.

- **Cloning the LXs:** The LX is created a single time for one type of OS then it is cloned and the name is modified with the tested OS (Android, IOS). Thus, there will be a test scenario for each of the two devices, Android and IOS.

How cloning LXs is performed in Xray - Jira:

1. Access the test (LX) you want to clone;
2. Press the “More” button;
3. Select “Clone” from the menu;
4. Press “Clone” to generate the new LX;
5. After creation, you can edit the new LX to make the necessary adjustments (e.g.: update test steps, modify Summary - where we add the used OS type).

- **Adding LXs on the „Defining scenarios” sub-task:**

After creation, the LXs are added in the Description field for visibility and validation by the PO/Application Owner.

- **Logging time:**

The tester daily logs the time allocated for writing and validating the scenarios in the „Defining scenarios” sub-task.

- **Validating scenarios:**

The scenarios are reviewed together with the PO/Application Owner. After validation, the sub-task is marked as DONE.

- **Updating and reusing tests:**

If the mobile flow already exists in the library, Call Test is used to avoid duplication. If it does not exist, it is documented and added for later reuse.

02. Executing Mobile Testing Scenarios (-)

The execution of mobile testing for a Mobile Banking/Wallet application consists in actually running the test scenarios on real devices, to validate the application's behavior under varied usage conditions. This process implies verifying essential functionalities like authentication, performing transactions, receiving notifications or navigating between sections, ensuring compatibility with different operating system versions (iOS, Android, Huawei). Testing is done manually, involving validations of the graphical interface, performance, security and user experience. Real scenarios such as internet connection loss, use of biometrics or accessing the application in the background are also simulated, to guarantee the application's stability and reliability in the mobile environment.

For more details see here 02. Executing Test Scenarios Workflow (*)

The execution process of test scenarios on mobile devices includes the following stages according to the working mode here: 02. Executing Test Scenarios Workflow (*)

1. Creating Sub Test Execution

For each User Story, a Sub Test Execution is created (automatically or manually). In it, the time necessary for testing on different devices and OS versions is estimated.

2. Adding the LXs

The scenarios (LXs) relevant for mobile testing, selected by number, name or JQL filter, are added into the Sub Test Execution. Only the tests that are to be run in that phase are included.

3. Executing the scenarios

Installing the Mobile application and testing Builds can be performed following the steps from: Instalare MBK and  Libra Wallet– Installation and testing builds guide

Testing on mobile is performed exclusively on physical phones with different operating systems (iOS, Android and Huawei). Before execution, the application is installed and the test environment specific to each device is configured. To display and control the mobile screen on the PC, ApowerMirror is used.

- On the UAT Testing, PRL Testing and Regression Testing Sub test Execution, testing is performed only on IOS and Android.
- On the Smoke Testing Sub test execution, testing is performed on all operating systems: IOS, Android and Huawei.

The tester runs the steps on real mobile devices accessing the Libra Mobile Banking or Wallet application and referencing the page, marking the result of each step (Pass, Fail, Blocked, Aborted). In case of Fail, bugs are reported and screenshots or recordings are attached.

4. Logging time

The tester daily logs the estimated time and the actually consumed time in the Sub Test Execution for mobile testing activities.

It is no longer necessary to attach evidence for any type of testing: LX execution, UAT or Prelive testing, SCRUM or Waterfall testing, including in bug fixes retesting; evidence in bug reporting remains.

03. Mobile Testing Bugs Reporting (-)

Reporting bugs in mobile testing is an essential step in the quality assurance process of mobile applications. A correctly and completely reported bug helps the development team quickly understand the problem and fix it efficiently.

- In case of identifying Bugs during scenario execution on any type of testing, a BUG type Issue is opened following the steps here.
- Attaching evidence on the Bug: Print screens or Video detailing the encountered problem → is performed with ApowerMirror.
- Before opening the Bug, the encountered problem is obligatorily tested on all OS types (IOS, Android; and Huawei → in the case of bugs appearing on smoke tests).
- To complete the bug details, it is necessary to specify the operating system (OS) and mention if the problem reproduces on a single device or on multiple in the "Bug detailing" field
- After fixing the Bugs the tester has the obligation to retest them, on all OS types according to the workflow defined in 04. Retesting a bug fix (*)
- Logging the time for retesting Bugs is performed according to 10. Logging time in Jira (*).

Additionally, details about the build tested on the mobile applications and the OS on which it was identified are added in the bug.

ApowerMirror

ApowerMirror is a program used for mirroring (projecting) the screen of a mobile device onto a PC. Within our team, the application is used exclusively for testing activities on mobile devices, especially for:

- holding Demo meetings, where the functionalities of the tested applications and their behavior on different mobile devices are presented.
- screenshots or recording bugs identified during testing sessions on phones.

ApowerMirror represents a convenient solution for displaying, capturing and recording the phone screen directly on the computer, offering the possibility to quickly report problems and demonstrate test scenarios in real-time.

1. Installing ApowerMirror Installing the ApowerMirror application can be done only by opening a ticket in RT to the IT support team, who handles installing the application on the requested computer.

2. Connecting the phone to the PC

There are two main connection methods:

- A. Via USB cable - Android**
1. Download ApowerMirror on the phone as well (Google Play). Start the application on both.
 2. On the phone, activate USB Debugging from (Settings → About phone → tap 7 times on Build number → Settings → Developer options → USB debugging ON).
 3. Connect the phone to the laptop via USB cable. A prompt appears on the phone: Allow USB debugging? → select Allow.
 4. ApowerMirror will automatically detect the device and start displaying the screen on the PC, following the steps from the print screens.

B. Via Wi-Fi (without cable) - IOS, Android and Huawei

1. Ensure the phone and PC are connected to the same Wi-Fi network.
2. Open the ApowerMirror application on both devices.
3. On the phone, press "Mirror" and select the Apowersoft name and PC name.

C. How we take screenshots and video from ApowerMirror

After the phones have been connected to the laptop, through any of the variants above: A and B, the screen is displayed on the Pc from where we can take screenshots or video.

For screenshots we use the first image and to take a video the second image.

Smartwatch testing (-)

The purpose of smartwatch testing for a mobile application that cannot be installed on the smartwatch is to verify the correctness and stability of the interaction and synchronization between the phone and watch, ensuring that notifications and supported functionalities are displayed and work correctly on the smartwatch, without direct installation, offering a fluid, safe and accessible experience to the user.

Orientation guide\Checklist

Tested aspect	Description	Concrete example / Testing steps	Expected result
Connectivity between devices	Bluetooth pairing and connection stability test	Pair phone and smartwatch; verify reconnection after interruptions	Connection is established quickly and is stable
Data synchronization	Correct transmission of notifications and data	Send notifications and verify synchronization on smartwatch	Notifications appear correctly and on time on the watch
Notifications display	Verifying clear and complete notification display	Send messages with emojis, links and small images; verify display	Notifications are clear and complete on the smartwatch screen

Tested aspect	Description	Concrete example / Testing steps	Expected result
Interaction with notifications	Testing notification response and management	Reply quickly, delete notifications and mark as read on smartwatch	Replies and modifications synchronize correctly
Behavior upon connection loss	Managing situations when connection is interrupted	Disable Bluetooth and observe app reaction	The app detects connection loss and manages it
Permissions and security	Verifying notification and access permissions	Revoke permissions and test impact on notifications	The app respects permissions and notifications don't appear if permissions are missing
User feedback	Confirming haptic, visual and auditory feedback	Interact with notifications on smartwatch and observe feedback	Feedback is clear, coherent and immediate
Smartwatch compatibility	Testing on different smartwatch models and versions	Test the app on Apple Watch, Samsung Galaxy Watch etc.	Functionalities run correctly on all tested models
Real usage scenarios	Testing notifications under usual conditions	Receive notifications with phone in pocket and answer on the move	The app works correctly in all real situations
Multiple notifications management	Simultaneous testing of multiple notifications	Send multiple notifications and verify order, grouping and their management	Notifications are displayed, grouped and managed correctly

UI\UX Testing for Mobile (-)

Laura looks here

UI/UX Testing (User Interface / User Experience Testing) is the process of verifying how an application looks and feels for users, to ensure it is intuitive, accessible and easy to use.

For the scrums: Scrum IBK, MBK, WALLET and DIGITAL, separate UI/UX folders were created in Xray.

Testing is done in both UAT and PRL test environments - UI/UX testing is done on each US individually.

to move into CREATING LX - The LX will be named UI/UX Testing and the name of the US and the UI/UX label

UI Testing (User Interface Testing)

Focuses on the visual aspect and graphical elements of the application:

- Verifies if UI elements (buttons, fields, menus, icons etc.) are:
 - Correctly displayed on all devices
 - Arranged and aligned properly
 - Functional (e.g., when you press a button, the expected action happens)
- Responsive design behavior is tested (UI behavior at various screen sizes)
- Ensures visual coherence (fonts, colors, consistent icons)

UX Testing (User Experience Testing)

Refers to the overall user experience when using the application:

- Is the app easy to navigate?
- Are actions intuitive and well explained?
- Does the user receive clear feedback when taking an action (e.g. transaction confirmation)?
- Is the app fast and crash-free?
- Does it work correctly for all users, including those with disabilities? (accessibility)

Concrete examples in UI/UX testing

1. Design consistency across different sizes and resolutions

- Verify if the UI adapts correctly on small, medium and large screens (e.g.: compact phones vs. phablets vs. tablets).
- UI elements must not overlap, disappear or be too small/large.
- Test on devices with different densities (DPI).

2. Adaptability to different orientations

- The UI must rearrange correctly in portrait and landscape modes.
- No element should disappear or be hard to access when the screen rotates.
- Flows must remain intuitive regardless of orientation.

3. Specific and natural gestures

- Verify all common gestures (tap, swipe, pinch, long press etc.) respond correctly.
- Test native platform gestures (e.g.: swipe back on iOS, back button on Android).
- Custom application gestures should be intuitive and conflict-free.

4. UI Performance under real conditions

- Response times to touch, scroll, animations without lag.

- The application should function fluently even on older devices or with limited resources.
- Immediate visual/haptic feedback to actions.

5. Application state management

- Test how the application behaves when moved to the background and then brought back to the foreground.
- Verify if data, current form or transactions in progress are preserved.

6. Testing on varied and offline networks

- The application must communicate clearly when it has no connection and offer offline functionalities where applicable.
- Verify responses to network interruptions, reconnections etc.

7. Clear feedback and confirmations

- Animations on press, changing button colors or popup messages for action confirmation.
- Haptic (vibration) or audio feedback where relevant.

8. Ergonomics and ease of use with one hand

- Verify if important elements are accessible with the thumb, especially on phones with large screens.
- Buttons too small or positioned in hard-to-reach areas affect UX.

9. Mobile accessibility testing

- Compatibility with screen readers (e.g.: TalkBack on Android, VoiceOver on iOS).
- Color contrast and font size should be good even in special modes (night mode, high contrast mode).
- Easy navigation and use without touchscreen (e.g.: voice commands).

10. Permissions and notifications management

- Clear messages when the application requests permissions (e.g.: location, microphone).
- Application behavior if permissions are refused.
- Correct interaction with push notifications and navigation from them.

11. Testing behavior in special conditions

- Battery saving mode (low power mode) should not degrade UX.
- Multitasking and split screen on Android should maintain functionality and readability.
- Testing in real conditions: bright light (outside), ambient noise, use while moving.

12. Simplicity and clarity of mobile flows

- Minimize the number of steps for usual actions (e.g.: payment, money transfer).
- Simple form, with live validations and useful messages.
- Avoiding overloading the user with useless elements on the screen.

This UI/UX checklist helps verify the visual and experience elements within a mobile application. The purpose of this checklist is to standardize the testing process, increase efficiency, better document UI\UX tests and cover this type of testing as well as possible. The final goal is to offer users an intuitive, accessible and pleasant experience, avoiding errors and frustrations in use.

This checklist must be treated as an orientation guide, not all actions can be tested in every application area.

Tested aspect	Description	Concrete example / Testing steps	Expected result
Tap (Simple touch)	Fast pressing of an element	Quickly press a button or link in the app	The element is selected/activated without delays
Double Tap	Fast pressing twice consecutively	Double tap on an image for zoom	The image enlarges/zoom activates correctly
Long Press	Holding pressed for a longer time	Hold an element pressed to show context menu	The contextual menu appears and works correctly
Swipe	Fast finger movement in one direction	Swipe from right to left to navigate between screens	Navigation is fluid, without blockages
Pinch to Zoom	Bringing together or spreading two fingers	Zoom in/out on a map or image	Zooming is fluid, without lag or errors
Spread	Movement with two fingers from center to exterior	Spread two fingers for quick zoom in	Content enlarges instantly
Rotate Gesture	Rotating an object with two fingers	Rotate an image using two fingers	The image rotates accordingly, without interruptions
Drag and Drop	Holding and moving an element	Hold and move a file into a folder	The element moves correctly and appears in the new location
Flick	Fast and short swipe	Flick quickly through a list of transactions	The list scrolls quickly and fluidly

Tested aspect	Description	Concrete example / Testing steps	Expected result
Pull to Refresh	Swiping down to update content	Swipe down on the notifications list	The list refreshes and displays new content
Edge Swipe	Swiping from screen edge for side menu	Swipe from the left edge to open the hamburger menu	The menu appears quickly and is functional
Two-Finger Scroll	Scrolling content with two fingers	Use two fingers to scroll enlarged content	Scrolling is smooth and without errors
Shake Gesture	Shaking the device for special action	Shake the phone to activate the "Undo" function	The Undo action is triggered correctly
Back Gesture (Swipe Back)	Swiping left-to-right for previous screen	Swipe from left to right to return to previous screen	Navigating back is fast and error-free
Force Touch / 3D Touch	Pressing with force on screen (compatible devices)	Press hard on an icon for quick menu	The quick menu appears and works correctly
Multitouch Gestures	Simultaneous use of multiple gestures	Apply pinch + rotate simultaneously on an image	Gestures are recognized and combined correctly
Double Finger Tap	Double tap with two fingers	Double tap with two fingers to reset zoom	Zoom resets instantly
Two-Finger Rotate	Rotating an element with two fingers simultaneously	Rotate an object using two fingers simultaneously	Rotation is fluid and precise
Hover Gesture (with stylus)	Suspending the stylus above the screen	Hold stylus suspended to display context menu	Contextual menu appears promptly

Tested aspect	Description	Concrete example / Testing steps	Expected result
Two-Finger Tap	Touch with two fingers simultaneously	Touch with two fingers simultaneously for a special function	The function activates/deactivates correctly
Triple Tap	Three fast consecutive touches	Perform triple tap for accessibility zoom	Accessibility zoom activates without delay
Swipe and Hold	Swipe followed by prolonged press	Swipe and hold to select multiple elements	Multiple selection activates and extra options appear
Pinch and Rotate Simultaneous	Combining zoom and rotation in one gesture	Apply pinch and rotate simultaneously on an image	Gestures are applied correctly and simultaneously
Press and Slide	Immediate press and drag	Hold pressed on the slider and drag to adjust	The slider responds precisely to adjustment
Double Tap and Hold	Double tap followed by holding	Double tap and hold for quick access to options	Quick menu opens without errors
Scroll	Vertical/horizontal movement with one finger	Scroll a long list of transactions	Scrolling is smooth, without blockages
Swipe with Velocity	Fast swipe with high speed for rapid scroll	Swipe quickly to navigate through long pages	Navigation is fast and fluid
Back button Android	Press back button to return to previous screen	Press hardware or software back button	Navigating back is correct and predictable
Home button / Gesture	Fast return to main screen	Press Home button or use Home gestures	Main screen appears without errors

Tested aspect	Description	Concrete example / Testing steps	Expected result
Screen rotation	Rotating phone between portrait and landscape	Rotate the phone and check UI	UI adapts correctly, without overlapping elements
Low battery mode	Testing in battery saving mode	Activate saving mode and use the app	App works normally with maintained performance
Offline mode	Deactivate connection and test functionalities	Turn off connection and check available messages and functions	App offers clear messages and works partially
Resume from background	Returning to app after minimizing	Minimize and return to the app	App keeps its state without resetting
Biometric authentication	Using fingerprint / facial recognition for login	Authenticate with fingerprint or face	Fast and secure login
Push notification interaction	Opening notification and checking navigation	Tap on a received notification	App correctly opens the relevant screen
Permission handling	Refusing permissions and checking UX messages	Revoke location permission and use the app	App displays clear messages and respects the decision
Vibration and sound feedback	Testing haptic and audio feedback on actions	Perform actions and check vibrations/sounds	Feedback is clear and synchronized
Gesture conflicts	Testing if gestures do not interfere	Apply different gestures simultaneously or successively	Gestures don't block and are recognized correctly
Multitasking	Maintaining data when app is in background	Switch app and come back	Data and state are kept

Tested aspect	Description	Concrete example / Testing steps	Expected result
Accessibility gestures	Testing special gestures for users with disabilities	Use accessibility gestures and check functionality	Gestures are recognized and work correctly
Multi-window mode	Testing in split screen on Android	Use split screen with the application	App works correctly in multi-window mode
Custom gestures	Testing application specific gestures	Use unique gestures defined in the app	Gestures are recognized and execute corresponding actions
Session timeout	Testing session expiration after inactivity	Leave app inactive and check session expiration	App logs out or shows timeout message
Visual feedback for actions	Checking animations and visual changes on interactions	Press buttons and observe visual effects	Visual feedback is clear and instantaneous
Form simplicity	Testing ease of filling out a form	Fill out payment form with validations and messages	Form is easy to fill, with clear messages
Design consistency	Checking consistency of style, colors, fonts between screens	Navigate between different screens	Design is coherent and professional
Error messages and instructions	Clarity and usefulness of messages in case of error	Enter incorrect data and check messages	Messages are clear, useful and help the user
Loading performance	Checking loading times and feedback	Open screens and observe loading time and animations	Loading is fast, with clear feedback for the user

10. Logging time in Jira (*)

Logging time in Jira is mandatory. Depending on the activities carried out in a day, the related time is logged on the dedicated tasks/subtasks.

To log the time spent on defining scenarios, UAT/PRL/Smoke PRL/Regression PRL testing, bug retesting, scrum ceremonies, access the option More → Log Work:

In the opened pop-up, only the 2 fields will be filled in:

-Time Spent: *the estimate of the time you spent writing the test scenarios*

-Date Started: *date and time* (The time is less important. Time logging can be done at any time of the day. It is recommended that by the end of the day, the time spent on the testing activity of that day be logged)

Logging in Jira tasks outside the team\project

In this task, the time dedicated by testers for courses, materials, and workshops specific to the testing activity will be logged. - [ATQ-40] Participating in workshops, courses, materials specific to the testing department - Jira

In this task, the time dedicated by testers for 1:1 meetings with the superior will be logged. - [ATQ-41] 1:1 with superior - Jira

In this task, the time dedicated by testers for documenting the activity in Confluence, as well as for internal presentations (e.g., internal Scrum teams Demo) will be logged. - [ATQ-42] Confluence documentation and internal presentations - Jira

In this task, the time dedicated by testers for internally organized meetings (e.g., TestersQM meeting) will be logged. - [ATQ-43] Testing department calls - Jira

In this task, the time dedicated by testers for buddy activities or on support activities for colleagues in the testing team will be logged. - [ATQ-44] Buddy/Testing support - Jira

In this task, the time dedicated by testers for defining Smoke scenarios will be logged. - [ATQ-39] Defining Smoke test scenarios - Jira

In this task, the time dedicated by testers for defining Regression scenarios will be logged - ATQ-47 - Jira project doesn't exist or you don't have permission to view it.

In this task, the time when the test environments are unavailable will be logged: [ATQ-36] Test Environments Unavailable - Jira

In this task, the time dedicated by testers during the Onboarding period will be logged. - [STAM-75] Time logging - Onboarding - Jira

In this task, the time dedicated by testers for administrative activities will be logged. - [ATQ-101] Administrative activities - QM Testing - Jira

11. Scrum teams back-up system workflow (*)

- **The purpose and benefits of the back-up system between scrum teams:**

We want to encourage active involvement in providing support to other teams when the testing volume is high. This collaboration brings benefits in several respects:

→ Increased productivity: involvement in testing other projects contributes to improving the results in the productivity report

- Reducing dead times – during less crowded periods, involvement in other teams ensures optimal loading.
- Professional development – you will have the opportunity to explore new applications, flows, and technologies, to see different approaches in other scrum teams.
- Consolidating collaboration between teams – the exchange of experience will help you collaborate better with each other and have common knowledge.

- **The moment of holding the training:**

- during Smoke tests (for general familiarization)
- during partial regression tests (for deepening the relevant functionalities)

- **Identifying training pairs:**

Each primary team will hold training for the allocated secondary teams, according to 01. Allocations primary team vs secondary team - QA - Confluence.

There will be 2-3 scrum teams that will be back-up for the primary team.

- **Responsibilities:**

Primary team:

- 1 representative will be designated at the team level to hold the training
- The designated trainer will announce in advance the teams for which they will hold the training, so they can free their schedule.
- Schedules the meeting in the calendar on Teams, with CC to the involved TLs.
- Holds the training meeting, records the training, and uploads it in the Confluence section – Testing Scrum Teams.
- Provides subsequent testing support.

Secondary team:

- Actively participates in the training sessions
- Asks questions related to the presented flows, so as to ensure they understand the specifics of the scrum.

- **Training structure:**

- The training will last approximately 3h, time in which the tester will execute the Smoke tests and a series of regression tests (the most important ones)
- A general presentation of the application and the tested flows will be made.
- Smoke Testing and Partial regression testing – demonstration and explanations for each validation
- Details about execution conditions, validation criteria, and other useful information are provided.
- The trainer will have to ensure that after the held training they will provide the support teams with the necessary materials: access links to applications, test environments and data, useful users, Confluence documentation etc.

- **Conducting the activity of the back-up team**

Testers who offer support will be able to test in several situations, depending on their experience and previous collaboration with the secondary scrum:

→ If the tester has experience in the secondary scrum: the complete testing of a User Story will be done: the stages of defining scenarios, UAT and PRL testing (will help increase the results in the productivity report for all KPI indicators);

→ If the tester has little experience collaborating with the secondary scrum: the execution of all tests will take place after the moment the AI-generated test scenarios are validated by the permanently allocated tester in the team;

→ If the tester has no experience at all and takes over requirements from the secondary scrum for the first time: they will run the smoke and partial regression tests.

- **Workflow for taking over activity in the secondary scrum**

When a Tester does not have enough activity in the scrum team they are allocated to, before taking over a testing task in the secondary team, they will open a Task in Jira (in the scrum team's project) with the category Available for tasks.

Upon opening a task with the category “Available for tasks”, the following notifications are automatically sent:

- Email to the Project Lead of the SCRUM team's project, the PM of the SCRUM team, and the Reporter
- Email to the TLTesting email group

Automatically sent email:

- Subject: Resource available for task allocation
- Body: A resource from the team is available for task allocation [task link]

[First Name Last Name Reporter] - [Role]

After completing the task in the secondary team, the Tester will close the task setting the Done status.

→ For situations where the support offered is during the 3-month project period that qualifies for bonuses, the Testers providing support will be included in the bonus system. Their allocation will be made in CSI based on the previously opened Task announcing the Availability to test in another scrum team.

01. Allocations primary team vs secondary team

The allocations in the table below are made at the Scrum team level.

In case the Testers have offered support in the past in other teams or due to the activity within the bank, they have interacted with another Scrum before, they can offer support in other teams compared to those mentioned in the "Team holding the training" column.

Team holding the training	Teams that will participate in the training
Scrum Flow	Scrum Flow Automatizari

Team holding the training	Teams that will participate in the training
	Administrare Credite
Scrum Bug-uri si Incidente	Scrum Flow Scrum_Administrarea Riscurilor
Scrum Flow Automatizari	Scrum Flow Scrum_Analiza Credite
SCRUM_Operatiuni	Scrum_Creatio Deschidere Client Scrum_CP
Scrum_Creatio Deschidere Client	SCRUM_Digital SCRUM Alte Divizii
Scrum_Decontari	Scrum_CP SCRUM_Digital HUB
Scrum_Analiza Credite	Scrum Flow Scrum Flow Automatizari

Team holding the training	Teams that will participate in the training
SCRUM_IBK	SCRUM_MBK SCRUM_LibraPay
SCRUM_MBK	SCRUM_IBK Scrum_Libra Wallet
SCRUM_Digital	Scrum_Creatio Deschidere Client SCRUM_Digital HUB
Scrum_CP	SCRUM Alte Divizii Scrum_Decontari
SCRUM_LibraPay	SCRUM_Carduri SCRUM_Digital
SCRUM_Digital HUB	SCRUM_LibraPay Scrum_Decontari
SCRUM_Administrare Credite	Scrum Flow Scrum_Analiza Credite

Team holding the training	Teams that will participate in the training
SCRUM_Carduri	SCRUM_IBK Scrum_Libra Wallet
SCRUM_Conformitate	Scrum_Administrarea Riscurilor
Scrum_Libra Wallet	SCRUM_Carduri SCRUM_MBK
Scrum_Administrarea Riscurilor	SCRUM_Administrare Credite SCRUM_Conformitate

12. Integrated Testing Workflow

Integrated testing in the UAT environment takes place after the completion of UAT testing within each involved cross team. Its purpose is testing end-to-end scenarios. In this stage, the integrated scenarios associated with the main flows are run, to confirm that all involved components work coherently and that the transitions between applications are stable.

Integrated testing is essential to ensure that the different software components are integrated without problems and work together without errors or defects. It plays a significant role in delivering high-quality software that respects the client's expectations and specifications.

Planning

- **UAT Integrated Testing:**
 - Can begin only after all teams have completed the UAT testing on the specifications to be integratedly tested.
 - Before the DEMO, to be able to present the integrated flow.
- **PRL Integrated Testing:**
 - After the final code merge (a dedicated period for the merge might be necessary).
 - Can begin only after all teams have completed the PRL happy flow testing on the specifications to be integratedly tested.
 - After the smoke tests, the happy flow LXs are validated, followed then by the regression testing.

1. Within the ICD, an "Integrated Testing" issue type is created, according to the information on the page Create "Integrated Testing" Issue type.
2. In the Xray library, it is necessary to create a sub-folder with the name of the cross project, following the steps from Create Xray sub-folder, in the "Integrated tests (Cross-team Projects)" folder.
3. Defining scenarios
 - Testing TL creates an Excel file and organizes a meeting with the BA and testers where the end-to-end flows are established.
 - Existing LXs are reused or created from scratch. More information about creating LXs can be found on page 02. Creating tests in Xray (*) - QA - Confluence.
 - LXs are moved into the integrated testing folder and labeled accordingly.
 - Collaboration between teams for the complete coverage of the flows.
4. UAT Integrated testing
 - A Test Execution is created on the "Integrated Testing" issue type.
 - The LXs from the integrated testing Xray folder related to the project are added.
 - The LXs are executed according to the end-to-end flow; steps marked Pass / Fail; bugs reported accordingly.
 - After execution and bug retesting, the Demo meeting is organized for obtaining acceptance.
5. PRL Integrated testing
 - A Test Execution is created on the "Integrated Testing" issue type.
 - The happy flow type LXs are added.
 - The LXs are executed strictly according to the main flow; steps marked Pass / Fail; bugs reported accordingly.
 - After execution and bug retesting, the flow can be promoted to Golive.

01. Create Integrated testing issue type

At the beginning of the project, in the planning phase, an "Integrated testing" issue type will be created, under the ICD project, with "Parent link" to the ICD for which the integrated testing is performed.

The person responsible for opening this issue and the following testing sub-tasks below is the QA Team Lead who is allocated as coordinator on the project.

Recommended sub-tasks

1. Defining Scenarios
2. UAT Integrated Testing
3. PRL Integrated Testing

02. Create Xray sub-folder

How to add a new sub-folder

1. Select the parent folder "Integrated tests (Cross-team Projects)" and press the “+” icon at the bottom of the page
2. Modify the sub-folder's name from "New folder" into the project's name

03. Defining scenarios

Within the UAT Integrated testing, defining the scenarios represents a critical activity:

1. The Testing Team Leader is responsible for:
 - creating an Excel file that will contain multiple sheets (one for each involved team) and a general sheet structuring the high-level flow and the preconditions for starting integrated testing;
 - organizing an alignment meeting with Business Analysts (BA) and testers, where the flows to be tested are discussed.
2. Identifying prerequisites / test data
 - All testers work in close collaboration with BAs, POs to identify and use relevant test data for testing the end-to-end flow.
 - The identified prerequisites will be noted in the Excel file created by the TL, so they can be handy for all involved testers.
 - Examples: test environment, users/rights, account types, client types, signature types, CUI/CNP etc.
3. Defining the end-to-end flow
 - The first sheet from the Excel file will contain the end-to-end flow, established following the meeting with the BA and testers;
 - This flow represents the basis for defining the integrated testing scenarios;
 - The Excel file (or relevant sheet) can be attached to the task where the testing will be performed.
 - End-to-end flows will contain both positive and negative scenarios and edge-cases.
4. Reusing and creating scenarios (LXs)
 - Existing LXs, created previously for testing at the User Story (US) level, will be reused as much as possible;
 - if other relevant scenarios are identified compared to the already defined ones, they will be defined, according to the established end-to-end flow.
 - Tests are created manually (not AI generated – the AI option to be evaluated in the future).
5. Organizing scenarios for integrated testing (Xray)
 - The LXs used for integrated testing will be:
 - moved / created in the dedicated integrated testing sub-folder of the cross project;
 - labeled (labels) with:
 - integratedtesting (testareintegrata)
 - the related ICD number.
 - the scenarios will be organized in the common scenarios folder, which can contain:

- scenarios merged into a single LX, coming from different teams;
- scenarios chronologically ordered in multiple LXs, so that the logical thread of the implementation can be followed when running them.

6. Collaboration and complete flow coverage

- The test prerequisites will be established from the beginning, and then all involved teams will use them progressively;
- All testers work in close collaboration to define LXs that:
 - cover all the transitions of a flow;
 - include all the influenced applications;
 - are adapted depending on the tested cases.

04. UAT Integrated testing

UAT Execution – Integrated testing

Test execution within UAT – Integrated testing is performed based on the previously defined scenarios (LXs) and aims to validate the end-to-end flows, according to business requirements.

1. Creating the Test Execution

- At the ICD level, an "Integrated testing" type issue will be opened and related to a User Story, which will be opened by the Testing TL on the ATQ testing project in Jira.

This User Story will contain the sub test execution for UAT Testing - "UAT Integrated testing".

2. Adding scenarios (LXs)

- Within the Test Execution, all LXs defined for the integrated testing will be added. For more details on how to add LXs, see the page 01. Adding LXs on Sub Test Execution (*) - QA - Confluence
- The scenarios will be added:
 - from the integrated testing folder;
 - appropriately labeled (integratedtesting, ICD number);
- The LXs must reflect the chronological end-to-end flow established in the scenario definition stage.
- The test data must be common for each flow (e.g., the same CUI/CNP used throughout the entire integrated testing flow - from application 1 to application 5). This implies close collaboration between BA, PO and testers, to ensure that the used data is relevant for the flows to be tested and that it will be used by all involved testers, depending on the moment the flow is at.

3. Preparation for execution

- Before running, the testers will verify:
 - the existence of all necessary LXs;
 - the logical order of the scenarios, where this is relevant;
 - the clarity of the steps and the correctness of the test data to be used for the end-to-end flow;
- Any identified unclarity will be corrected before execution begins.

4. Executing scenarios

- The LXs will be executed according to the established order, following the transitions between applications and the complete validation of the flows;
- For each LX:
 - steps will be marked as Pass / Fail;
 - relevant observations will be added, if applicable.
- Each tester will be able to run their steps from a common LX or their own LXs, by modifying the assignee on the sub-test execution.

5. Defect management

- If a defect is identified:
 - a bug associated with the failed step will be created. The steps for opening a bug are found in Integrated Testing Bug - QA - Confluence
 - the created bugs will be linked through "relates to" with the "Integrated testing" issue type

6. Finalizing UAT execution

- After running all LXs:
 - the general status of the Test Execution will be checked;
 - the complete coverage of the end-to-end flows will be confirmed;
 - it will be confirmed that critical defects have been resolved (by closing all bugs).

7. Requesting DEMO Preview meeting

- Being a project with high complexity, a DEMO Preview meeting will be requested for validating the deliverables before the official demo, according to the workflow described here: 2. Validating flows - DEMO Preview meeting - QA - Confluence

8. Holding the DEMO meeting

After completing the tests in the UAT test environment and after retesting the bugs, the Demo meeting is held to obtain acceptance:

- Scrum Master sends an invitation for the Demo meeting to: Product owner, Project Beneficiaries and other involved persons (BAs, developers, other people from business or involved in the project).
- The Demo meeting is held by the involved Testers, each of them being responsible to present the tested area, respecting the steps from 04. DEMO Presentation in Agile\SCRUM with the purpose of obtaining acceptance.
- After obtaining acceptance following the Demo meeting, the UAT Integrated testing Sub Test Execution status is modified from In Progress to Done. (the time spent in the Demo meeting is logged on the UAT Integrated Testing Sub Test Execution).

05. PRL Integrated testing

Execution in PRL follows the same structure and the same steps as the UAT Execution – Integrated testing, with the following specific differences:

1. Creating the Test Execution

- The User Story opened by the Testing TL on the ATQ testing project in Jira and related to the "Integrated testing" type issue will contain, besides the sub-test execution for UAT Testing, the sub test execution for PRL Testing - "PRL Integrated testing".

2. Purpose of testing

- In PRL, exclusively happy flow end-to-end scenarios are validated;
- The complete coverage of end-to-end flows or all possible transitions is not pursued.

3. Selecting scenarios (LXs)

- Only happy flow type LXs are included;
- Do not add:
 - negative scenarios;
 - alternative scenarios;
 - edge cases;
- Existing LXs (scenarios previously validated in UAT) can be reused, without expanding them beyond the main flow.

4. Executing scenarios

- The scenarios are executed strictly according to the main flow, without intentional deviations.
- Each involved tester will execute the steps related to the functionality developed within their Scrum team, using the same test data throughout the flow.

5. Completion criteria

- PRL execution is considered finalized when:
 - all happy flow scenarios are executed;
 - there are no critical blockages on the main flow;
- The result supports the decision to promote to Go Live.

06. Integrated Testing Bug Reporting

Bugs will be reported according to the working mode described in 03. Reporting a Bug - QA - Confluence

The differences in reporting Integrated Testing bugs are covered in the dedicated Integrated Testing Bug page.

13. Testing Technical Tasks Workflow

These tasks will be able to be opened in case there is no activity on projects.

These tasks can be opened if needed by any role or for any activity that the Team Lead considers appropriate to be taken over.

13. Testing Technical Tasks Workflow

These tasks can be opened in case there is no activity on projects.

These tasks can be opened if needed by any role or for any activity that the Team Lead considers appropriate to be taken over.

01. Exploratory Testing Task (-)

To rewrite the text below to make it understood that it is only for Exploratory Testing. To add details about the specific name of this type of Task, how the details from this Task are filled in: Category, Story Points etc., to evaluate if a more static equivalency from hours to SP or a table with hours and SP isn't more suitable

The additional task for extended testing represents an activity created when the testing effort of a User Story or an application exceeds the initial estimate or when additional testing is necessary (e.g.: exploratory testing).

This task can be opened including for exploratory testing of an application that we want to evaluate or understand better.

The task is opened by the TL in the Scrum_TA project filling in the description field with the following information:

Testing Session / Task

Build:

Filled in by the tester.

Environment:

Specifies the environment where the testing is performed.

Testing objective

Defines the area validated within the task (functionalities, flows, integration, regression etc.).

The objective can be established at the beginning of the testing or adjusted along the way, depending on the results and discoveries during testing.

Results

Identified defects:

They are documented and linked to this task.

Relevant observations:

Unexpected behaviors, limitations or aspects that require clarification.

Identified risks:

Areas with potential functional or technical impact.

Conclusion

Establishes the stability level of the tested area:

Stable / Medium risk / High risk

When it is opened

- High functional complexity
- The need for exploratory testing (US or new application)
- Large volume of scenarios or data combinations
- Additional investigations following defects
- Unclear requirements that need additional analysis and validation
- Major changes that appeared after the initial estimation

Estimation

- It is estimated separately in Story Points (SP)
- The estimation is done by mutual agreement of the Team Leader and Tester
- It is based on complexity, impact area and risks

Other situations where additional tasks estimated in SP for QA can be opened

- Preparation or updating of complex test data
- Special environment setup or configurations
- Post-release validations (extended smoke, production checks, pilot project)
- Support investigations for critical bugs
- Impact analysis for major technical modifications

02. Update Regression\Smoke Tests Suite Task

Must be detailed with:

- Story Points and how they are added : ? SP in hours or an equivalence table
- clear output: which LXs were updated and them linked to this Task
- the modified LXs must contain comments with the made modifications

Output example

Task: Update Regression Suite – Checkout Module

Scope:

- 85 test cases analyzed
- 20 smoke tests included
- coverage last 3 releases

Activities:

- Complete test cases review
- Duplicate elimination
- Steps update according to new requirements

- Creation of test cases for new flows
- Naming convention alignment

Results:

- 85 reviewed
- 18 updated
- 7 eliminated
- 12 created
- +10% real coverage estimation

03. Execute Regression Tests on area Task

Must be detailed with:

- Story Points and how they are added : ? SP in hours or an equivalence table
- clear output: which LXs were updated and them linked to this Task
- the modified LXs must contain comments with the made modifications

Example below

B. Scope (very clearly defined)

Includes:

- Module/Feature: ex. Payments
- Release: 2.4
- Environment: QA / UAT
- Build version
- Total no. of planned test cases

Example:

- 120 test cases
- 25 smoke included
- 95 regression
- Executed on QA build 2.4.0.15
- Cross-browser: Chrome + Edge

3. Measurable metrics (mandatory)

This makes the task trackable.

Quantitative metrics:

- Total planned test cases: 120
- Executed: 120

- Passed: 104
- Failed: 10
- Blocked: 6
- Reported defects: 8
- Duplicate defects: 2
- Success rate: 86.6%

Time metrics:

- Estimate: 3 days
- Real: 2.5 days
- Average time per test case: 7 minutes

Defects metrics:

- Severity:
 - 2 Critical
 - 3 Major
 - 3 Minor
- Affected areas

4. Activities included in the Task

Clearly listed:

- Executing test cases according to approved suite
- Logging defects in Jira
- Re-test for fixed defects
- Updating status in Test Management Tool
- Final regression report

04. Business Flows Testing Documentation Task

The information can be added here Banking flows - QA - Confluence
example below

QA WIKI – Complex Multi-System Flows Functional Testing

1 Flow Overview

- Flow name: Ex: „Order → Payment → Invoice → Notification”
- Involved systems:
 - Web Application (Frontend)
 - Core API

- Payments System
- Billing System
- Email System
- Flow type: synchronous / asynchronous / mixed
- Criticality: High / Medium / Low
- QA Owner

2 Simplified Functional Architecture (QA View)

Not a complex technical diagram, but a view from the tester's perspective:

User → Web App → API → Payment Service → Billing → Notification

Purpose: for the tester to know where the data must be validated.

3 Test Users (Cross-System)

Client	Web App	qa_client_01	Without restrictions
Admin	Backoffice	qa_admin_01	Can change status
Finance	Billing System	qa_fin_01	Can validate invoice

Include:

- Permissions on each system
- If the user exists in all systems
- User ID mapping between systems (important!)
- Necessary pre-existing data (e.g.: positive balance)

4 Test Data – Correlated between Systems

A. Initial data

- Active product
- Available stock
- Valid payment method
- Active tax configuration

B. Data generated along the flow

- Order ID (Web)
- Transaction ID (Payment)
- Invoice ID (Billing)
- Notification ID

The Wiki must indicate:

- ✓ where each ID appears
- ✓ how it is verified
- ✓ in which system

5 Functional Breakdown of the Flow (Cross-System)

◆ Step 1 – Order Creation (Web)

User action:

- Selects product
- Enters data
- Confirms order

Validations:

- Total correctly calculated
- Order ID generated
- Status = „Pending Payment”
- Order saved in DB

◆ Step 2 – Payment (Payment Service)

User action:

- Enters card data
- Confirms payment

Validations:

- Correct redirect
- Transaction ID generated
- Payment Status = Success
- Order ID ↔ Transaction ID correlation

Critical point:

- What happens if the payment fails?
- What happens at timeout?

◆ Step 3 – Invoice Generation (Billing)

Trigger:

- Payment Success

Validations:

- Invoice automatically generated
- Invoice ID associated to Order ID
- Correct values (taxes, discount)
- Invoice status = „Issued”

◆ Step 4 – Notification (Email/SMS)

Validations:

- Email sent
- Correct content
- Correct attachment
- Correct Order ID association

6 Critical Synchronization Points

This is the essential section for complex flows.

- Status consistency between systems
- Processing delay
- Retry mechanism
- Event duplication
- Orphan records
- Lost asynchronous messages

7 Cross-System Functional Scenarios

Happy Flow

The entire complete flow without interventions.

Negative Scenarios

- Payment failed → Order returns to Pending
- Payment success but invoice not generated
- Email service unavailable
- Double click on confirmation

Edge Cases

- External timeout
- Manual retry
- Manual reprocessing in backoffice

8 Inter-System Validations

Order ID	Web	API	Consistency
Payment Status	Payment	Web	Synchronization
Invoice Amount	Billing	Order	Correctness
Email content	Notification	Order	Correct data

Typical Problems in Complex Flows

- Desynchronized status
- Different values between systems
- Uncorrelated IDs
- Retry that creates duplicates
- Lack of rollback

Environment Reset After Test

Important for complex flows:

- Order deletion
- Invoice cancellation
- Transaction refund
- Manual status reset

How to make it measurable as a Task

Metrics example:

- 1 complex flow documented
- 5 systems analyzed
- 12 inter-system points identified
- 8 cross-system negative scenarios
- 15 synchronization validations defined
- 6 generated data documented

05. Analysis Task

1. Defect Analysis. Examples:

Recurring Defects Analysis

2. Reproducing closed bugs from past releases
3. Testing times analysis

4. Identifying test data (hard to find) and what can be automated. Example of what Andreea did on Credit Analysis [RAP-6649] Report automation request – Identifying eligible clients for Credit Analysis testing...

can use Most used client types - QA - Confluence and Most used users - QA - Confluence

5. how PowerBI can be used, started here PowerBI Access - DWH Database - QA - Confluence or Database Testing - QA - Confluence
6. update productivity report documentation Manual Testing Productivity Report - QA - Confluence
7. Review on the information here: Most used users - QA - Confluence and Most used client types - QA - Confluence
8. to continue API Testing - Swagger - QA - Confluence
9. Review AI Test Manager prompts

06. Testing processes improvement Task

examples:

1. Creating checklists: Smoke testing, Regression etc.
2. Defining a regression strategy
3. Review what can be improved in the workflow with Test Manager
4. review what can be improved in Jira - fields, statuses, workflows on User Story, Sub-tasks, Bugs, Incidents etc.
5. review what can be improved in the productivity report
6. review what can be improved in testing efficiency: proposals etc.
7. Review Confluence workflow

07. Automation Testing activities Task

Maintenance of Auto Tests that are failed

Creating negative scenarios or edge cases starting from existing happy path auto tests

02. Guides for specific testing types

02. How to split LX type tests into smaller Test Cases using AI

Within the manual testing processes, LX type tests (large, monolithic test cases) can become hard to manage, maintain and reuse. With the help of artificial intelligence, these tests can be automatically processed to generate granular, easy to understand Test Cases that can be imported back into the test management systems.

Objective

Using AI, we want to:

- Extract the logical structure of a large test.
- Decompose it into independent test units.

- Automate formatting and generating output compatible with Jira Xray and UiPath Test Manager.

Process 1. Input: An XML file exported from Jira/Xray containing the sequence of steps (Action, Data, Expected_Result).

2. Processing: The AI analyzes the sequences, identifies logical pairs (action + verification) and generates a distinct Test Case for each. (ChatGPT recommendation)

3. Output: A structured file (e.g. .xlsx) containing:

- Unique identifier (TC-nnn)
- Automatically generated name from the action content
- Details about steps (action, data, expected result)

AI prompt example (for ChatGPT)

I have an XML file exported from Jira/Xray containing a very large manual test, consisting of many steps. I want you to transform this test into a set of smaller Test Cases, each grouping the steps in a logical and coherent way.

For each Test Case:

- Automatically detect how many steps need to be grouped together (depending on the action, context and expected result).
- Generate a descriptive and clear title that reflects the test's purpose, based on the first significant step. Do not include the entire text, just the essential information.
- Ensure the title is completely displayed in the final file (without truncation).

Generate an .XLSX file with the following structure:

- Test Case: formatted as TC-001 - [descriptive title]
- Step N Action
- Step N Data
- Step N Expected Result

Do not impose a fixed limit of steps per test. The final structure must be suitable for import into Jira Xray or UiPath Test Manager.

Bug Generation from Google Chrome Developer Tools - Network using AI

Google Chrome must be used - the user must be signed-in to Google

AI Innovation must be activated from Settings in Developer Tools (F12)

Then from the Network tab go to the element on the page with Error

And enter the prompt for generating the Bug:

"write me a detailed bug report from this network error including title, summary, description steps to reproduce, expected and actual results. write it in romanian language"

Okay, I can help you with that! Here's a detailed bug report in Romanian, based on the network request information you provided.

Raport de Eroare

Titlu: Imagine GIF Depășită Livrată în Loc de Format Optimizat și Apelată din Contextul Greșit

Sumar:

Imaginea doc_delete.gif, potențial folosită ca iconiță sau element vizual pentru ștergerea unui document, este livrată ca un fișier GIF depășit. Imaginea a fost modificată ultima dată în 2017 și nu folosește formate de imagine moderne și mai eficiente, precum WebP sau AVIF, care sunt suportate de browserul clientului. Mai mult, cererea imaginii a provenit din pagina de autentificare, dar este legată de o acțiune de document.

Descriere:

Cererea de rețea pentru imaginea doc_delete.gif rezultă în livrarea unui fișier GIF vechi și potențial ineficient. Antetul Last-Modified arată că imaginea a fost modificată ultima dată în 2017, sugerând că este depășită. În plus, deși browserul clientului acceptă formate de imagine moderne precum WebP și AVIF, serverul livrează în continuare un fișier GIF. Acest lucru rezultă într-o oportunitate ratată de optimizare a performanței și de reducere a dimensiunii fișierului. În plus, imaginea doc_delete.gif este apelată din pagina de login, dar logica ei este legată de un document, ceea ce e incorect.

Pași pentru Reproducere:

- Accesați pagina <https://librahome/users/login>.
- Deschideți Developer Tools al browserului (de obicei apăsând F12).
- Navigați la tab-ul "Rețea" (Network).
- Reîmprospătați pagina.
- Filtrați cererile căutând doc_delete.gif.
- Selectați cererea de rețea doc_delete.gif.
- Observați antetele de răspuns, în special Content-Type și Last-Modified.
- Observați tab-ul "Inițiator" și verificați lanțul.

Rezultate Așteptate:

- Imaginea doc_delete ar trebui să fie într-un format de imagine modern (WebP sau AVIF) pentru a oferi o compresie mai bună și o calitate vizuală superioară.
- Imaginea doc_delete ar trebui să fie actualizată conform ghidurilor de design curente.
- Imaginea doc_delete nu ar trebui să fie apelată din pagina de login.

Rezultate Reale:

- Content-Type este image/gif, indicând un format de imagine mai vechi, potențial mai puțin eficient.
- Data Last-Modified este Thu, 02 Nov 2017 15:48:55 GMT, arătând că imaginea este depășită.
- Lanțul din "Inițiator" arată o origine greșită.

Severitate:

Medie. Deși imaginea se încarcă, există oportunități de îmbunătățire a performanței, a designului și a logicii.

Prioritate:

Medie. Rezolvarea acestui aspect este benefică pentru performanța generală a site-ului și pentru consistența designului, dar s-ar putea să nu fie un blocaj critic.

Mediu:

- Browser: Chrome 133 (conform User-Agent)
- Sistem de Operare: Windows 10 (conform User-Agent)
- Server: Nginx

Note Adiționale:

- Luați în considerare actualizarea tuturor imaginilor din directorul /images la formate moderne.
- Revizuiți designul și relevanța imaginii doc_delete.gif.
- Verificați logica pentru a vedea de ce imaginea este apelată din pagina de autentificare.

Explanation of Romanian Terms:

- Raport de Eroare: Bug Report
- Titlu: Title
- Sumar: Summary
- Descriere: Description
- Pași pentru Reproducere: Steps to Reproduce
- Rezultate Așteptate: Expected Results
- Rezultate Reale: Actual Results
- Severitate: Severity
- Prioritate: Priority
- Mediu: Environment
- Note Adiționale: Additional Notes
- Antet: Header
- Rețea: Network
- Inițiator: Initiator
- Depășită: Outdated
- Optimizat: Optimized
- Apelată: Called
- Contextul greșit: Wrong context.

This detailed bug report in Romanian should be helpful for your team to understand and address the issues with the doc_delete.gif image. Let me know if you have any other requests!

Linking Test Manager drafts to Xray folders

After importing the Test Cases in Test Manager, they are automatically loaded into the Xray Test Repository as well.

Each main folder of the scrum team has a subfolder named "Test Manager Drafts" . In each team's folder, all the Test Cases generated with Test Manager can be found.

Below you can find the links to the Test Manager Draft subfolders assigned to each scrum team in Xray.

Scrum Team	Draft Xray Link
Scrum Alte Divizii	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20ALTE%20DIVIZII%2FDrafturi%20Test%20Manager
Scrum Carduri	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20CARDURI%2FDrafturi%20Test%20Manager
Scrum Conformitate si Antifrauda	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20CONFORMITATE%20SI%20ANTIFRAUDA%2FDrafturi%20Test%20Manager
Scrum IBK	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20IBK%2FDrafturi%20Test%20Manager
Scrum MBK	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20MBK%2FDrafturi%20Test%20Manager
Scrum Libra Wallet	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20LIBRA%20WALLET%2FDrafturi%20Test%20Manager
Scrum Digital	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20DIGITAL%2FDrafturi%20Test%20Manager
Scrum Digitalizare Fluxuri	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20DIGITALIZARE%20FLUXURI%20CLIENTI%2FDrafturi%20Test%20Manager

Scrum Team	Draft Xray Link
i Clienti	
Scrum Librapay	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20LIBRAPAY%2FDrafturi%20Test%20Manager
Scrum CP	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20CP%2FDrafturi%20Test%20Manager
Scrum Operazioni	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20OPERATIUNI%2FDrafturi%20test%20Manager
Scrum Creazione Deschidere Client	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20CREATIO%20DESCHIDERE%20CLIENT%2FDrafturi%20Test%20Manager
Scrum Decontari	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20DECONTARI%2FDrafturi%20Test%20Manager
Scrum Digital Hub	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20DIGITAL%20HUB%2FDrafturi%20Test%20Manager
Scrum Flow Credite	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20FLOW%20CREDITE%2FDrafturi%20Test%20Manager
Scrum Administrare Credite	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20ADMINISTRARE%20CREDITE%2FDrafturi%20Test%20Manager

Scrum Team	Draft Xray Link
Scrum Analiza Credit e	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20ANALIZA%20CREDITE%2FDrafturi%20Test%20Manager
Scrum Administrare a Riscurilor	https://jira.libra.bank/secure/XrayTestRepositoryAction!default.jspa?entityKey=LX&path=Scrum%20Administrare%20RISC%2FDrafturi%20Test%20Manager

UiPath Test Manager Workflow - Creatio Client Opening Pilot

To start writing scenarios and automating, user stories without written scenarios are chosen. The query below can be used

type = Story AND status in ("Ready For Testing", "In testing") AND project = SCF

Jira User Stories are synchronized with UiPath Test Manager. To find the User Story search for the requirement in UiPath by title in the Requirement section for the Requirements - Test Manager project

The workflow requires creating the tasks below in each User Story

Defining scenarios (existing task): manual tester and/or auto tester work on designing the scenarios

- UiPath Test Manager is used to generate AI test cases
- the 2 prompts are used:
 - US Validation - UAT Testing - Negative scenarios
 - US Validation - UAT Testing - positive scenarios
- all generated tests that are considered relevant are selected. Upon generation, the tests will be named “negative scenario”, “positive scenario”, “regression scenario”
- upon generation, the tests will also have labels: "negative-scenario", "positive-scenario", "regression scenario"
- manual tester and/or auto tester evaluate the AI scenarios
- where AI generated tests are incomplete (missing info, expression problems) an edit is made in UiPath Test Manager
- manual tester and auto tester evaluate if the AI scenarios cover everything needed to ensure complete testing;
- in case extra test scenarios are needed, scenario generation is forced using AI with new prompts

- AI generated tests automatically appear in the LX project for the tested project (the query type = test and issue in testRepositoryFolderTests("LX", 'Scrum CREATIO DESCHIDERE CLIENT', "true") and reporter = jira.automation ORDER BY createdAt desc can be used)
- The Test Case is created with Test Type = Manual ; it will be modified later by Test Automation

After being generated and synced to Jira, the LXs must be edited for the following information:

- reporter\assignee fields - if assignee cannot be edited: a solution to know which LXs you worked on is to make the transition to status Defining -> Finalized Scenarios; then a Jira query can be used to track type = test and issue in testRepositoryFolderTests("LX", 'Scrum CREATIO DESCHIDERE CLIENT', "true") and reporter = jira.automation and status changed by currentUser()
- added labels (project specific)
- create links to the tested User Story
- added Priority according to the orientation guide 04. LX Priorities (*)
- change status to the corresponding one:
 - if scenarios are considered complete status Defining -> Finalized Scenarios\Live
 - if scenarios are considered incomplete or there were creation mistakes\errors status Defining-> Archived

Attention! AI generated scenarios must also be tested (manual or auto);

The scenarios are sent for validation to the PO as before via email

The LXs will be listed in the email

UAT Testing (existing task) – manual tester * scenarios are tested manually as in the as-is workflow: test case execution, manual testing etc.

Developing Auto Tests new task – auto tester * auto tests development starts at the same time as UAT testing. To be able to identify LXs generated with AI the Jira filter can be used: type = test and issue in testRepositoryFolderTests("LX", 'Scrum CREATIO DESCHIDERE CLIENT', "true") and reporter=jira.automation and status ="Scenarii finalizate"

- LXs will be automated in priority order;
 - Priority = Urgent will be the first to be automated;
 - Not a priority will be the last to be automated
- Evaluate if screens\objects already exist;
- Completely new implementations require adding object repository (activity separate from framework) screens + objects
- Developed tests must be parameterized (using arguments) to be able to run on UAT and PRL) from UiPath Studio
- The 3 tests created by UiPath (AI generated) are used as a starting point; tests are imported into UiPath Studio
- “generate with autopilot” functionality is used

- Search by ID from Test Manager
- if the generated auto test runs ok at Run and is considered complete publish is done in Orchestrator\Shared
- if the generated auto test does not run ok at Run or is considered incomplete -> manual edit is done
- when the test is considered complete publish is done in Orchestrator\Shared -> the test appears in orchestrator
- After automating the test, the LX will be edited in Test Type = Automated – The test modifies test coverage dashboard + 1 to Automated Tests

PRL Testing (existing task) – manual tester

- run from Orchestrator using Test Robot on the auto tests created for UAT testing
- Manual or auto tester from Test Manager adds the test case into a test set and then the automatic test is run from Test Manager
- Using arguments the PRL environment will be chosen

UiPath Test Manager Workflow - Digital team Pilot

To start writing scenarios and automating, user stories without written scenarios are chosen.

Jira User Stories are synchronized with UiPath Test Manager. To find the User Story search for the requirement in UiPath by title in the Requirement section for the Requirements - Test Manager project

The workflow requires creating the tasks below in each User Story

Defining scenarios (existing task): manual tester and/or auto tester work on designing the scenarios

- UiPath Test Manager is used to generate AI test cases
- the 2 prompts are used:
 - US Validation - UAT Testing - Negative scenarios
 - US Validation - UAT Testing - positive scenarios
- all generated tests that are considered relevant are selected. Upon generation, the tests will be named “negative scenario”, “positive scenario”, “regression scenario”
- upon generation, the tests will also have labels: "negative-scenario", "positive-scenario", "regression scenario"
- manual tester evaluates the AI scenarios
- where AI generated tests are incomplete (missing info, expression problems) an edit is made in UiPath Test Manager
- manual tester evaluates if the AI scenarios cover everything needed to ensure complete testing;
- in case extra test scenarios are needed, scenario generation is forced using AI with new prompts
- AI generated tests automatically appear in the LX project for the tested project (the query type = test and issue in testRepositoryFolderTests("LX", 'Scrum Digital', "true") and reporter = jira.automation ORDER BY createdAt desc can be used)

- The Test Case is created with Test Type = Manual

After being generated and synced to Jira, the LXs must be edited for the following information:

- reporter\assignee fields - if assignee cannot be edited: a solution to know which LXs you worked on is to make the transition to status Defining -> Finalized Scenarios; then a Jira query can be used to track type = test and issue in testRepositoryFolderTests("LX", 'Scrum CREATIO DESCHIDERE CLIENT', "true") and reporter = jira.automation and status changed by currentUser()
- added labels (project specific)
- create links to the tested User Story
- added Priority according to the orientation guide 04. LX Priorities (*)
- change status to the corresponding one:
 - if scenarios are considered complete status Defining -> Finalized Scenarios
 - if scenarios are considered incomplete or there were creation mistakes\errors status Defining-> Archived

Attention! AI generated scenarios must also be tested

The scenarios are sent for validation to the PO as before via email

The LXs will be listed in the email

Useful Prompts

UiPath Test Manager

UAT Testing Prompt

Generate test cases that test the requirement end-to-end.

There should be at least 1 positive test case marked in the title as "scenariu pozitiv"

There should be at least 1 negative test case marked in the the title as "scenariu negativ"

There should be at least 1 regression test case marked in the title as "scenariu regresie"

Generate only 3 test cases that respect the previous conditions

All Test Case steps need to have expected behavior

The case cases have to be in Romanian language

Proposal for Manual-Automation Testing workflow in teams

Preconditions

- UiPath Test Manager license must exist (1 license at organization\tenant level)
- available Automation Developer Cloud licenses must exist for testers who will generate LXs using AI (1 license per user)
- Planview integration (described here Planview\Tasktop Configuration) must be done for the team's SCRUM Project ↔ Test Manager Project (replicated from the integration done for Creatio Client Opening) - planview requires license - 1 license per bank. Minimum necessary described below

Requirements\User Stories Integration

Minimum necessary integration is unidirectional Jira User Story (Description field) → UiPath Test Manager (Requirements section)

Test Cases Integration

Minimum necessary integration is unidirectional UiPath Test Manager Test Case → Jira\Xray Test Case (LX)

If automated tests already exist : Initial recommendations for Test Automation * evaluating already automated tests for the team

- if they exist : mapping to existing LXs : - see last point
- if they don't exist → they must be created; they can be created either manually as a "placeholder" or generated with AI using Test Manager (described below)
- LXs must be set with priority according to 04. LX Priorities (*)
- LXs must be set with Test Type = Automated
- LXs must be set status = Finalized Scenarios
- the automated test (the script) should be linked with the LX
- in case there is an application to make the link : Git, UiPath Test Manager? - not implemented yet
- in case there is no method to make a link a comment can be put on the LX with the repository path for the automated test
- Following the evaluation, prioritization and settings on the LXs above, the tests will appear in the test automation coverage Dashboard: Test Automation Coverage

To start writing scenarios and automating them, user stories without written scenarios are chosen: it is recommended to choose User Stories from the current sprint

Jira User Stories are synchronized with UiPath Test Manager. To find the User Story search for the requirement in UiPath by title in the Requirement section for the project configured in Test Manager

The workflow requires creating the tasks below in each User Story

Defining scenarios (existing sub-task): manual tester and/or auto tester work (together) on designing the scenarios

- UiPath Test Manager is used to generate AI test cases
- the “UAT Testing Prompt” prompt is used - must be created at project level; the prompt is described here Useful Prompts
- only the first 3 generated test cases are selected which will be named “negative scenario”, “positive scenario”, “regression scenario”
- manual tester and/or auto tester evaluate the AI scenarios
- where AI generated tests are incomplete (missing info, expression problems) an edit is made in UiPath Test Manager

- manual tester and auto tester evaluate if the AI scenarios cover everything needed to ensure complete testing;
- in case extra test scenarios are needed, scenario generation is forced using AI with new prompts
- AI generated tests automatically appear in the LX project for the tested project (the query type = test and issue in testRepositoryFolderTests("LX", 'Team Test Repository Path', "true") and reporter = jira.automation ORDER BY createdAt desc can be used)
- The Test Case is created with Test Type = Manual ; it will be modified later by Test Automation

After being generated and synced to Jira, the LXs must be manually edited with the following information:

- the assignee field - the LXs have assignee = unassigned when they are created by the Planview sync between UiPath Test Manager and Jira
- added labels (project specific)
- create links to the tested User Story
- added Priority according to the orientation guide 04. LX Priorities (*)
- change status to the corresponding one:
 - if scenarios are considered complete status Defining -> Finalized Scenarios\Live
 - if scenarios are considered incomplete or there were creation mistakes/errors status Defining-> Archived

Attention! AI generated scenarios must also be tested (manual or auto);

The scenarios are sent for validation to the PO\Application Owner as before via email . The LXs will be listed in the email to the PO\Application Owner

UAT Testing (existing sub-task) – manual tester * scenarios are tested manually as in the as-is workflow: test case execution, manual testing etc described here 01. Manual Testing Workflow

Developing Auto Tests (new sub-task) – auto tester Recommendations for workflow

- auto tests development starts at the same time as UAT testing (when deployment is ready and testing can begin). To be able to identify LXs generated with AI the Jira filter can be used: type = test and issue in testRepositoryFolderTests("LX", 'Team Test Repository', "true") and reporter=jira.automation and status ="Scenarii finalizate" Order by Priority desc
- LXs will be automated in priority order;
 - Priority = Urgent will be the first to be automated;
 - Not a priority will be the last to be automated
- Evaluate if screens\objects already exist;
- Completely new implementations require adding object repository (activity separate from framework) screens + objects
- Developed tests must be parameterized to be able to run on UAT and PRL
- The 3 tests created by UiPath (AI generated) are used as a starting point;

- Ideally the test should be pushed to a central test repository to be able to be run remotely (on another machine than the local one)
- After automating the test, the LX will be edited in Test Type = Automated – The test modifies test coverage dashboard + 1 to Automated Tests

PRL Testing (existing sub-task) – manual tester or auto tester

- verify if LXs were automated
- LXs that were automated
 - if manual tester has access to run automated LX → runs the auto test on PRL environment
 - if manual tester does not have access to run automated LX → auto tester runs the auto test on PRL environment
- LXs that were not automated
 - Manual tester manually executes tests in the PRL environment

API Testing

API Testing - Postman

- How to install Postman
- How to import Postman API collections
- How to do auth
- How to run an endpoint
- How to modify the body
- How to test responses 200, 400, 500 etc
- How to interpret the result
- What is Swagger?

How to do auth

To perform authentication in Postman we will use 2 methods, such as: Basic Auth and Bearer Token.

- **Basic Auth**

→Click on the “+” tab to create a new request. From the dropdown list select the POST request type. Subsequently, the API URL to which we want to connect + the endpoint will be entered.

→Go to the Authorization tab from the middle section. At Auth Type, select “Basic Auth” and enter Username and Password.

→In the Body section we will add the following keys, which we can observe in the image below:

→And in the Script section we will add the following script:

→Click on the Send button and we can observe the status 200 – OK, and in the Body section the access token was generated.

- **Bearer Token**

How to import Postman API collections

Postman collections are imported through several methods as follows:

- Click on “Import”, then add the file you want to import with drag&drop, and the imported collection will be found on the left side.
- Click on “Import”, subsequently click on files or folders and add the collection you want to import.

How to install Postman

→Access the link <https://www.postman.com/downloads/> and click on Windows 64-bit.

→Subsequently, fill in the fields and create the account.

How to interpret the result

Interpreting the results will be done according to the table below:

HTTP Response	Status	Description
200	OK	The request was fulfilled successfully
201	Created	The request has been fulfilled, resulting in the creation of a new resource
204	No Content	The request was performed but no content is returned
302	Found	Redirect
400	Bad request	Similar to 403 Forbidden, but specifically for use when authentication is required and has failed or has not yet been provided
401	Unauthorized	Similar to 403 Forbidden, but specifically for use when authentication is required and has failed or has not yet been provided
403	Forbidden	The request was valid, but the server is refusing action. The user might not have the necessary permissions for a resource
404	Not found	The requested resource could not be found but may be available in the future
405	Invalid method	The method is not allowed on that specific endpoint

HTTP Response	Status	Description
408	Request timeout	The request timed out
415	Unsupported media type	The media type supplied is unsupported
429	Too many requests	The TPP has exceeded the number of requests allowed
500	Internal server error	An error happened
503	Service unavailable	The server is currently unavailable

How to modify the body

How to run an endpoint

Click on the “+” tab to create a new request. From the dropdown list select the GET/POST/PUT/DELETE request type. Enter the API URL to which we want to connect + the endpoint and press SAVE.

In the image below, the endpoint is /token:

How to test responses 200, 400, 500 etc

Swagger Documentation

Swagger is a set of open-source tools for designing, documenting, and consuming RESTful APIs, based on the OpenAPI specification. Its main components include:

- **Interactive Documentation:** Provides a user interface that allows developers to explore APIs and make requests directly from the documentation.
- **Swagger Editor:** An online editor for writing and validating OpenAPI specifications.
- **Swagger Codegen:** Automatic generation of client SDKs and server stubs in various programming languages.

For more information, you can access the 2 links below:

ibk-test.libra.bank/IBK_WEB_API_DEV2/

https://ibk-test.libra.bank/WEBAPI_IMBK_13287//swagger/ui/index#/

API Testing - Swagger

Windows Game Bar screen\video capture

What is Windows Game Bar

Windows Game Bar (Xbox Game Bar) is a built-in application in Windows 10 and 11, which allows:

- Video recording of the screen (with or without sound);
- Screenshots;
- Monitoring PC performance;

You can record any application (browser, desktop application, testing tool, etc.)

How to open Game Bar

Press the keys:

Windows + G

If the current application allows recording, the "Game Bar" interface appears over the current screen.

How to start a video recording

◆ Method 1: From the Game Bar interface

1. Open the application you want to record (e.g.: banking application, browser, Word, etc.).
2. Press Windows + G to open the Game Bar.
3. In the Capture panel (icon with a video camera ) , press:
 -  Start Recording (the small white circle)
 - During recording, a small floating bar appears with the duration and the  Stop Recording button.
4. When you are done, press Stop.

You will see a "Game clip recorded" notification in the top right corner.

Method 2: Quick shortcut (without opening the bar)

If you want to record immediately:

Windows + Alt + R

- Starts / stops recording directly, without opening the Game Bar.
- It is the most used shortcut in testing or reporting bugs.

Recording with or without sound

Before starting:

1. Open Windows + G.
2. In the Capture panel, check the microphone icon  :
 - Active = will record the microphone sound as well.
 - Inactive = will record only the screen image.

You can enable/disable the microphone during recording as well (from the small control bar).

Where the video files are saved

By default, recordings are saved automatically in:

C:\Users\<username>\Videos\Captures

You can quickly access:

- Open File Explorer (Windows + E)
- Navigate to Videos → Captures,

or type directly in the address bar:

%userprofile%\Videos\Captures

Files have the .mp4 extension and are named automatically (e.g.: AppName - Date - Time.mp4).

Verifying video captures

After stopping the recording:

- A notification appears in the top right corner.

👉 Click on it → the video clip opens.

- You can also view all clips by:

Windows + G → Capture → See my captures

Additional options (duration, quality, audio)

You can adjust the recording settings:

1. Go to Settings → Gaming → Captures.
2. You can set:
 - Captures save location;
 - Video quality (Standard / High);
 - Frame rate (30 FPS / 60 FPS);
 - Audio capture: microphone + system sound or just image;
 - Maximum recording duration (up to 4 hours).

Important limitations

⚠ Game Bar cannot record the entire screen or an empty desktop.

It records only active windows (applications).

⚠ Does not work in some system applications (e.g.: File Explorer, Task Manager).

⚠ Recording is limited to a single active window – if you change the application, it may stop recording.

Useful shortcuts

Action	Shortcut	Description
Open Game Bar	Windows + G	Opens the main panel
Start/Stop recording	Windows + Alt + R	Starts or stops the video
Turn microphone on/off	Windows + Alt + M	Records audio or not
Take a screenshot	Windows + Alt + PrtScn	Captures the current image
Open Captures folder	%userprofile%\Videos\Captures	Video files location

Recommendations for testers / QA

- ✓ Use Windows + Alt + R when you reproduce a visual or behavioral defect.
- ✓ Add the .mp4 file directly into JIRA / SharePoint as visual evidence.
- ✓ Name the file before upload:

[TestID]_[Feature]_[DefectID].mp4

- ✓ You can use the clips for UAT evidence or defect reproduction steps.

Quick Summary

1. Open the test application —
2. Start Game Bar Windows + G
3. Start Recording  or Windows + Alt + R
4. Stop Recording  or Windows + Alt + R
5. Open the video file %userprofile%\Videos\Captures

03. Test Manager and Jira integration from Plan View

Description of the configuration and integration process of UiPath Test Manager with Jira (Xray), using the Plan View platform for the automatic synchronization of requirements and tests between the two systems.

Login and accessing Test Manager

Users connect to the UiPath Cloud platform and access the Test Manager application from the main menu. This represents the starting point for creating and managing testing projects.

Creating folders for projects

For each Jira project, a dedicated folder is created in Test Manager, having the same name to ensure correspondence between systems. Access rights for users and groups are defined and custom fields necessary for synchronization are configured (e.g. Link, Status, Label, Test Run Status).

Test Manager integration with Jira in Plan View

Integration is performed through the Plan View platform, in the Integration section, and involves configuring connections between Jira projects and related Test Manager folders.

Two main types of integration are managed:

- **Test Manager Requirements** – syncing requirements from Jira to Test Manager.
- **Test Manager Test Cases** – syncing tests generated in Test Manager to Jira/Xray.

Test Manager Requirements

This section allows the automatic import of requirements defined in Jira into the Test Manager structure.

By configuring routes (Artifact Routing), each Jira project is associated with the corresponding folder in Test Manager, allowing the automatic update of requirements.

Test Manager Test Cases

In this stage, tests created or generated in Test Manager are automatically exported to Jira, in dedicated Xray subfolders (Test Manager Drafts).

The bidirectional integration ensures full traceability between requirements and test scenarios.

Final Result

After complete configuration, each project has an active link between Jira and UiPath Test Manager, allowing:

- full visibility over requirements and tests,
- automatic update of statuses,
- unitary management of the testing process.

Synchronized projects

Jira Project	Test Manager Folder
SCRUM Administrarea Riscurilor	Scrum Administrarea Riscurilor
SCRUM LibraPay	SCRUM LibraPay
SCRUM_Administrare credite	Scrum Administrare Credite
Scrum_Alte_Divizii	Scrum Alte Divizii
SCRUM_Analiza Credite	Scrum Analiza Credite
SCRUM_Carduri	Scrum Carduri

Jira Project	Test Manager Folder
SCRUM_Conformitate	Scrum Conformitate
Scrum_CP	Scrum CP
SCRUM_Decontari	Scrum Decontari
SCRUM_Digital_HUB	Scrum Digital Hub
Scrum_Digitalizare_Fluxuri_Clienti	Scrum Digitalizare Fluxuri Clienti
SCRUM_LibraWallet	Scrum LibraWallet
Scrum_MBK	Scrum MBK
SCRUM_Operatiuni	Scrum Operatiuni
Scrum_IBK	Scrum_IBK
SCRUM Flow	CreditFlow
SCRUM_Digital	Scrum Digital
SCRUM_CRM	Scrum CRM
Proiecte Waterfall	Proiecte Waterfall
Scrum_Creatio_Flows	Scrum Creatio Deschidere client

1. Test Manager-Uipath

Login to UiPath Test Manager

1. Access the UiPath Cloud platform: <https://cloud.uipath.com>
2. Authentication: Enter one of the created users for which we already have licenses, the password and then Sign In.
3. Navigate to Test Manager: From the main menu select Test Manager.

Creating folders for projects

1. After accessing the Test Manager Submenu, page navigation takes you straight to the "Projects" menu
2. Click on "Create new project" and fill in the project name - as much as possible it should be the same name as the project name in Jira. The prefix fills automatically with the first letters of each word (if only two letters are filled in it is mandatory to manually add one more letter/digit)
3. Click "Create" and check the folder's appearance in the list.
4. Open the folder and access the "Project Settings" - "Manage Access" option and grant rights to all users using the "Assign users/groups" option
5. Add the "Test Manager QM" Group - then click the Add button in the bottom right corner.
6. Also in "Project Settings" access "Custom field definition" - adding settings in this page is done to receive on each "Requirement" and "Test Case" the information created in Jira. Ex: Link for US and LX, Status, Label and Test Run Status. Adding settings is done manually, step by step and can be copied from any other already created folder.

I also attached an example of how they can be added.

Settings for another already created folder:

After finalizing all changes on the folder, we must perform the integration of Test Manager with Jira from Plan View.

After the synchronization is performed, all requirements from Jira that do not have Live, Postponed and Ready for Release status will appear in the newly created folder.

2. Jira Integration from Plan View

Test manager integration with Jira is performed from Plan View

<https://planview.libra.bank:8443/?continue#/home>

Login data:

Access Home Button: Integration

Modifications for integration are performed only in: Test Manager Requirements and Test Manager Test Case

Modifications are performed on both sections:

1. Test Manager Requirements: requirements written in Jira are imported into Test manager.

- **UiPath Jira Story Collection -Manage Projects** In Available Projects we find the Scrum projects/teams that have not been added for integration and in Project Collection those that have already been added.

Select from Available Projects the project/Scrum Team you want to add in Project Collection by accessing the Selected > button, Done.

After the modification was performed, the Save button is accessed.

- **Uipath Test Manager - Manage Projects**

In Available Projects we find the folders that have not been added for integration and in Project Collection those that have already been added.

Select from Available Projects the newly created folder for adding in Project Collection by accessing the Selected > button, Done.

After the modification was performed, the Save button is accessed.

After the Scrum projects/teams and the new created folders have been added we can start the integration between Jira and Test Manager on the Test Manager Requirements section by selecting the Artifact Routing option.

After selecting the Artifact Routing option, the list below is displayed with the projects that already have the integration performed - The projects from Jira with the folders created for each project in test manager.

In Uipath Jira Story Collection and Uipath Test Manager Requirement Collection the Scrum project/team is added using: Route More Projects

- Select the Scrum project/team(Jira) that has not been added
- Select the newly created folder for the Project added from Jira

After the Scrum team and the folder created in Test Manager have been selected, access the Connect button, so the integration is finalized, then the Done button in the top right corner of the page.

After the integration in Test Manager Requirements has been performed between Jira and Test Manager for the new folder, we must verify in Test manager that the synchronization of the folder starts in test manager and in the Requirement Submenu on the newly created folder the requirements are synchronized.

2. Test Manager test Cases: Scenarios generated with test manager are imported into Jira

- **Uipath Jira Test Collection:** here the Xray Project is added, nothing is added or modified.
- **Uipath Test Manager Manage Projects -Manage Projects**

In Available Projects we find the folders that have not been added for integration and in Project Collection those that have already been added.

Select from Available Projects the newly created folder for adding in Project Collection by accessing the Selected > button, Done.

After the modification was performed, the Save button is accessed.

After the newly created folders have been added we can start the integration between Jira and Test Manager on the Test Manager Test Case section by selecting the Artifact Routing option.

After selecting the Artifact Routing option, the list below is displayed with the Links from Xray for each folder individually The newly created folder in Test Manager that already has the integration performed.

In Xray - Xray Test Repository each Scrum already has a folder created / or a new one is created. Each folder created for Scrum Projects/Teams must obligatorily have a Sub folder named "Test Manager Drafts".

In Uipath Test Collection and Uipath Test Manager Test Case Collection the Jira subfolders and the Test manager Folders are added using: Route More Projects

- Select the Xray subfolder that has currently not been added
- Select the folder from Test Manager Created for the project on which we will generate the scenarios.

Then save from the top of the page using the "Save" button.

After the integration in Test Manager Test Case has been performed between Jira and Test Manager for the new folder, we can verify in Test manager. On the Folder where the Requirements and Test Case integration was performed, In Test Manager select the Requirements Option, select any requirement for which we generate a Test Case.

After the Tests have been generated, we select a single scenario just to ensure that the synchronization is performed correctly and appears in Xray in the dedicated folder.